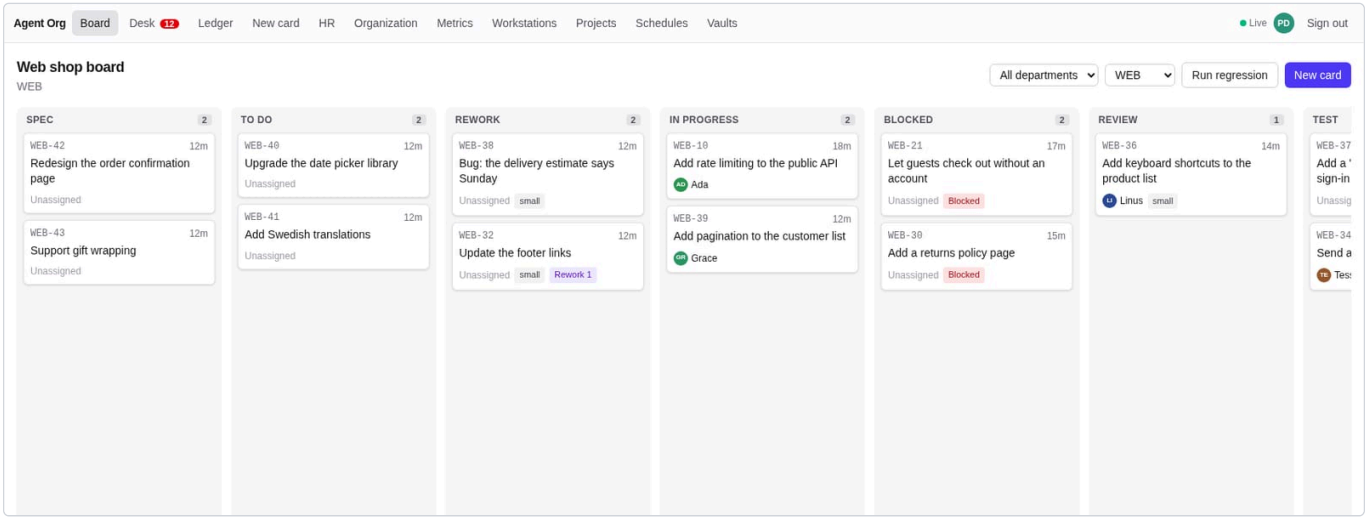
AGENT ORG MANAGER

Agent Org Manager

Run an organization of AI coding agents from one board

User guide & handover for Pieter

1 Create a card · 2 Builder builds · 3 Reviewer reviews · 4 Tester tests · 5 BA finalizes · 6 Questions reach your desk · 7 HR board tunes the team



Date	**4 October 2026**
Version	Guide 1.0, written for Agent Org Manager 1.0 (milestones M1 to M10, security review fixes, real-Claude sandbox fixes, Windows/WSL2 default)
Repository / branch	`exinarium/agent-org-manager`, branch `main`
Prepared for	Pieter (owner). This is your final check: read it instead of clicking through the app.
Data shown	Only fake demo data (`@demo.local`, `example.com`). No real Claude sessions or external posts appear.

Contents

1 At a glance 4

2 Roles, users and access 5

2.1 Administrators and members 5

2.2 Organization owner and additional managers 5

2.3 What a manager may do on a shared project 5

2.4 Agent roles and seniority 6

2.5 The card decision policy 6

3 Walkthroughs: daily work 7

3.1 Signing in and invitations 7

3.2 The board and every card state 9

3.3 Creating and specifying a card 13

3.4 Card detail 18

3.5 The decision desk and the ledger 25

3.6 Chat with agents 32

3.7 The HR board 34

3.8 Metrics 41

4 Intake, outbox and external systems 44

4.1 Intake from a ticket 44

4.2 Tickets that match no project 46

4.3 The outbox: updates back to a ticket 47

4.4 Edit, reject, failed posts and blocked drafts 48

4.5 Guardrails and credentials 51

5 Testing, review and integration 52

5.1 Risk tiers and review 52

5.2 The tester loop and evidence 52

5.3 Bug cards and waiting on a bug 53

5.4 Rebase conflicts 55

5.5 Finalize and completion actions 55

5.6 Large merges need your approval 56

5.7 Full suite and regression runs 57

5.8 The tester bottleneck alert and environment check 57

6 Research, feeds and schedules 59

6.1 Research projects: question to published note 59

6.2 Citations and the fact check 59

6.3 Feed projects and digests 61

6.4 Rating digests and taste.md 64

6.5 Schedules and Run now 65

6.6 The overflow rule 66

7 Projects, vaults, DevOps and sharing 67

7.1 Projects 67

7.2 Project settings and repositories 68

7.3 Vaults 69

7.4 Sharing a project with another manager 70

7.5 Removing a member 71

7.6 The DevOps engineer 72

8 Organization designer 74

8.1 Roles 74

8.2 Extending built-in roles 75

8.3 Workflows and the graph 75
8.4 The editor and validation 76
8.5 Departments 78
8.6 Designing with the BA and approving on the desk 79
8.7 For additional managers on a shared project 80

9 Administration and on a phone 81
9.1 The Admin page 81
9.2 Deactivating and unlinking 82
9.3 On a phone (390 px) 82

10 Workstations and the runner 85
10.1 The Workstations page 85
10.2 Pairing a workstation 86
10.3 Windows: WSL2 is the default 87
10.4 macOS and Linux 87
10.5 Native Windows (unsandboxed opt-in) 88
10.6 What a sandboxed agent can and cannot touch 88
10.7 The aom command inside a shift 89
10.8 toolchainPaths and Claude credentials 90
10.9 The unsandboxed warning everywhere 90

11 Running locally and deploying 92
11.1 Requirements and local development 92
11.2 Deploying to a VPS with Docker Compose 92
11.3 Configuration reference 93
11.4 Backups, upgrades and operations 93

12 Quality and verification 95
12.1 Test suites and counts 95
12.2 The independent security review and its fixes 95
12.3 The real Claude Code smoke run 95
12.4 How the screenshots of this guide were made 96

13 Decisions taken, known limitations 97
13.1 Decisions that shape the product 97
13.2 Known limitations 97
13.3 Things that look odd but are intended 98

14 Your to-do list before and after first real use 100
14.1 Before first real use 100
14.2 After first real use 100

15 Appendix: where everything lives, commit history 102
15.1 Where everything lives 102
15.2 Commit history by milestone 103

How to read this guide

Chapters 1, 13 and 14 are the short version: what was built, what was decided and what you still need to do. Chapters 3 to 9 show every screen of the running app with the rules behind it, so you can review the behavior without opening the app. Chapters 10 to 12 are the technical handover. Purple boxes are business rules, amber boxes are things to watch, red boxes are actions for you, green boxes are verified results.

About the screenshots

Every screenshot was taken from the real app running locally with a fake demo organization (`@demo.local`, `example.com`). The agents in the demo run on the mock runtime, a scripted stand-in for Claude Code that costs no tokens and never touches a real service, so agent replies, shift logs and handoff notes in the pictures read "[mock] ..." or "mock build round 1". Real agents write real work. No real Claude session, GitHub or Jira was used.

1 At a glance

Agent Org Manager is a self-hosted website for running an organization of AI coding agents. You create projects and cards. Builder, reviewer, tester and other agents claim cards one at a time, work in git worktrees on your own workstations through a small runner program, and hand the work on through a board. Everything that needs a human reaches one place, the decision desk.

1 Write a card: context, requirements, acceptance criteria, test plan

2 A builder claims it exclusively and builds on a branch

3 A senior reviewer approves or bounces it with a score

4 A tester runs the plan and records evidence

5 The BA finalizes: pull request, push or merge

6 Anything unclear lands on your decision desk

What you get and how it works

Topic	What you get	How it works
The board	One board per project: who has what, how old it is, what is blocked	Columns come from the workflow. A card is claimed by one agent at a time; nobody waits on another agent
Decisions	A single desk for questions, approvals, alerts and ratings, most urgent first	Questions pass a senior peer and the BA first. Your answer goes into the ledger and into every later shift
Memory	A ledger of decisions, requirements and standing orders	Searchable, shared with agents; standing orders are loaded into every agent's next shift
Quality	Review with scores, a tester with evidence, bug cards, a full suite after a merge	Small changes need one review, large ones two senior reviews and your approval to merge
External systems	Intake from Jira or GitHub, updates back to the ticket	The BA reads tickets on your workstation; nothing is posted before you approve the text
Research and feeds	Fact-checked notes with citations; scheduled digests you rate	Notes are published into your vault; ratings become explicit rules in `taste.md`
The team	A HR board with stats against the team average and one-click recommendations	Rule-based advice with the evidence behind it; applying one is an ordinary, logged change
Your own design	Custom roles, workflows and departments	Validated by the server, approved by you on the desk before they exist
Safety	Agents run in an OS sandbox on your machines	Linux and WSL2 use bubblewrap, macOS uses Seatbelt; an unsandboxed workstation is a visible, confirmed opt-in
Where it runs	One server (Docker Compose) and a runner on each workstation	The runner connects outbound; code, credentials and vault content never leave your machines

Status on 4 October 2026

277 / 277 shared contract tests

393 / 393 server unit and integration tests (real Postgres)

104 / 104 web tests

288 passed, 1 skipped runner tests

102 / 102 Playwright end-to-end tests (real UI, server, runner)

PASSED real Claude Code smoke run in the sandbox

All ten milestones of the plan are built and tested. An independent security review found issues from high to low severity; all were fixed and re-tested (chapter 12). A real Claude Code run in the sandbox found four more problems, now fixed. Not yet verified with a real Claude: macOS and native Windows (chapter 13).

What you need to do next (full list in chapter 14)

1. Read chapters 3 to 9, or sign in to the demo and click through the screens.
2. Deploy the server to a VPS with Docker Compose, set `AOM_PUBLIC_ORIGIN` and create your admin with `admin:create` (chapter 11).
3. Pair your WSL2 workstation, log Claude in for the runner, install `bubblewrap` and `socat`, and set `toolchainPaths` (chapter 10).
4. Run one real shift on a small repository and watch it on the card.
5. Then hire your first team, feed work through intake and look at the HR board after a week.

2 Roles, users and access

2.1 Administrators and members

Agent Org Manager has two global roles: **administrator** and **member**. The first administrator is created on the server with a command (see docs/DEPLOY.md, step 3). Administrators invite other users, deactivate accounts and see all workstations paired by any user. Members work only in projects they own or have accepted an invitation to.

> **What administrators can do**
>
> - Invite users and set their role (admin or member).
> - Change a user's role (admin or member) and deactivate or reactivate user accounts. You cannot deactivate yourself, and the last active administrator can be neither demoted nor deactivated.
> - Resend or revoke invitations.
> - See all workstations paired by any user (the Admin page section "All workstations") and unlink them.
> - See all users and their invitations.

Being an administrator gives no access to other users' projects, cards, agents or desk items. To work on someone's project, an administrator must be invited to it as an additional manager, the same as any member.

2.2 Organization owner and additional managers

Every user has one organization of their own. The organization owner is the owner of every project created in it. When the owner shares a project, they invite another user as an **additional manager**. The owner and an additional manager have different permissions.

Action	Organization owner only	Additional manager
Create a project in the owner's organization	**Yes**	No (a project you create is in your own organization)
Invite and remove managers to a shared project	**Yes**	No
Change project settings (repositories, workflow) and register the project's vault	**Yes**	No
Hire org-wide agents (available to all projects)	**Yes**	No
Hire project-scoped agents (this project only)	**Yes**	**Yes**
Change or fire agents	**Yes (any agent of the organization)**	**Yes (only agents this manager hired)**
See the board and the desk of the project, create cards, comment, and chat with agents scoped to the project	**Yes**	**Yes**
Add standing orders (project-wide or organization-wide)	**Yes**	No
Design custom roles, workflows and departments	**Yes**	No
Answer questions and approvals on a card	**Only if they own the card or the card's policy is `any_manager`**	**Only if they own the card or the card's policy is `any_manager`**

When the owner removes an additional manager from a shared project, the claims of that manager's agents are released (the cards go back to the queue with the last handoff attached), the agents pass to the owner and are marked as needing a new workstation, and the manager's cards in the project pass to the owner. Cards homed on the manager's workstations lose that home. If an administrator deactivates the account, the same happens in every project where the user is an additional manager, the user's workstations are unlinked and the user is signed out.

2.3 What a manager may do on a shared project

An additional manager on a shared project can see the board and the desk, create and comment on cards, and manage cards they own. They cannot change the project settings or invite and remove managers. Answering questions and approving decisions

depends on the card's **decision policy** (see section 2.5).

> **Manager access on the shared project's desk**
>
> - They see every open desk item of the project. Items about a card they own, or about a card whose policy is `any_manager`, they can answer. Other items are read-only for them.
> - Items that belong to no card (for example approvals of the project) can be answered by the project owner only. Ratings of a digest can be given by any member.
> - Only the project owner can add project standing orders. Only the organization owner can add organization-wide ones.

2.4 Agent roles and seniority

The system ships with ten built-in agent roles, each with its own permissions and workflow stages. A role allows junior agents, senior agents, or both.

Role	Purpose	Seniority
Builder	Implements specified cards with unit tests in a git worktree.	Junior, senior
Reviewer	Reviews builder output against the card and the repository conventions.	Senior only
Tester	Runs the test plan and owns end-to-end tests.	Junior, senior
Technical writer	Updates documentation for finished changes.	Junior, senior
Business analyst (BA)	Specifies cards, runs intake and finalizes finished work.	Senior only
Researcher	Answers research questions with sourced findings.	Junior, senior
Fact checker	Verifies research claims against their citations before publishing.	Junior, senior
Scout	Scans sources for candidate items for a feed.	Junior, senior
Curator	Selects and summarises feed candidates into a digest.	Junior, senior
DevOps engineer	Maintains the workstation environment. One per workstation, hired automatically (you cannot hire it yourself).	Senior only

Some stages are marked senior only (in the built-in code workflow, Review is one of them). A junior agent cannot claim them, even if it holds the role. There is no cap on how many agents you hire.

2.5 The card decision policy

By default, only the card's owner can answer questions and approve decisions on that card. To let any project manager answer, set the decision policy to `any_manager`.

Policy	Meaning
`owner_only`	Only the card owner can answer questions and approve decisions on this card. This is the default.
`any_manager`	Any project manager (owner or additional managers) can answer questions and approve decisions on this card.

The decision policy is set on the card detail page, in the Details panel, in the field **Who answers its questions**. The choices are "Only the card owner" and "Any manager of the project". The card owner and the project owner can change it. It only affects desk items of that card. It does not change who can see the desk, and the project owner does not answer a card's items unless they own the card or the policy is `any_manager`.

3 Walkthroughs: daily work

This chapter shows every screen of the daily work in the order a manager meets them: signing in, the board, creating and specifying cards, and the details of each card. Each screen shows what it displays, how the buttons work, what errors say, and who sees what. The demo runs on the MOCK runtime: agent replies and shift logs are scripted ([mock] text in the screenshots); real Claude Code agents write real work.

3.1 Signing in and invitations

Every new user arrives at the **Sign in** page. Existing users enter their email and password; new users are invited by the admin.

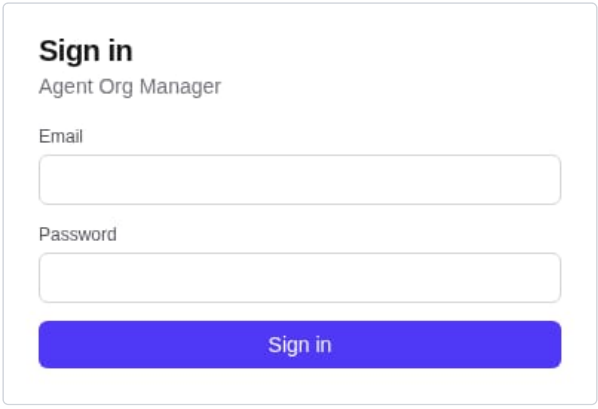


Figure 1 — The sign-in page (not signed in)

The form has two fields: `Email` and `Password`. Both are required. The button reads **Sign in**. If the email is not a valid address, the form shows `Enter a valid email address` under the field. If the password is empty, it shows `Enter your password`. If the email or password are wrong, the page shows a red message: `Incorrect email or password`.

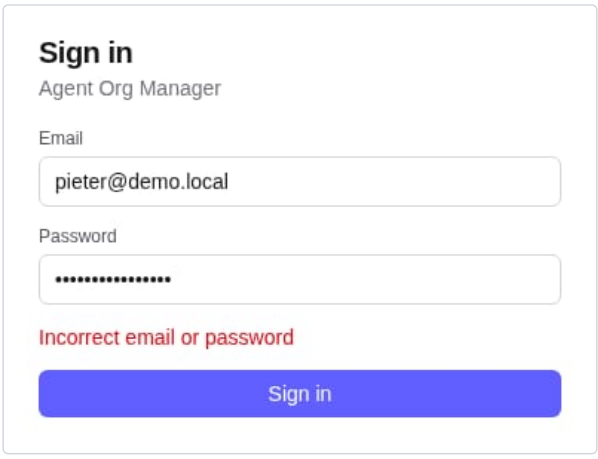


Figure 2 — After entering a wrong password for pieter@demo.local (not signed in)

When a user opens the link in the invitation email, they land on the **Accept invitation** page. The line under the heading reads "Creating an account for" followed by the invited email address; it cannot be changed. The user enters a display name, a password (12 to 256 characters), and confirms the password.

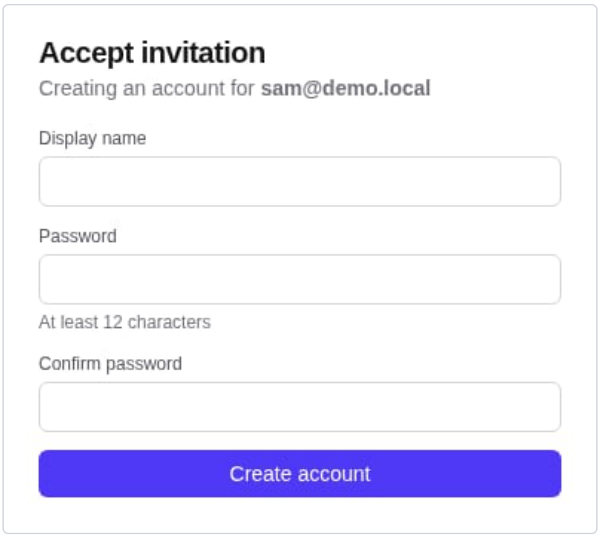


Figure 3 — Accept-invitation form for sam@demo.local (not signed in)

All three fields are required. The display name may be up to 120 characters; if it is empty the error reads `Enter your name`. The password field shows the hint "At least 12 characters". If the password is shorter than 12 or longer than 256 characters, the error reads `Password must be 12 to 256 characters`. If the two passwords differ, the error under `Confirm password` reads `Passwords do not match`. The button reads **Create account**. After the account is created you are signed in.

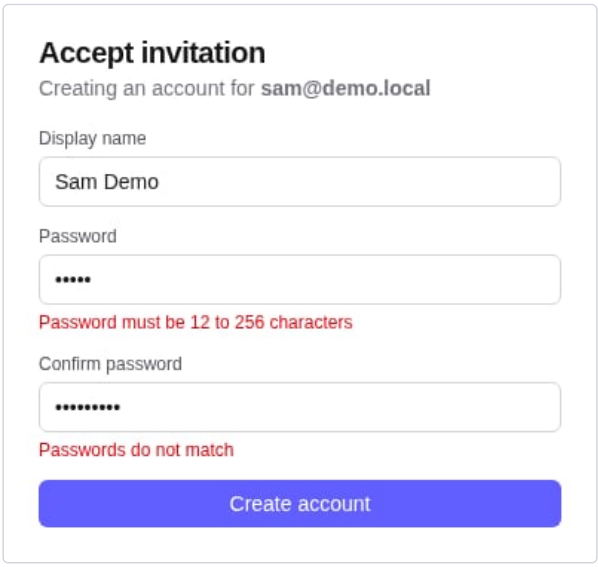


Figure 4 — Validation errors on the invitation form: a five-character password and a confirmation that differs (not signed in)

If the invitation link is invalid, expired, or already used, the page shows "Invitation unavailable" with the message "This invitation link is invalid, expired, or already used. Ask your admin for a new one." A link below reads **Go to sign in**.

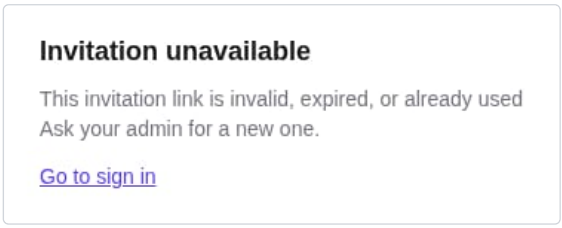


Figure 5 — Invalid invitation link (not signed in)

Top menu and navigation

After sign-in, every page shows a sticky header with the app name **Agent Org** and a horizontal menu. The menu includes:

- **Board** — The project's card board.
- **Desk** — Decisions waiting for the manager (questions, approvals, alerts). A red badge shows the open count (e.g., "12").
- **Ledger** — Requirements, decisions and standing orders that agents rely on.
- **New card** — Create a new card.
- **HR** — The HR board: your agents, hiring and firing, and their stats.

- **Organization** — Roles, workflows, and departments.
- **Metrics** — What the team delivered, where it waited on you, and where quality slipped.
- **Workstations** — Paired workstations and their DevOps requests.
- **Projects** — List of projects, with their settings.
- **Schedules** — Feed schedules.
- **Vaults** — The git repositories of markdown that your agents read.
- **Admin** (admins only) — Invite users, manage users, see all workstations.

The Desk badge is red and shows the count of open desk items (e.g., "12" means 12 items waiting). The right side of the header shows a live indicator (a green dot and "Live" if connected, or a gray dot and "Offline"), the user's avatar (initials in a circle), and a **Sign out** button.

After sign-in: where you land

After sign-in you land on the board of the project you viewed last, if you can still see it. Otherwise you land on the board of the first project in your list. Only if you have no project at all do you land on the Projects page. If you were sent to the sign-in page from another page, you return to that page.

What admins see extra

The **Admin** menu item appears only to users with the admin role. On the Admin page they invite users, change roles, deactivate and reactivate users, and see all workstations. Being an admin gives no access to other people's projects: only the owner of a project manages its members, and everyone sees only the projects they own or have accepted.

3.2 The board and every card state

The board is a kanban view of all cards in a project. Cards move through columns from left to right as they progress. The columns come from the project's workflow; the "Code" workflow (for code projects like WEB) has these columns: Spec, To do, Rework, In progress, Blocked, Review, Test, Test (waiting on bug), Docs, Finalize, and Done.

The board header shows the project name and key. On the right are a department filter (shown only when projects have departments), a project switch, the **Run regression** button, and the **New card** button. Feed projects also have **Feed** and **Run now** buttons.

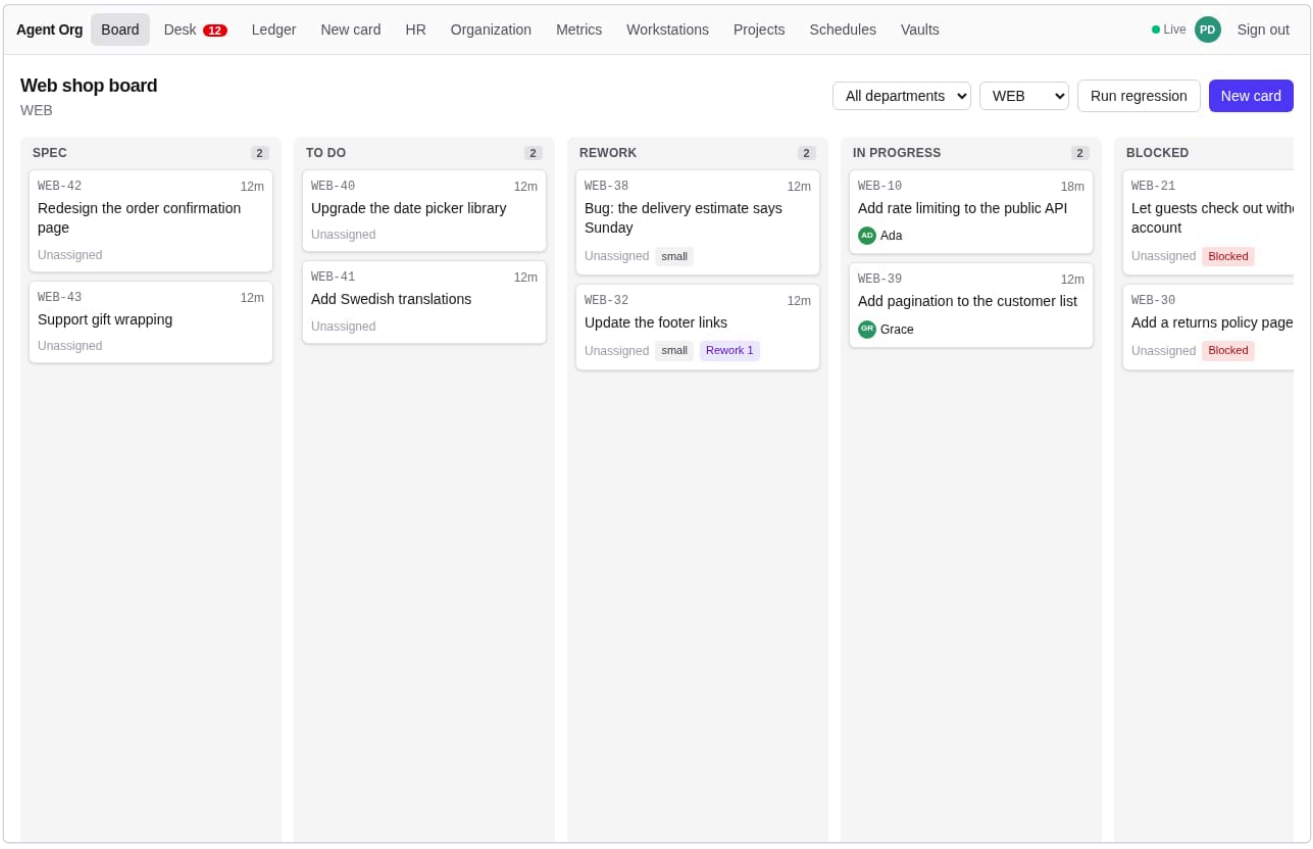


Figure 6 — Board of WEB, left side: Spec, To do, Rework, In progress, and the start of Blocked (Pieter Demo)

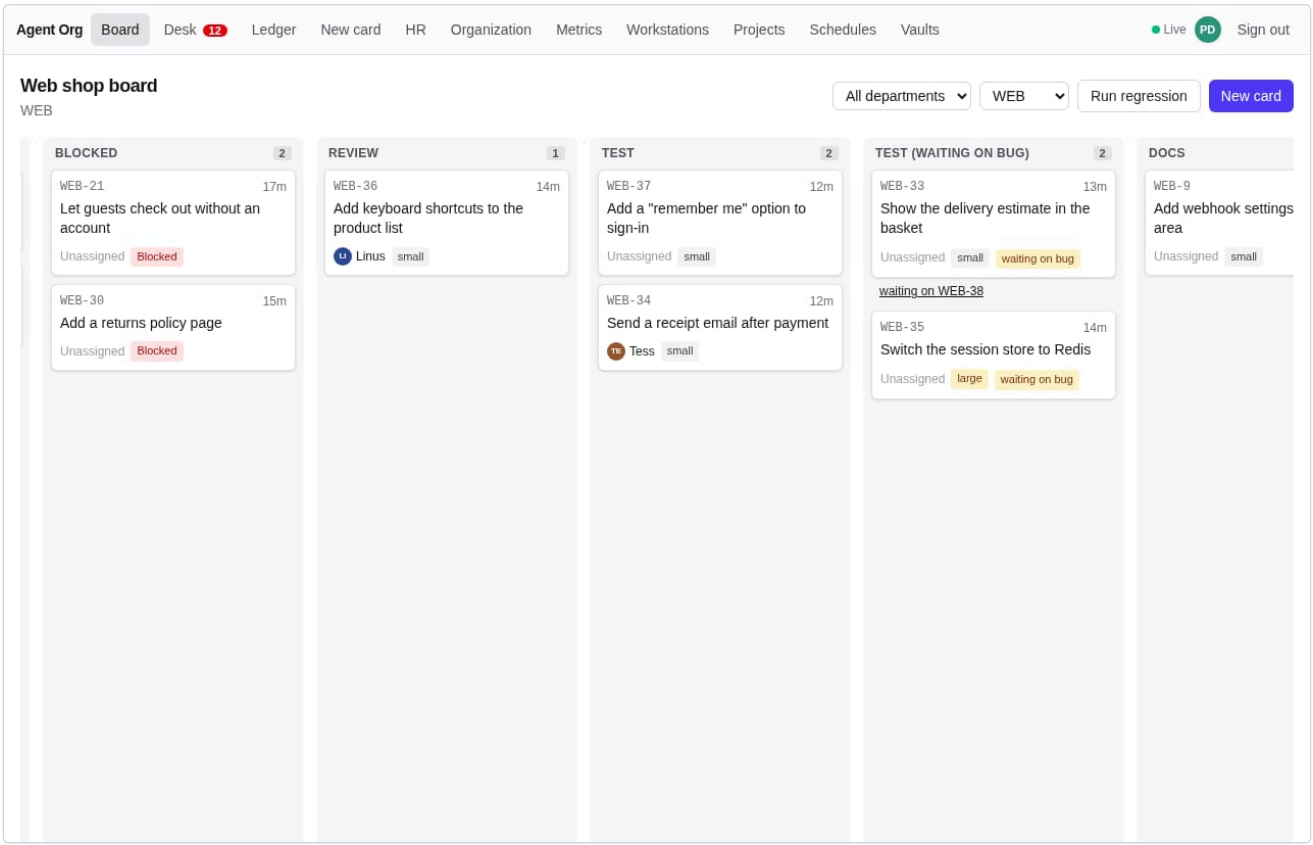


Figure 7 — Board of WEB, middle: Blocked, Review, Test, Test (waiting on bug), Docs (Pieter Demo)

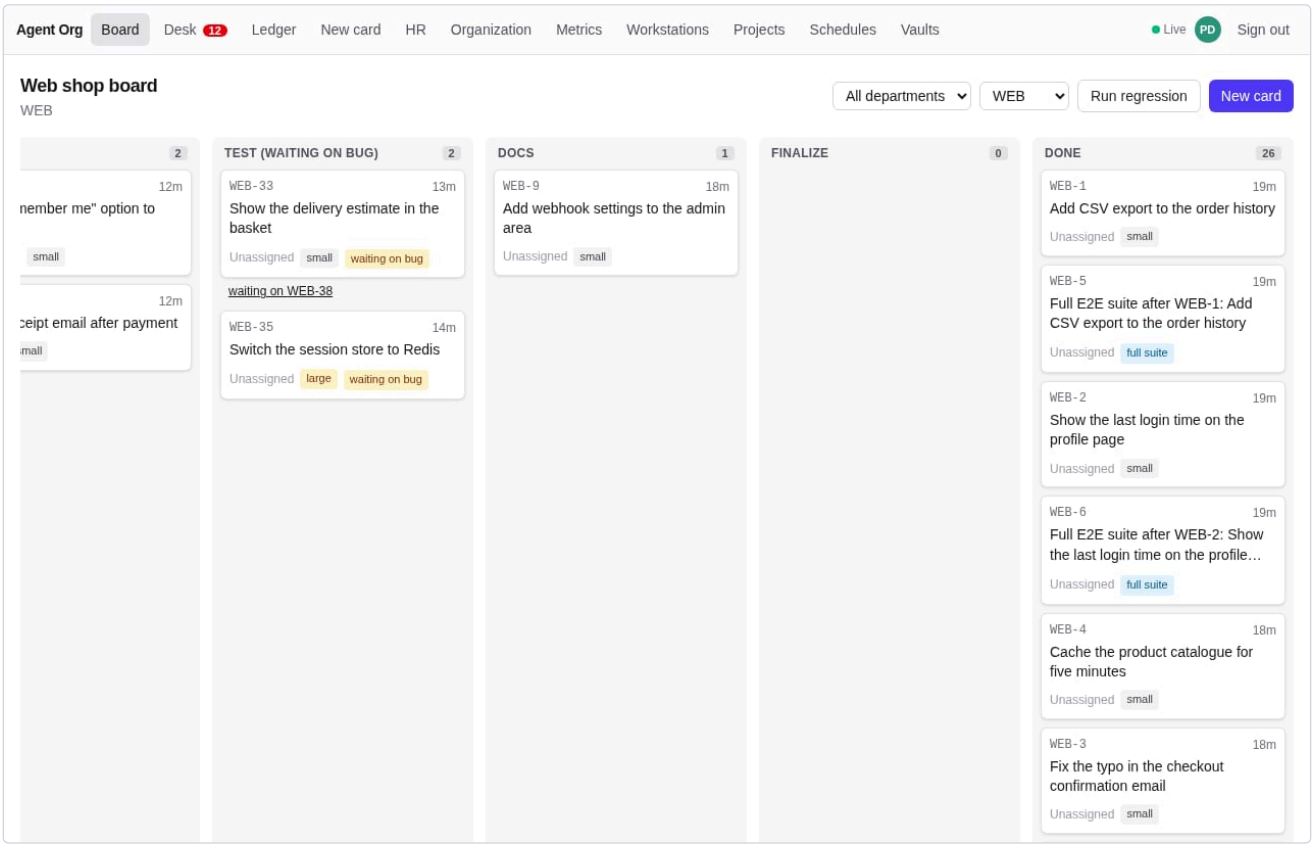


Figure 8 — Board of WEB, right side: Test (waiting on bug), Docs, Finalize (empty), Done (Pieter Demo)

Columns and workflow stages

Columns are ordered left to right in the workflow. Each column header shows the column name and the card count. Each card tile shows:

- Card ref (e.g., `WEB-42`) in the top left.
- Age (e.g., "now", "3m", "6m") in the top right.

- Title (line-clamped to 2 lines).
- Assignee (avatar + name) or "Unassigned".
- Risk tier badge if set (amber for "large", gray for "small").
- `Rework n` badge if the card has been sent back for rework n times (violet badge).
- `Blocked` badge if the card is blocked on a question (red badge).
- `waiting on bug` badge if the card is waiting for a bug fix (amber badge).
- `full suite` or `regression` badge if the card was created by a full-suite run or a regression run (blue badge).

When a card has status "waiting" and the card it waits on is known, a line below the tile reads "waiting on `WEB-38`" (a link to that card).

Card status and column mapping

The column a card appears in depends on its status and stage. The table below shows all statuses and their columns:

Card status	Column	Meaning	Who acts next
draft	Spec	The card is in the Spec stage and not yet fully specified.	Manager or BA
queued	To do	The card is queued for a builder but not claimed yet (first build).	Builder (claiming the card)
queued	Rework	A queued build card that went back to the builder after review (Rework 1+ badge), or a queued bug card.	Builder (claiming the card)
claimed	In progress	A builder is actively working on it in the build stage. Claimed cards in other stages show in the column of their stage (Review, Test, Docs, Finalize).	The agent holding the card
blocked	Blocked	A question is raised and needs an answer to proceed.	Whoever may answer the question on the desk
waiting	Test (waiting on bug)	The card waits for something else. In Test it waits for a bug card it spawned (a "waiting on WEB-xx" line shows). At Finalize it waits for the manager to approve a large merge, or for the merge itself. The board shows every waiting card in this column.	The bug card's builder, or the manager (large merge)
done	Done	The card finished its workflow.	None
cancelled	*not on board*	The card was cancelled and leaves the board.	None

The card's stage determines its column when the status is not "draft", "blocked", or "waiting". For example, a card in the build stage with status "claimed" appears in the "In progress" column. A card in the review stage appears in the "Review" column whether it is queued or claimed.

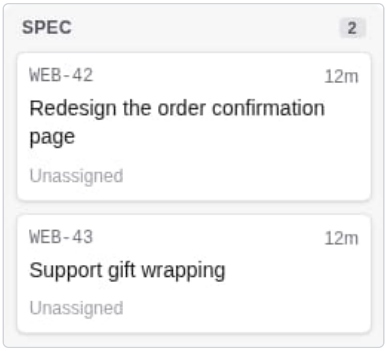


Figure 9 — Spec column: two draft cards, WEB-42 and WEB-43 (Pieter Demo)

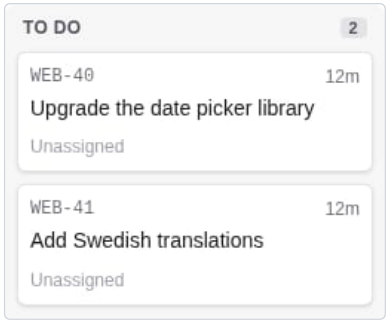


Figure 10 — To do column: WEB-40 and WEB-41, queued for a builder and not claimed yet (Pieter Demo)

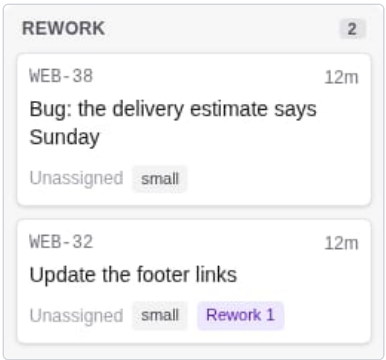


Figure 11 — Rework column: WEB-32 (Rework 1 badge) and the queued bug card WEB-38 (Pieter Demo)

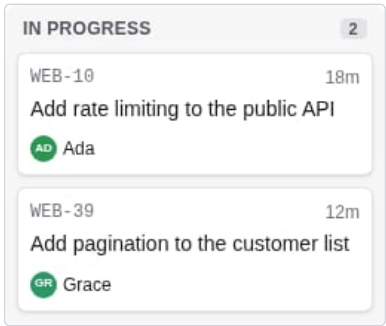


Figure 12 — In progress column: WEB-10 (Ada) and WEB-39 (Grace) (Pieter Demo)

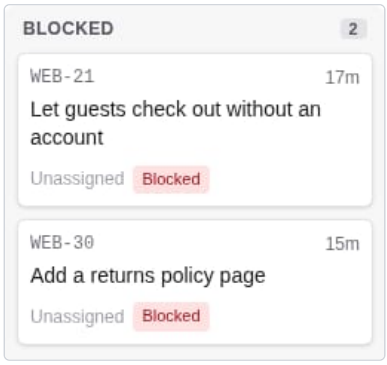


Figure 13 — Blocked column: WEB-21 and WEB-30, each with the red Blocked badge, until their question is answered (Pieter Demo)

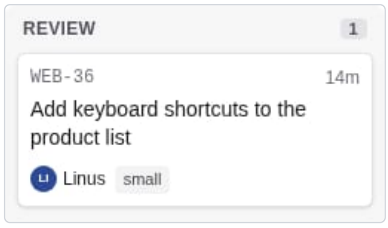


Figure 14 — Review column: WEB-36, being reviewed by Linus (Pieter Demo)

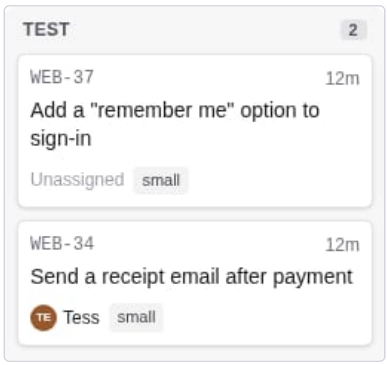


Figure 15 — Test column: WEB-34, being tested by Tess, and WEB-37, not yet claimed (Pieter Demo)

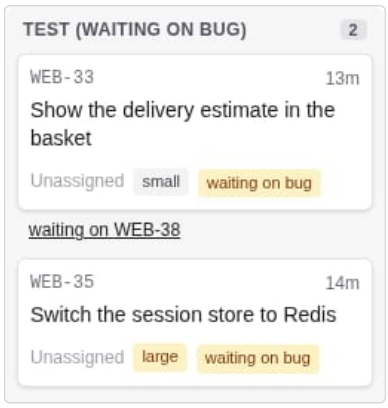


Figure 16 — Test (waiting on bug) column: WEB-33 waits on its bug card WEB-38 (link under the tile). WEB-35 (large) carries the same amber "waiting on bug" badge, but it waits for the manager's approval at Finalize: the board shows every card with status "waiting" here (Pieter Demo)

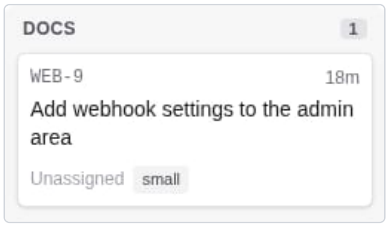


Figure 17 — Docs column: WEB-9, in the Docs stage with no one assigned (Pieter Demo)

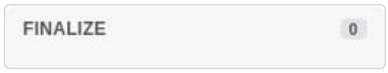


Figure 18 — Finalize column: empty in the demo (count 0). A card waiting at Finalize shows in the Test (waiting on bug) column instead (Pieter Demo)

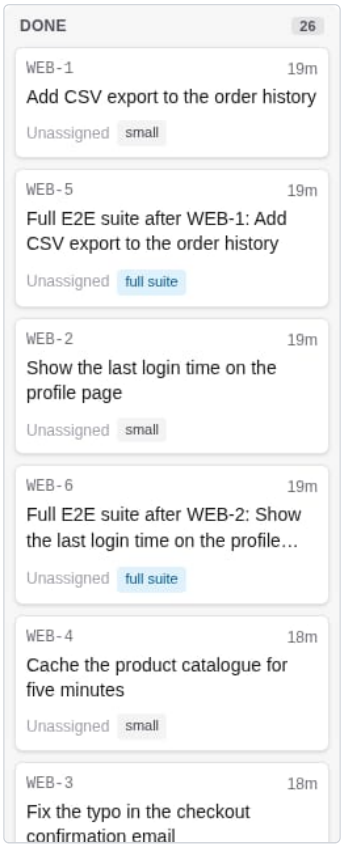


Figure 19 — Done column: 26 finished cards, including full-suite cards marked "full suite" (top of the column shown) (Pieter Demo)

3.3 Creating and specifying a card

A project member creates a card by clicking **New card** in the menu or on the board header. The form starts blank and shows the fields of the selected project's workflow. The page heading is **New card**, with the project key and workflow name below it.

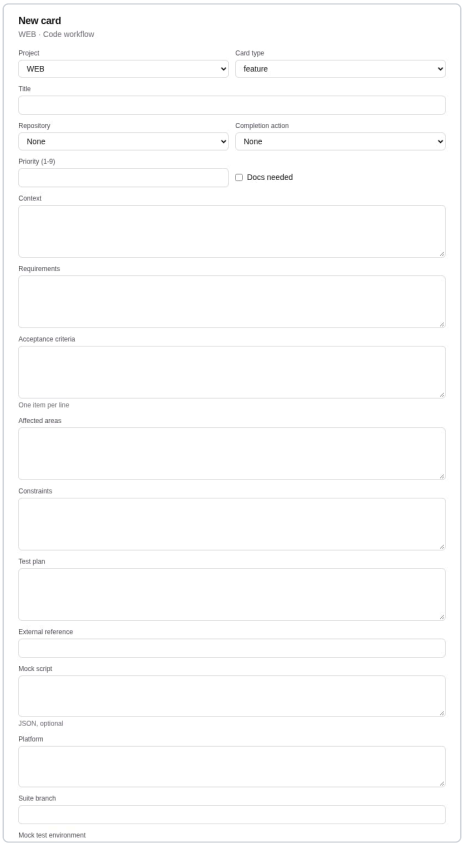


Figure 20 — New card form for the Code workflow (Pieter Demo)

Card form fields (Code workflow)

The form has these fields (for code projects; research and feed projects have different fields). Labels are as on the screen:

- **Project** (dropdown) — The project this card belongs to. Changing it reloads the form for that project's workflow.
- **Card type** (dropdown, required) — `feature`, `bug`, `support`, `refactor`, `docs` or `regression`.
- **Title** (text, required, max 300 chars) — The card title.
- **Repository** (dropdown, optional for a draft, required to Specify) — Which repository to work in. If the project has exactly one repository and you leave it on `None`, the server uses that one.
- **Completion action** (dropdown, optional for a draft, required to Specify) — What the BA does with finished work: `pr` (open a pull request), `push_branch` (push the card branch) or `merge` (merge into the working branch).
- **Priority (1-9)** (optional) — A whole number from 1 to 9. Agents pick up lower numbers first, so 1 is the highest priority. If you leave it blank the card gets 2.
- **Docs needed** (checkbox, optional) — Whether the card goes through the Docs stage, where a technical writer updates the documentation.
- **Context** (required to Specify) — Background and why this work matters.
- **Requirements** (required to Specify) — What must be built.
- **Acceptance criteria** (required to Specify) — How to verify the work is done. The hint reads "One item per line".
- **Affected areas** (required to Specify) — Which parts of the codebase are touched.
- **Constraints** (required to Specify) — Limits the change must respect.
- **Test plan** (required to Specify) — How to test the change.
- **External reference** (optional) — The ticket this card comes from (for example a Jira key or a GitHub issue).
- **Mock script** (JSON, optional) — A test aid for the mock runtime only; real users leave it blank.
- **Platform** (optional) — For regression and full-suite cards: the platform under test, `web` or `mobile`. The field is a plain text box.
- **Suite branch** (optional) — For suite cards: the branch the suite runs on.
- Below these the form has two more fields for the mock runtime, **Mock test environment** and **Mock outbound draft**. Real users leave them blank.

Save draft vs. Specify

The form has two buttons at the bottom:

- **Specify** (primary) — Create the card and move it to the build stage. The card needs a title, a type and every field the build stage requires (Repository, Completion action, Context, Requirements, Acceptance criteria, Affected areas, Constraints, Test plan). The form checks this first. If a field is missing, it shows the errors and does not create the card.
- **Save draft** — Create the card as a draft in the Spec column. Only the title and the type are required (and the priority, if given, must be 1 to 9).
- **Cancel** (link) — Back to the board of the project.

When you press **Specify** with missing required fields, the form shows a red error under each of them, for example "Context is required to specify this card", and the message "Fix the highlighted fields." appears above the buttons. A missing title reads "Title is required". The cursor jumps to the first field with an error.

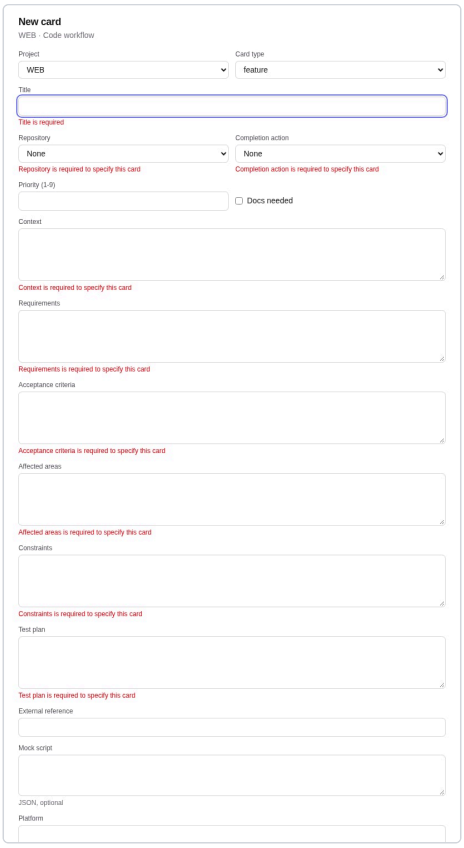


Figure 21 — Validation errors after pressing Specify on a blank form: "Title is required" and one error per missing required field; the buttons are further down the page (Pieter Demo)

The same checks apply when you fill in only some fields. In the next picture the title and Context are filled in, and every other required field is flagged (Repository, Completion action, Requirements, Acceptance criteria, Affected areas, Constraints, Test plan); nothing is saved. Only if the server finds a required field missing that the form did not, the card is saved as a draft and the form shows "Saved as draft", the card ref and "still missing" with the field names.

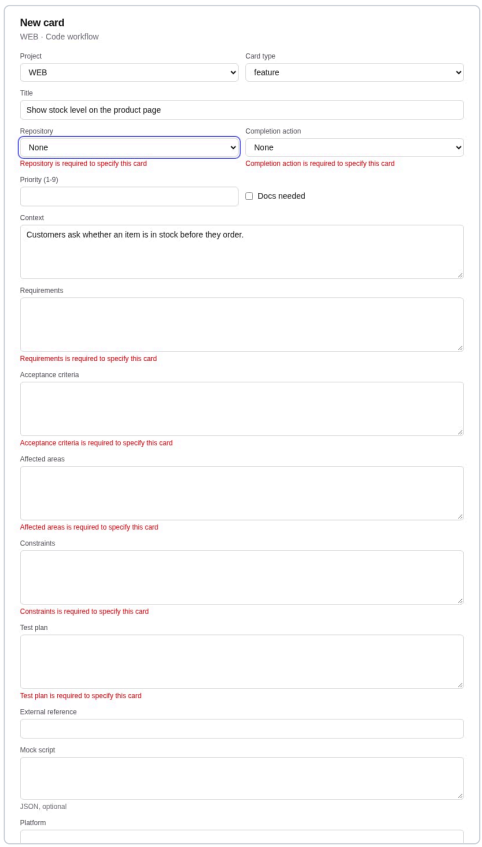


Figure 22 — Title and Context filled in, Specify pressed: the missing required fields are flagged (Pieter Demo)

Research card form

Research projects like SHOOT use a different card form. It has no Repository, Completion action or Docs needed. After Title and Priority (1-9) it has **Question**, **Scope**, **Depth** (a select: `quick`, `standard` or `deep`), **Sub-category**, **Preferred sites** (one per line), **Max per kind** (JSON) and two mock-runtime fields, **Mock script** and **Mock research claims**. The card type is `research`.

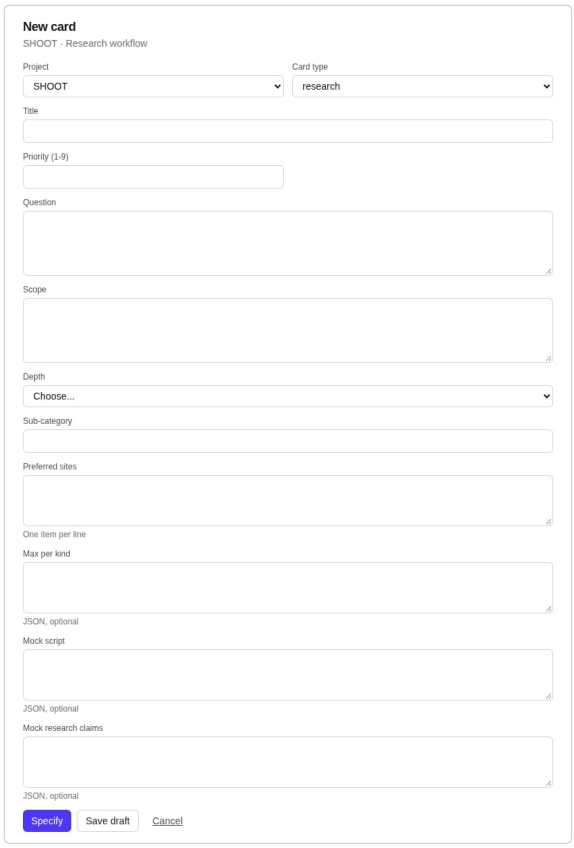


Figure 23 — New card form of the research project SHOOT (Pieter Demo)

Feed card form

Feed projects like NEWS have their own form. After Title and Priority (1-9) it has **Topic**, **Window (hours)**, **Preferred sites**, **Max per kind** (JSON) and the mock-runtime fields **Mock script** and **Mock feed items**. The card type is `feed`.

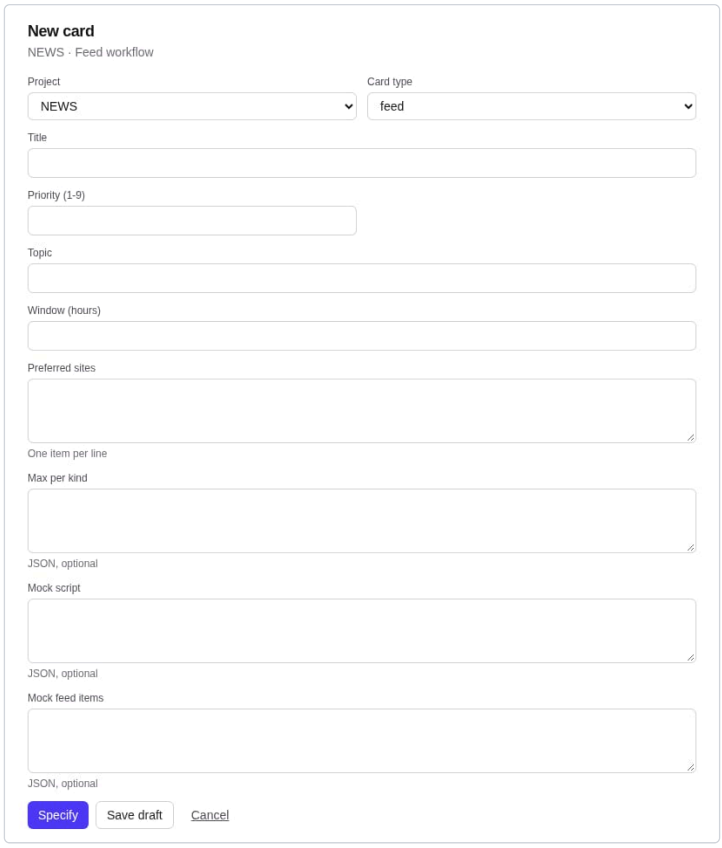


Figure 24 — New card form of the feed project NEWS (Pieter Demo)

Draft and specifying later

A draft card appears in the Spec column. Its detail page has a **Specify** button for the card owner and the project owner. If required fields are still empty, the page shows "Cannot specify yet. Missing:" followed by the field names, and the card stays a draft. The card page has no form to edit fields, so fill in all required fields when you create the card.

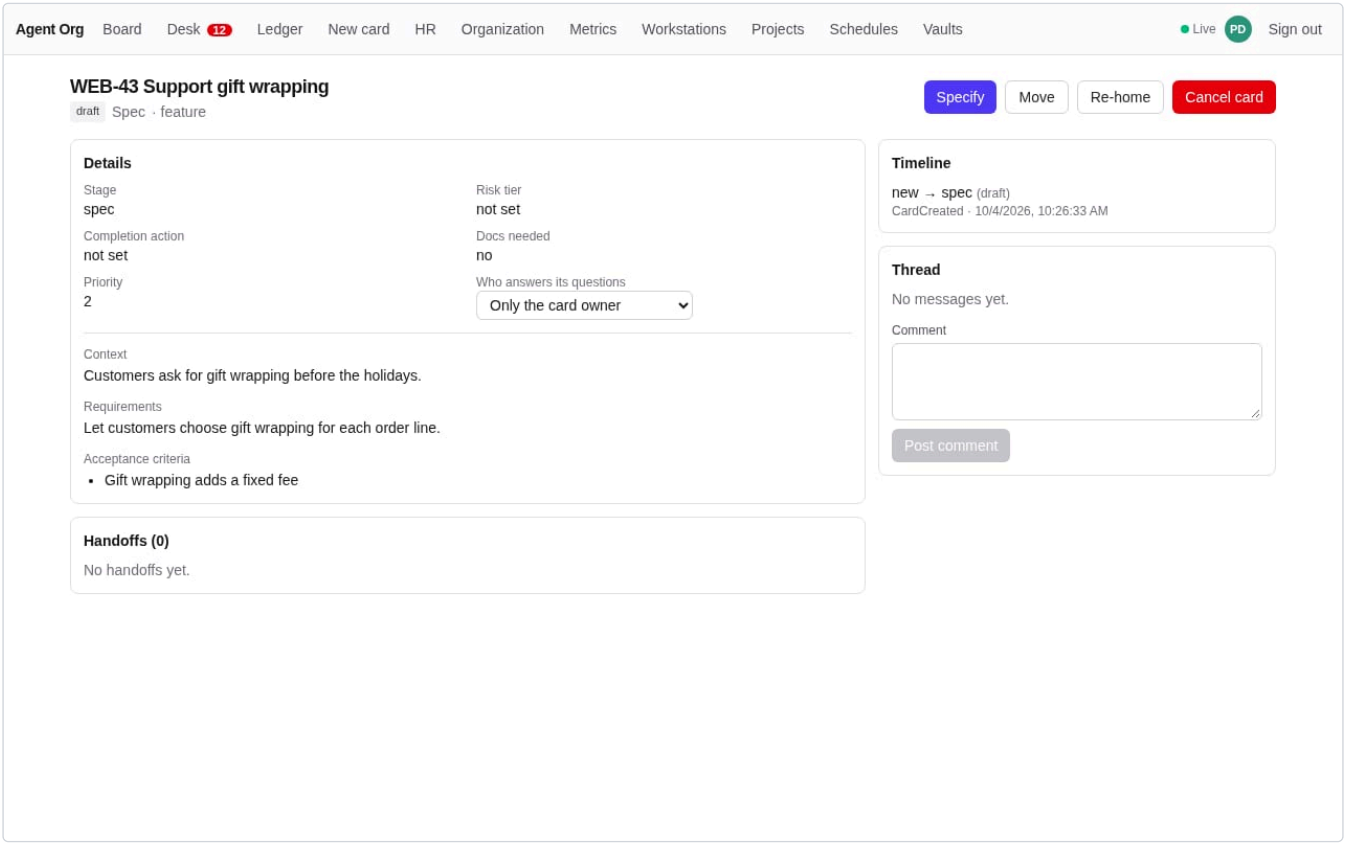


Figure 25 — The draft card WEB-43 with the Specify, Move, Re-home and Cancel card buttons; the "Who answers its questions" select is in the Details panel (Pieter Demo)

What Specify does, in this order:

1. The form (or, on the card page, the server) checks that every field the build stage requires is filled in.
2. On the form, a missing field stops the save and is flagged. On the card page, the card stays a draft and the page lists the missing fields.
3. If nothing is missing, the card moves from the Spec stage to the build stage and is queued: it shows in the To do column.
4. A builder claims it from there, ordered by priority (1 first).

3.4 Card detail

Every card has a detail page showing its status, details and spec fields, handoffs, timeline and thread. Click a card tile on the board to open it.

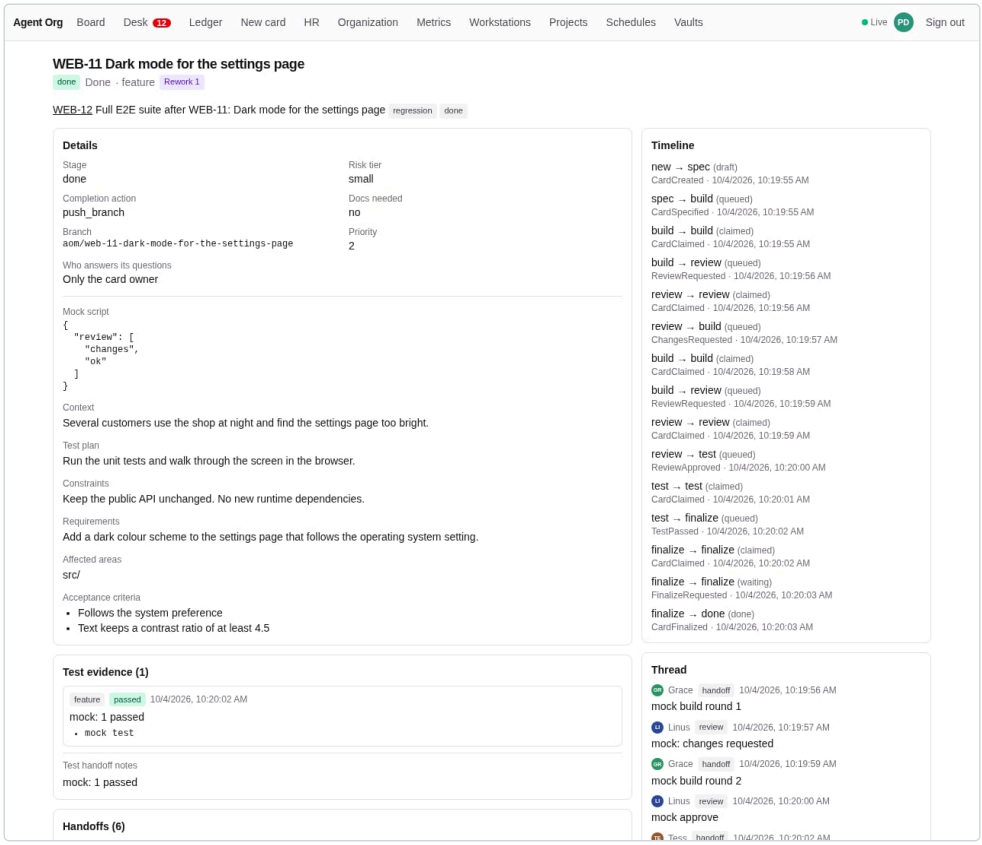


Figure 26 — Card detail page for WEB-11 (a finished card that was bounced once by review) (Pieter Demo)

Header and action buttons

At the top of the page is the card ref and title (e.g., "WEB-11 Dark mode for the settings page"). Below that are the status badge, the board column and the card type (e.g., `done` "Done · feature"), a `Rework n` badge if the card was sent back, and the name of the agent holding the card, if any. On the right are action buttons. They are shown only to the card owner and the project owner; other members see none of them. A finished card has none either.

- **Specify** (blue, draft only) — Move the draft to the build stage if all required fields are filled in.
- **Move** — Override the workflow and move the card to another stage. Shown while the card is open (not done or cancelled); opens a dialog.
- **Release** — Release the active claim: the card goes back to the queue. Shown while an agent holds the card.
- **Re-home** — Forget this card's workstation and worktree, so its current stage restarts from the base branch. Shown while nobody holds the card and it is not done or cancelled.
- **Cancel card** (red) — Cancel the card (it leaves the board). Shown while the card is open; opens a dialog.

Move card dialog

The dialog is titled **Move card** ("Override the workflow and move this card to another stage."). It has a **Target stage** select with the stages of the workflow and a **Reason** box; an empty reason is refused ("A reason is required"). The buttons are **Close** and **Move card**. You can move a card to any stage that is not the final one, bypassing the normal progression. Any claim is released. The card is queued in the new stage (or becomes a draft if you move it to Spec), and the thread gets a system message "Moved from X to Y by name: reason". Moving to a stage whose required fields are not filled in is refused.

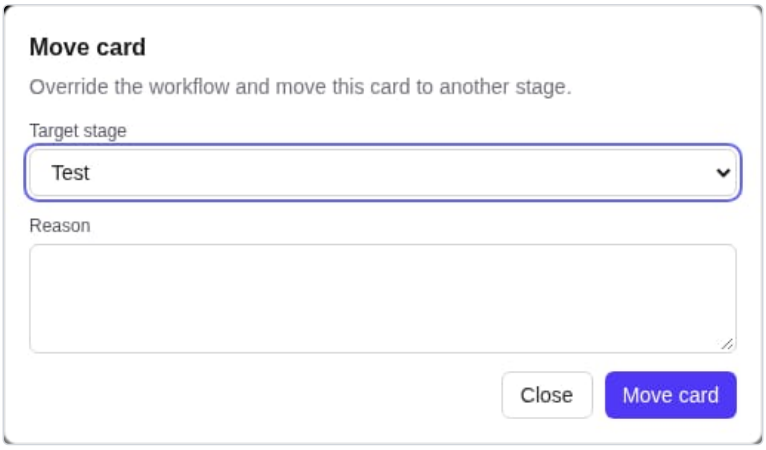


Figure 27 — Move card dialog with the Target stage select and the Reason box (Pieter Demo)

Cancel card dialog

The dialog is titled **Cancel card** ("The card leaves the board. Any active claim is released."). It asks for a **Reason**; an empty reason is refused. The buttons are **Keep card** and **Confirm cancel**.

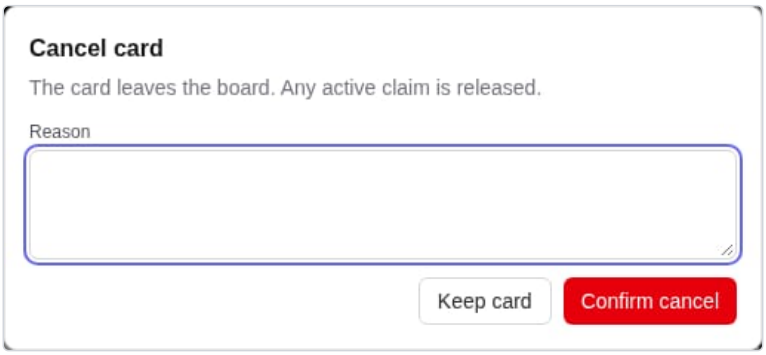


Figure 28 — Cancel card dialog (Pieter Demo)

Blockers banner

If the card is blocked on a question, a red banner appears below the action buttons: "Blocked by **question title**. Answer it on the desk to put this card back in the queue." The question title is a clickable link to the desk item.

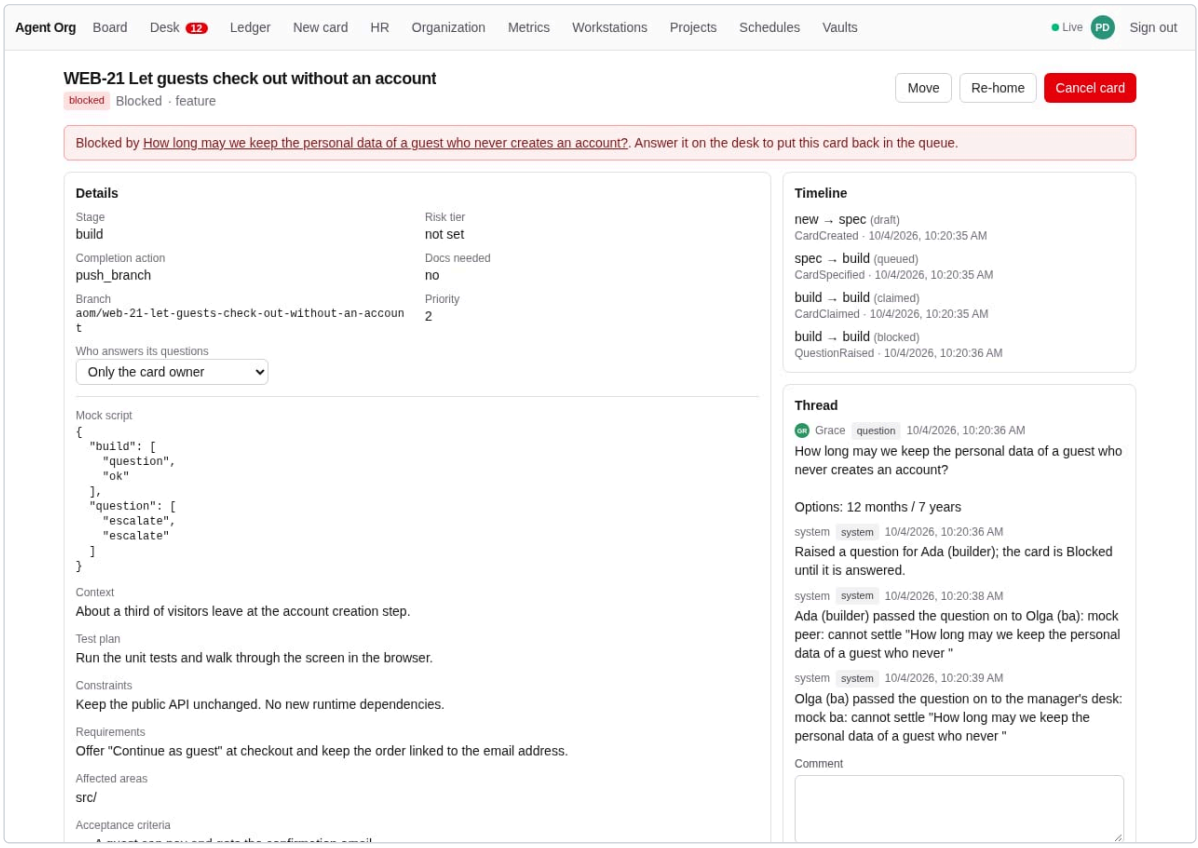


Figure 29 — Card WEB-21 blocked on a question: the red "Blocked by" banner, and the thread with the question and the two system messages that pass it on to Olga and then to the manager's desk (Pieter Demo)

Details panel

The left panel shows card metadata organized in rows:

- **Stage** — Current workflow stage (e.g., "done").
- **Risk tier** — "small" or "large" (or "not set").
- **Completion action** — How the work merges (`pr`, `push_branch`, `merge`, or "not set").
- **Docs needed** — "yes" or "no".
- **Priority** — Priority number 1 to 9 (2 if none was given).
- **Branch** — Git branch name (if set).
- **Who answers its questions** — Decision policy: "Only the card owner" or "Any manager of the project" (see below). A select for the card owner and the project owner, plain text for others.

Below the metadata rows are the spec fields the card has (for example Context, Requirements, Acceptance criteria, Constraints, Test plan, Affected areas, External reference), shown as read-only text or bullet lists. The demo cards also show their **Mock script** as JSON.

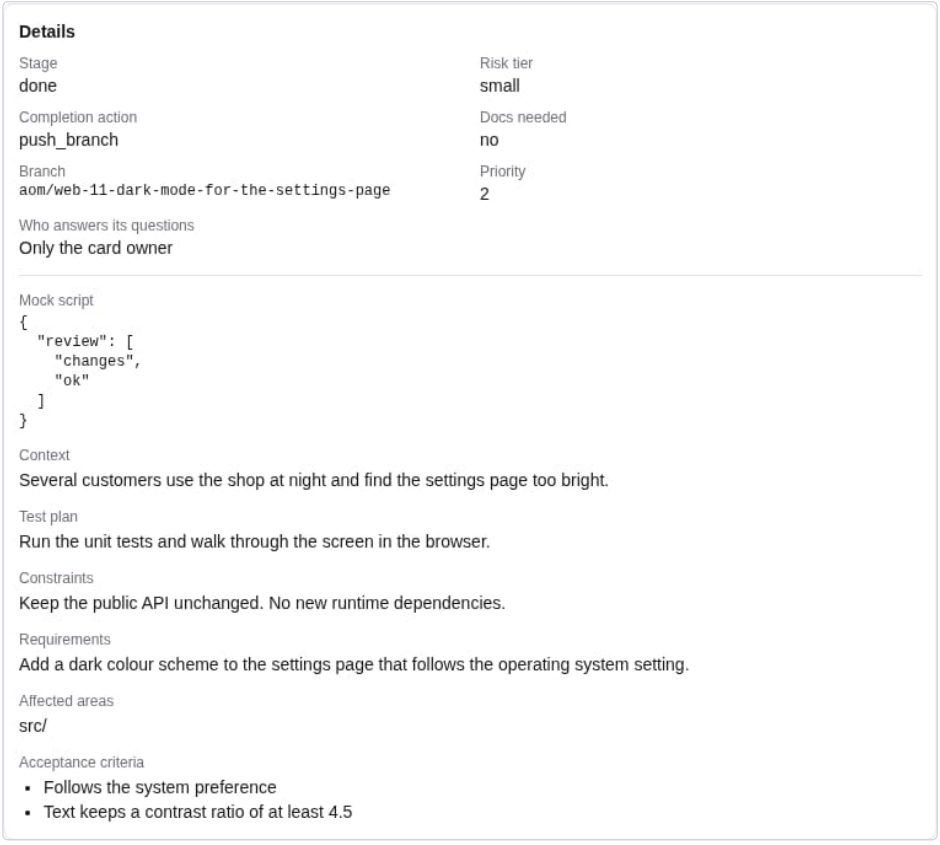


Figure 30 — Details panel of WEB-11: metadata, the Mock script and the spec fields (Pieter Demo)

Who answers its questions (decision policy)

Every card has a decision policy: "Only the card owner" (the default) or "Any manager of the project". It is shown in the Details panel. It controls who can answer the card's questions and approvals on the desk. With "Only the card owner", only the card's owner can answer; the project owner cannot, unless they own the card. With "Any manager of the project", every member of the project can answer. Only the card owner and the project owner can change it.

Handoffs panel

The panel is titled **Handoffs (n)** with the number of handoffs. Each handoff shows the agent's avatar and name, a badge with the stage and the outcome (e.g., "build: handoff", "review: approve"; green, or red for "changes" and "fail"), the time, and the agent's note (e.g., "mock build round 1"). With no handoffs the panel says "No handoffs yet."

A review handoff can carry more: a grey badge "Score n/5", a red badge for each reason (e.g., "missed requirement") and a line "Missed:" with the criterion that was missed. They show why review sent the card back to Rework.

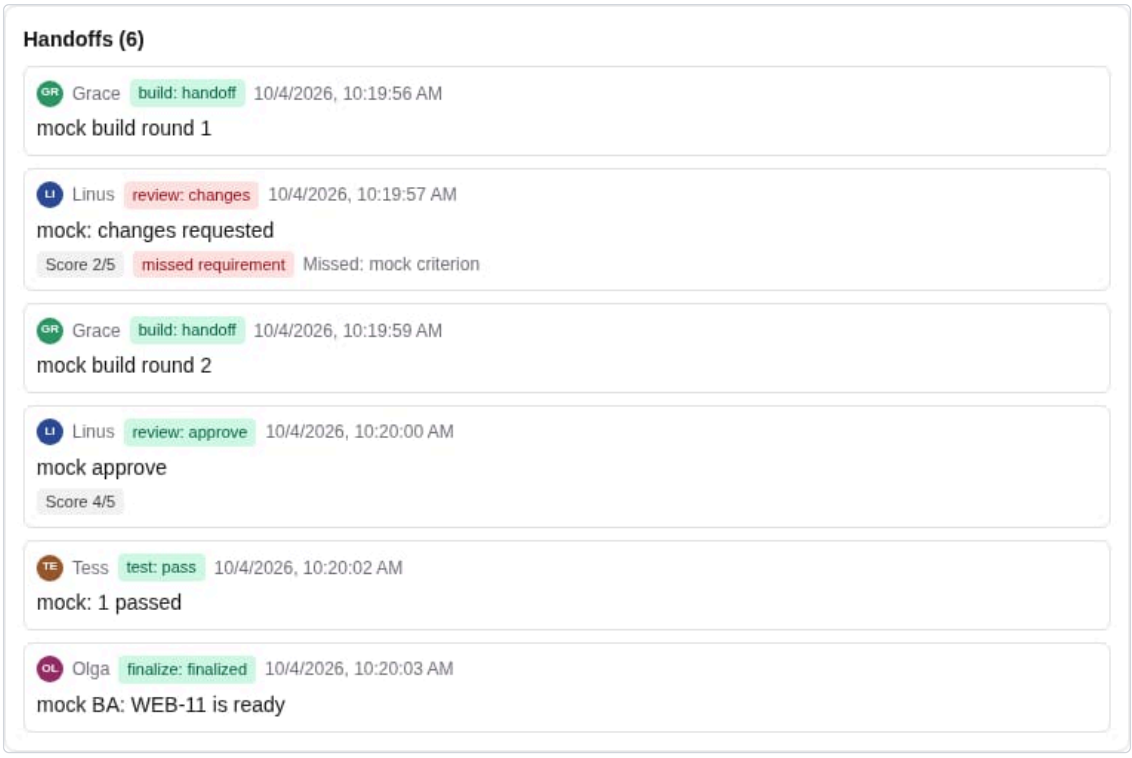


Figure 31 — Handoffs panel of WEB-11 with six handoffs: two builds by Grace, Linus's review with changes (Score 2/5, "missed requirement"), Linus's approval (Score 4/5), Tess's test pass and Olga's finalize (Pieter Demo)

Timeline panel

The Timeline panel (right column, above the Thread) shows the history of the card. Each row shows the previous stage, an arrow, the new stage and the resulting status in brackets (e.g., "build → review (queued)"), and below it the name of the event that caused it (e.g., CardCreated, CardSpecified, CardClaimed, ReviewRequested, ChangesRequested, ReviewApproved) and the time.

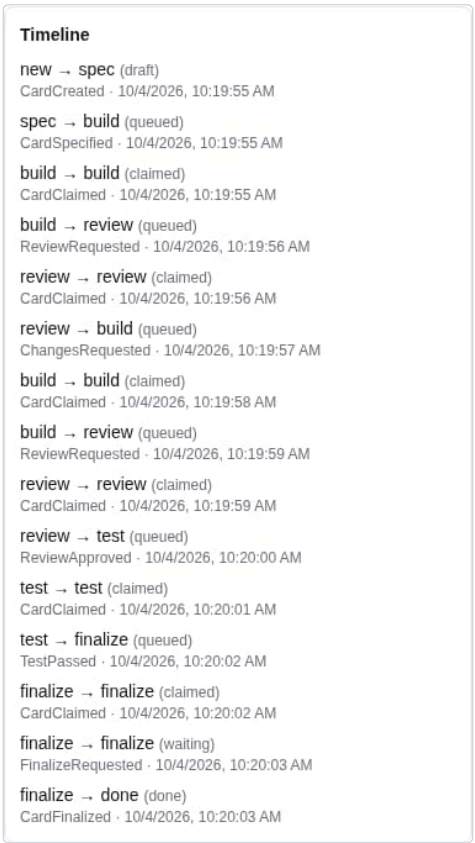


Figure 32 — Timeline panel of WEB-11 from CardCreated to CardFinalized, with the review bounce (ChangesRequested) in the middle (Pieter Demo)

Thread panel

The Thread panel shows all messages and system events on the card in chronological order. Messages can be:

- **Agent handoffs** — Agent notes when they hand off the card (e.g., "mock build round 1").
- **Agent reviews** — Reviewer notes (e.g., "mock: changes requested").
- **Agent evidence** — Test results and pass/fail messages.
- **Questions** — Questions raised to the manager (with options listed).
- **Answers** — Manager answers to questions (recorded when answered on the desk).
- **Manager comments** — Comments posted by managers on the card.
- **System messages** — Events like "Shift rejected by the runner" or "Moved from X to Y by name: reason".

Each thread message shows the author (agent, user or system), a badge for the message kind (e.g., "handoff", "review", "question", "system"), the time and the message text. Below the messages is a **Comment** box with a **Post comment** button, for every member of the project. With no messages the panel says "No messages yet."

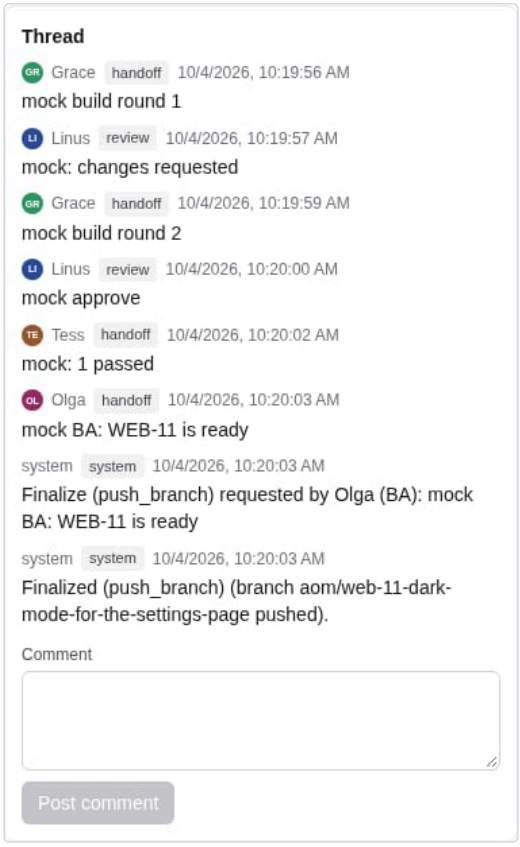


Figure 33 — Thread panel of WEB-11: handoff and review messages, two system messages about the finalize, and the Comment box (Pieter Demo)

Live shift panel

While an agent has a shift open on the card, the card shows a **Live shift** panel at the top of the left column. Its heading names the agent and the stage (e.g., "Live shift · Ada build"). Below it is a black log with time-stamped lines of what the agent reports; it scrolls as new lines arrive and keeps the last 200. Before the first line it reads "Waiting for output…". In the demo the lines read "mock hanging", because the mock runtime is scripted to hang on this card.



Figure 34 — Live shift panel: Ada's build shift, with the scripted "mock hanging" lines (Pieter Demo)

On the full card page the Live shift panel sits above the Details panel.

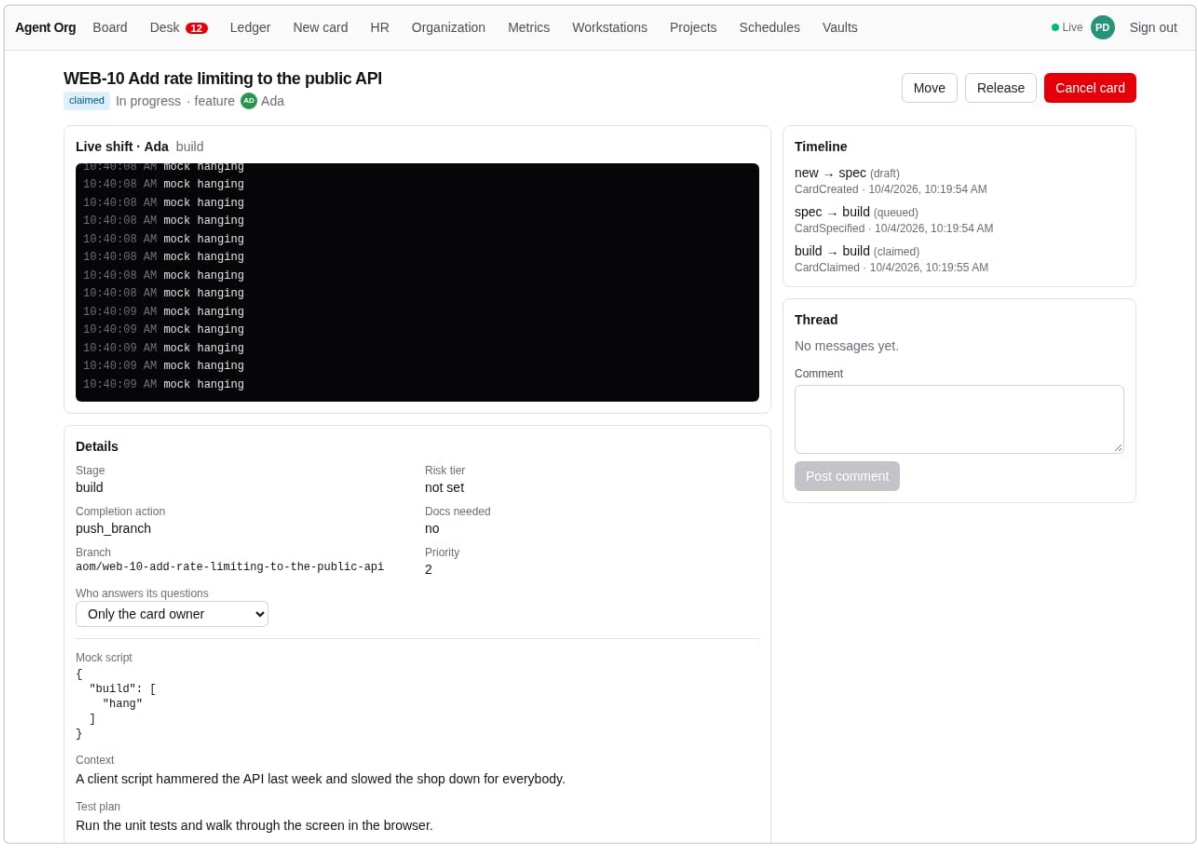


Figure 35 — Full card page for WEB-10, claimed by Ada in the build stage, with the Move, Release and Cancel card buttons (Pieter Demo)

What Maya sees (second manager)

Maya is a second manager of the WEB project. She sees the same panels as Pieter: Details, Handoffs, Timeline, Thread and, while a shift is open, Live shift. What she may do depends on who owns the card:

- On a card she owns (one she created), she gets the same buttons as Pieter (Specify on a draft, Move, Release, Re-home, Cancel card), can change "Who answers its questions", and can answer the card's questions on her desk.
- On a card owned by someone else, such as Pieter's WEB-21, she sees no action buttons, the decision policy is plain text, and she cannot answer its questions unless the policy is "Any manager of the project".
- On every card of the project she can read the panels and post comments in the Thread.

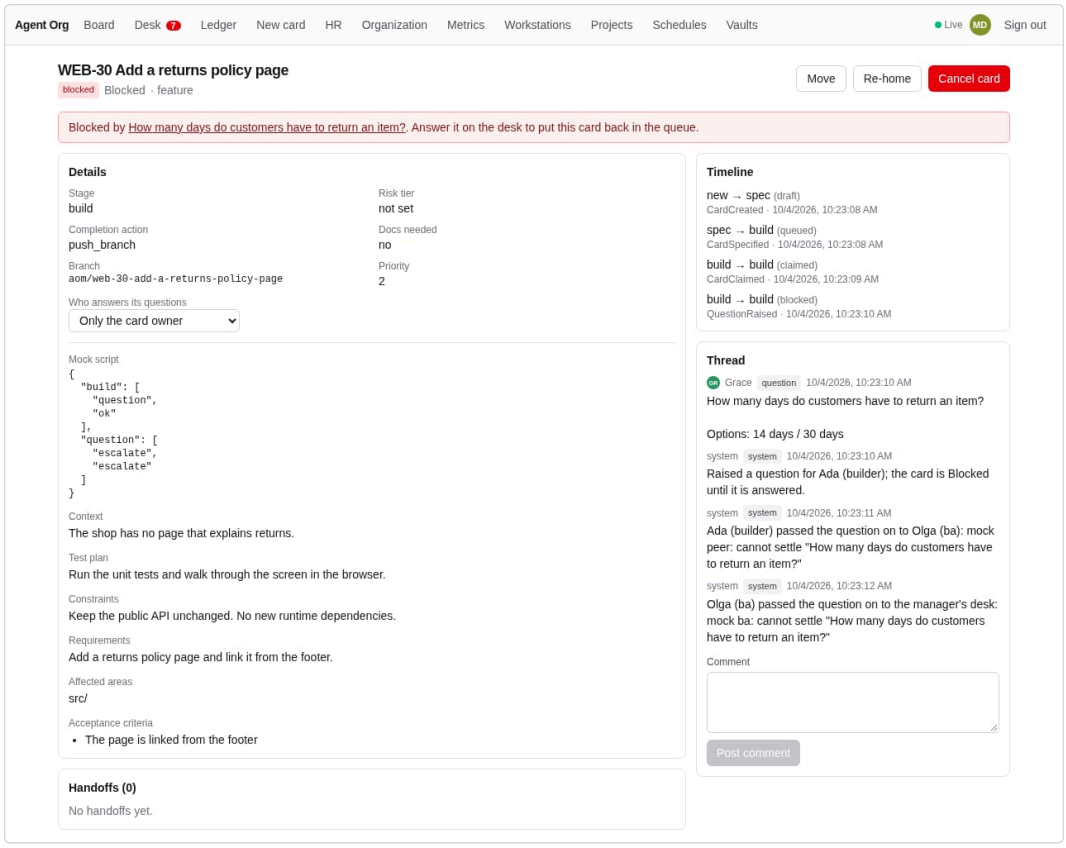


Figure 36 — Card WEB-30 as seen by Maya, who owns it: blocked by the question "How many days do customers have to return an item?", with the Move, Re-home and Cancel card buttons and the policy select (Maya Demo)

3.5 The decision desk and the ledger

The **Decision desk** is the manager's page where every question needing an answer, every approval, and every alert lands. The main menu shows a red badge with the number of open items next to `Desk` (for example 12), so the manager sees at a glance that something is waiting. Under the page title the desk shows the same numbers as text: `12 open · 3 blocking`. At the top right are three buttons: `Add to ledger`, `Intake` and `Chat`. When nothing is waiting the page says *"Nothing is waiting on you."*

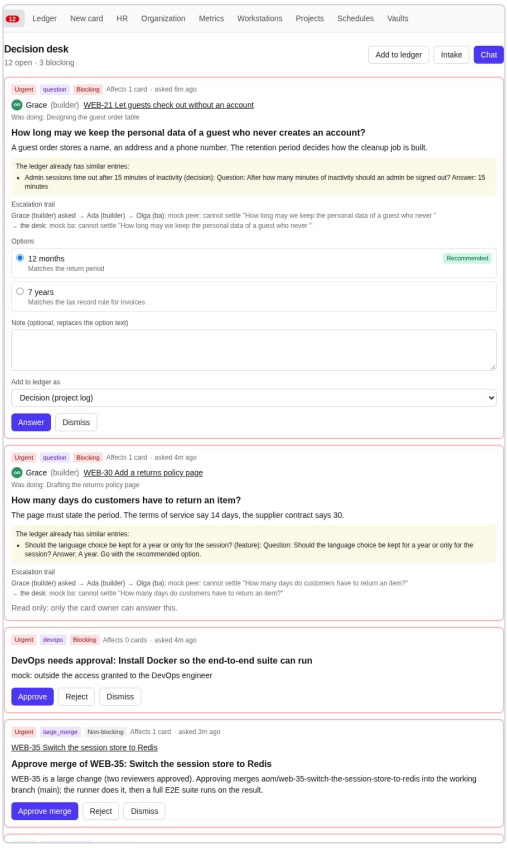


Figure 37 — Decision desk top: "12 open · 3 blocking", the Add to ledger, Intake and Chat buttons, then the pinned items: two Urgent blocking questions from Grace (WEB-21, WEB-30), a DevOps approval and the WEB-35 large merge (Pieter Demo)

What reaches the desk

An item has a **kind** and is either **Blocking** (a card is held up until it is answered) or **Non-blocking** (an approval or heads-up that waits for the manager). Items on the desk are:

- **Questions**: an agent working on a card asks which option to take. A question first goes to a senior agent of the same role or the card's previous holder, then to the BA, and only then to the desk (see "Escalation trail" below), so some questions never reach the desk.
- **Outbound approvals** (chapter 4): a comment, resolution or decision drafted for a Jira ticket or GitHub issue (usually by the BA). The manager approves, edits and approves, or rejects. Only the card's owner decides.
- **Large-merge approvals** (chapter 5): a large-risk card that two reviewers approved waits for the manager to approve the merge into the working branch (the button reads `Approve merge`).
- **Tester bottleneck alerts** (chapter 5): a card has waited for a tester for more than 5 minutes while every tester is busy or offline; the item offers `Hire tester` and `Acknowledge`. It resolves itself when the tester queue is empty again.
- **Digests to rate** (chapter 6): when a feed has produced a digest, the manager rates its items (Useful, Irrelevant or Boring, with an optional reason) so taste rules improve. Any active member of the project may rate.
- **Taste-rule proposals** (chapter 6): the curator proposes a new taste rule from the verdicts; the rule text can be edited before it is approved.
- **DevOps approvals** (chapter 7): the DevOps engineer wants to do something destructive or outside the access granted to the workstation; the manager approves or rejects. These items belong to the workstation owner only.
- **Unmapped tickets** (chapter 2): an incoming ticket could not be matched to any project; the manager picks the project.
- **Definition approvals** (chapter 8): a new role or workflow awaits approval before it becomes available.

Item anatomy: badges, affected cards, age, who asked, the question

Every desk item shows a row of badges, then (when an agent raised it) who asked, the card it is about, and what the agent was doing. Then the title and body.

- **Badges**: `Urgent` (red; the item is pinned to the top); the item's kind in violet (some are shown raw: `question`, `devops`, `large_merge`, `tester_bottleneck`, `taste_rule`; others have a plain label: `Unmapped ticket`, `Update to a ticket`, `Posting failed`, `Role or workflow`); `Blocking` or `Non-blocking`; `Affects n card(s)` (how many cards wait on this item); and `asked 6m ago`.
- **Who asked**: avatar, agent name and its role in parentheses (builder, reviewer, ba, and so on). Items raised by the app itself (a large-merge approval, the tester alert) have no asker.
- **Card**: a linked card reference if the question is about a card. Clicking it opens the card.
- **Was doing**: a short line the agent wrote about what it was doing (for example *"Was doing: Designing the guest order table"*).

- **Title and body**: the question or approval text. Below them come the ledger box and the escalation trail (when present), then the answer form or buttons.

Ordering rule and what Urgent means

Open items are sorted by: (1) pinned items first (these carry the Urgent badge), (2) blocking items ahead of non-blocking, (3) items that hold up more cards first, (4) oldest first. **Urgent** is set by the app, not by the manager, and cannot be changed on the page. Items that are pinned: blocking questions, DevOps approvals, large-merge approvals, the tester bottleneck alert, vault sync conflicts, and failed finalize or test-environment problems.

Answering a question: options, recommended default, note, and ledger

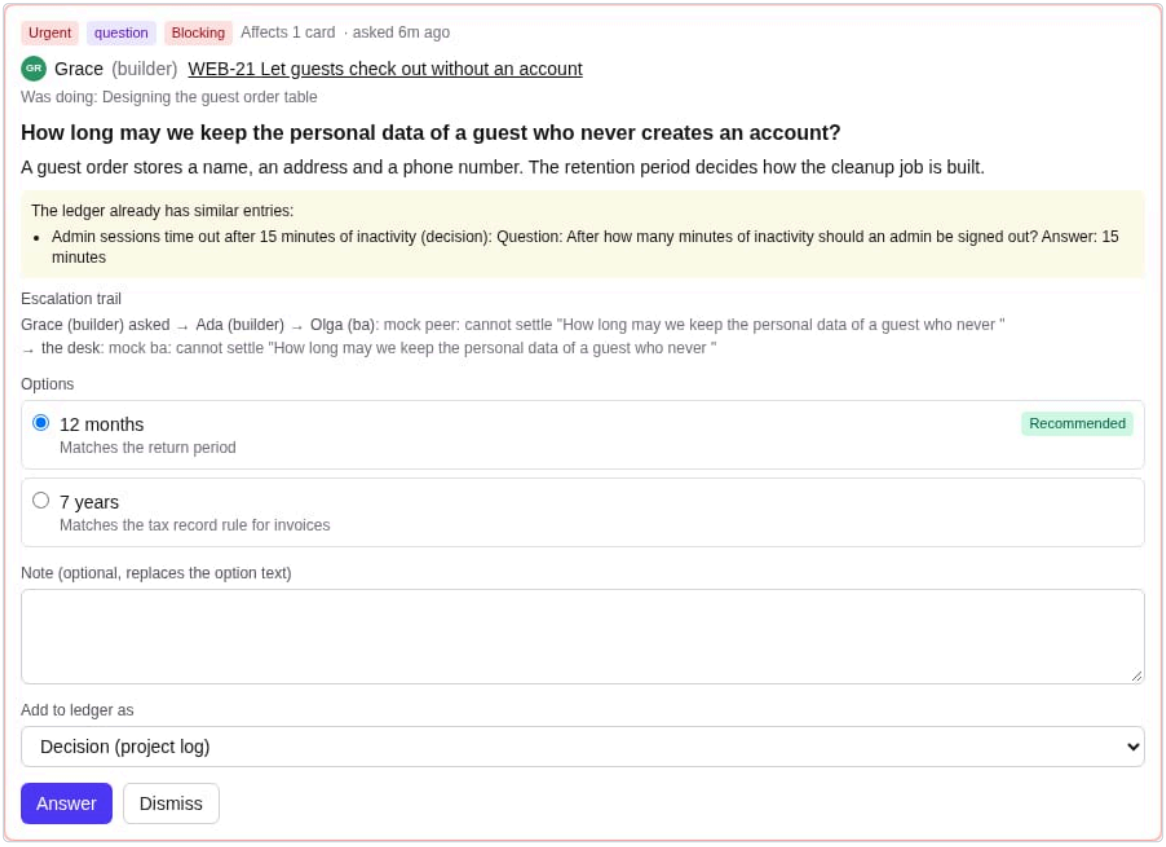


Figure 38 — Grace's question on WEB-21: ledger box, escalation trail, two options (12 months marked Recommended), the Note field, the "Add to ledger as" list set to "Decision (project log)", and the Answer and Dismiss buttons (Pieter Demo)

A question item shows its **options** as radio buttons, each with a short description. If the agent gave a **recommendation**, that option carries a green `Recommended` badge and is pre-selected. Otherwise nothing is selected.

The manager chooses an option, or types a note in the field `Note (optional, replaces the option text)`. What is typed is the answer; if the note is empty the label of the chosen option is the answer. With no option and no note the form says *"Choose an option or write an answer"*. The list `Add to ledger as` decides how the answer is recorded:

- **Decision (project log)**: the default. A decision entry of the project, linked to the card.
- **Requirement (this card)**: a requirement entry linked to this card.
- **Standing order (this project)**: a rule for every card of this project.
- **Standing order (every project)**: a rule for every project of the organization. Agents load standing orders in the brief of their shifts.

Who may add standing orders: a project standing order only by the project owner; a standing order for every project only by the organization owner. A guest manager can answer with a decision or a requirement. Agents cannot add ledger entries; they can only search the ledger. The manager can also add an entry directly with the `Add to ledger` button at the top, without a question prompting it. The form has the fields `Kind` (Standing order, Requirement or Decision), `Project`, `Title` and `Text`. A standing order may leave the project blank (`Every project`); a requirement or decision needs a project (*"Choose the project this applies to"*).

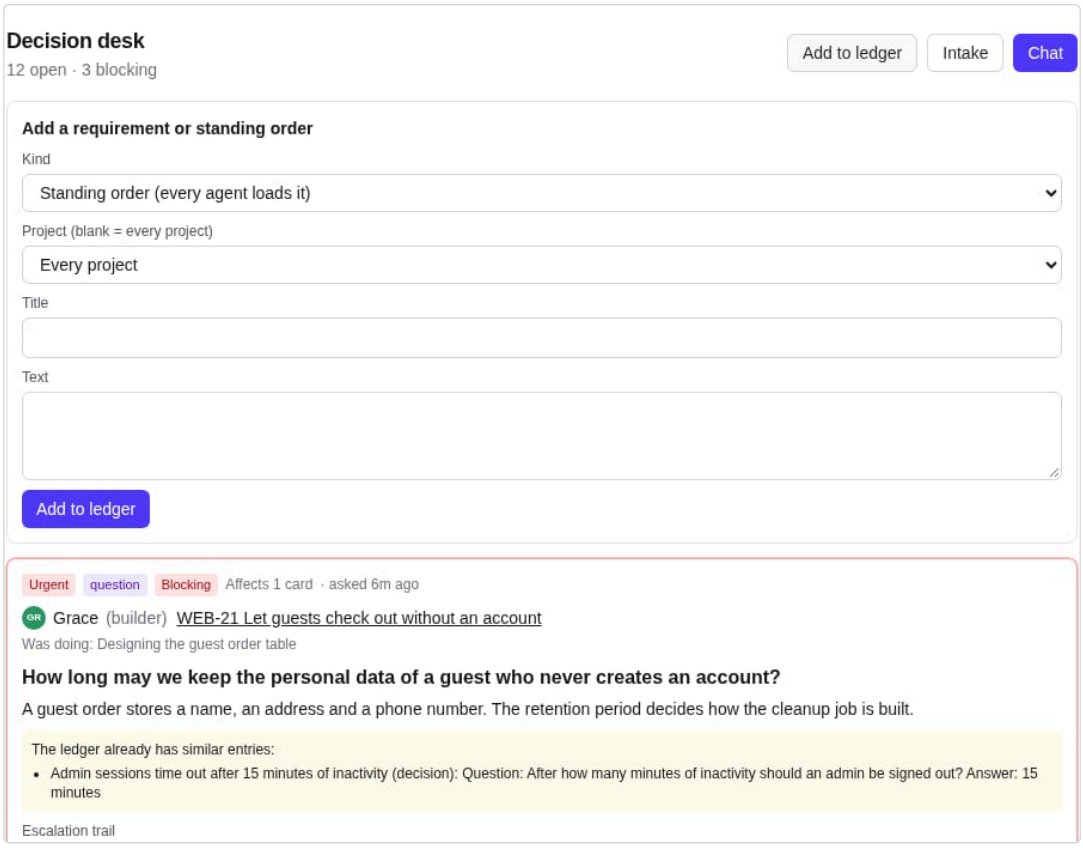


Figure 39 — Desk with the "Add a requirement or standing order" panel open at the top: Kind ("Standing order (every agent loads it)"), Project ("Every project"), Title, Text and the Add to ledger button (Pieter Demo)

`Dismiss` closes the item without an answer (cards waiting on it go back to the queue as well); `Answer` records the answer, writes the ledger entry and releases the cards waiting on this item. Other items have their own buttons: `Approve` or `Approve merge` and `Reject` for approvals, `Acknowledge` for alerts, plus `Dismiss` on all of them.

What happens when an answer is sent

What answering a question does, in this order

1. The answer is recorded on the desk item, which becomes answered and moves to `Answered and closed`.
2. A ledger entry of the chosen type is written (title: the question, text: "Question: … Answer: …"). When a vault exists for it, a decision file or standing-orders entry is also written to the vault through a workstation.
3. The answer is posted to the thread of the card.
4. Every card that was blocked only by this item goes back to the queue (status queued, same stage). A card that still has another open blocking item stays blocked.
5. The next shift of such a card gets the card thread (with the answer) and the card's ledger entries in its brief.

The ledger already has similar entries

When an agent raises a question, the app searches the ledger (entries of the project plus standing orders) and keeps up to three matches. If there are any, the desk item shows an amber box *"The ledger already has similar entries:"* with the title, the type and the start of the text of each. Read it before answering: the question may already be settled.

Escalation trail

A question travels in steps: the asking agent, then a senior agent of the same role (or the agent that last held the card), then the BA, then the desk. When it has travelled beyond the asker, the item shows `Escalation trail` with the steps. In the figure above the trail reads: Grace (builder) asked, Ada (builder), Olga (ba): *mock peer: cannot settle …*, the desk: *mock ba: cannot settle …* (the notes are scripted by the mock runtime). If a peer or the BA answers, the item never reaches the desk; in `Answered and closed` it reads *"answered by Ada (without reaching the desk)"*.

Who may answer: the decision policy

Every card has a setting `Who answers its questions` in its Details panel: `Only the card owner` (the default) or `Any manager of the project`. With the default, another manager of the project sees the item but gets `Read only: only the card owner can answer this.` instead of the form. Items that belong to no card (for example the tester alert) can be answered only by the project owner (`Read only: only the project owner can answer.`). Ratings of a digest can be given by any active member of the project.

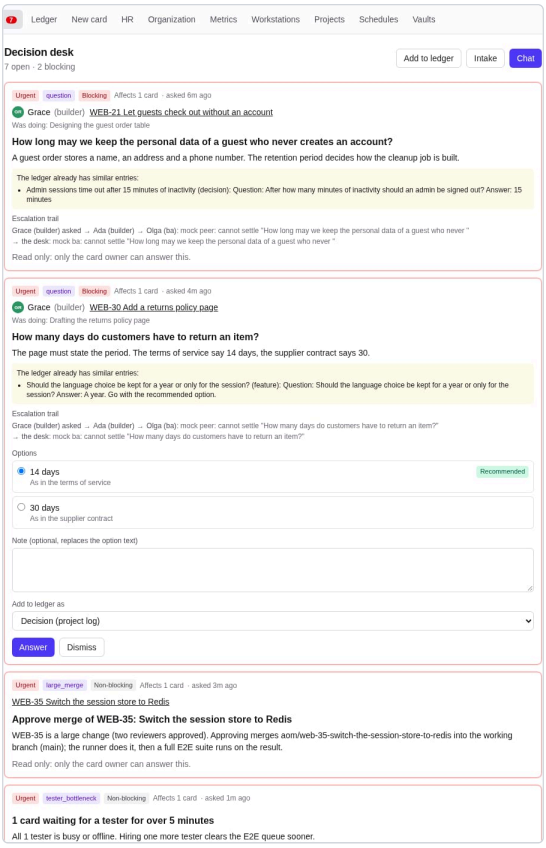


Figure 40 — Desk as Maya sees it: "7 open · 2 blocking". WEB-21 and the WEB-35 large merge show "Read only: only the card owner can answer this."; WEB-30 is Maya's own card and shows the full answer form (Maya Demo)

A manager of a shared project like Maya sees the items of every project she is an active member of, but not the owner's organization-level items (DevOps approvals, vault conflicts). In the figure, WEB-21 and WEB-35 are cards of Pieter, so Maya can read them but not answer. WEB-30 is Maya's own card, so she answers it. Standing orders for a project can only be added by the project owner, also when answering.

Answered and closed list

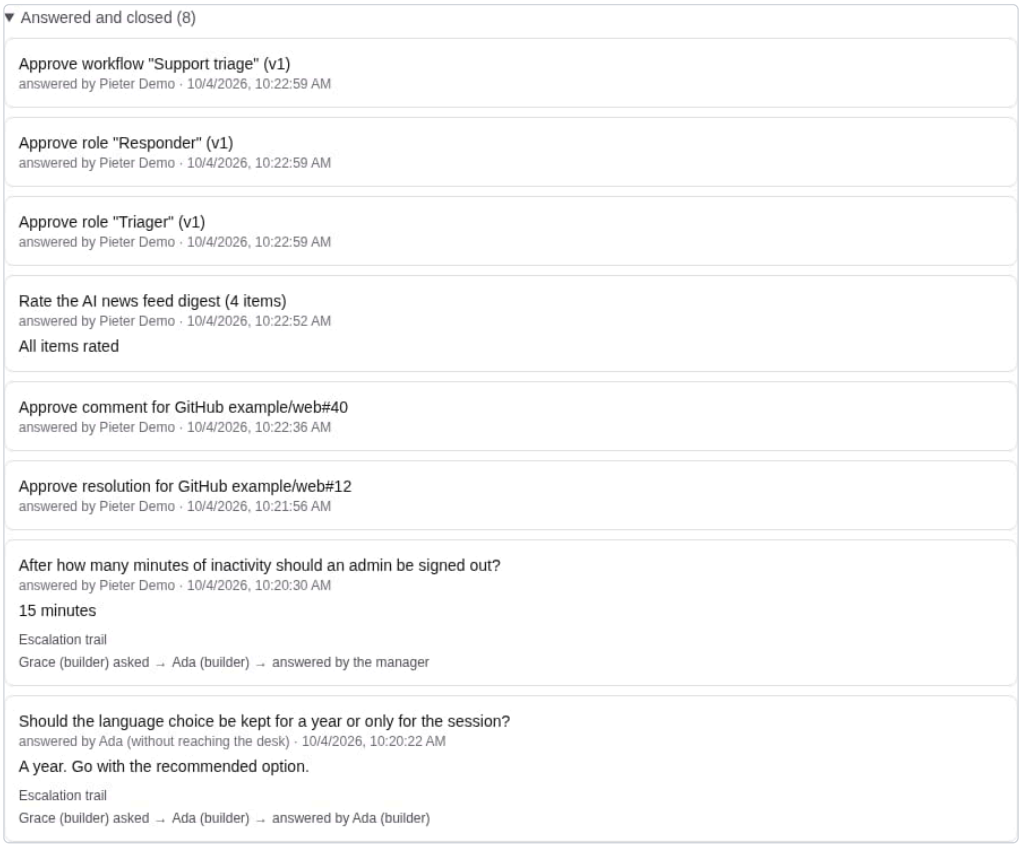


Figure 41 — Answered and closed (8), expanded: approved workflow and roles, a digest rating, two outbound approvals, a question answered by Pieter Demo and a question answered by Ada without reaching the desk (Pieter Demo)

At the bottom of the desk is a collapsible `Answered and closed (n)` section with the most recent closed items (up to 100), newest first: approvals, ratings, answered or dismissed questions. Each shows the title, the status and who closed it (*"answered by Pieter Demo"*), the date and time, the answer text and the escalation trail. If an agent answered it, the line says *"answered by Ada (without reaching the desk)"*.

The Ledger page: search, scope, project, entries, and Add to ledger form

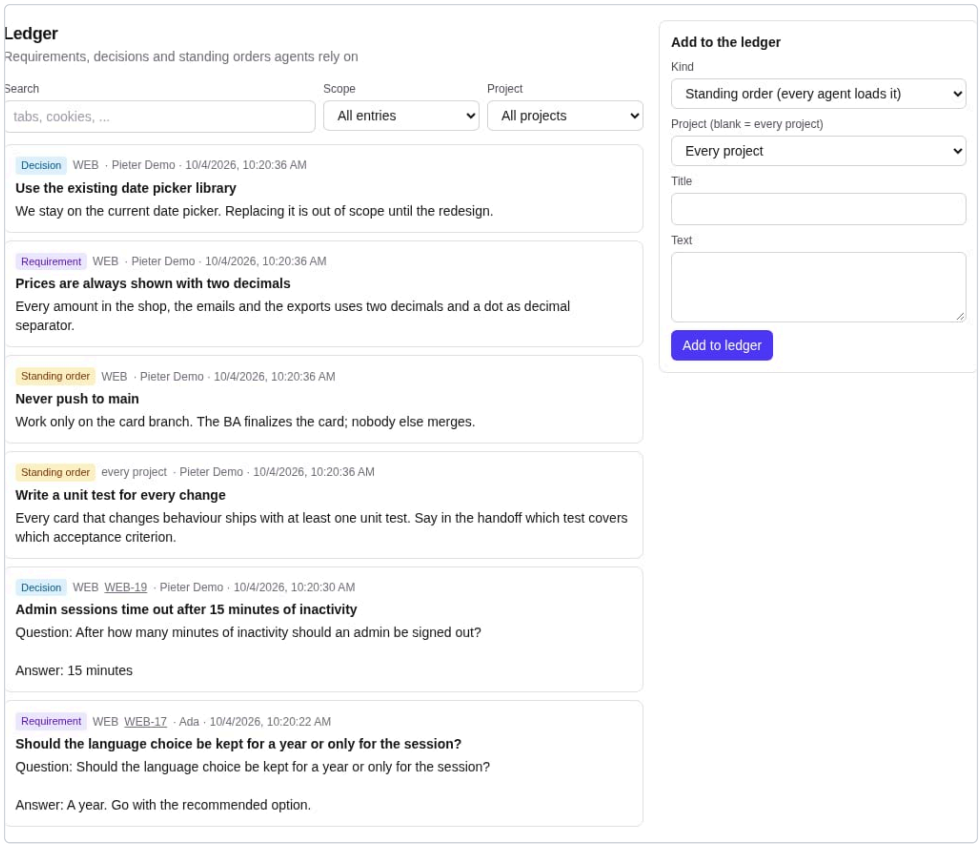


Figure 42 — Ledger page: Search box, Scope and Project filters, six entries (decisions, requirements, standing orders; WEB and "every project") and the "Add to the ledger" form on the right (Pieter Demo)

The **Ledger** page ("Requirements, decisions and standing orders agents rely on") shows the entries of your organization and of the projects you belong to. It has a `Search` box, a `Scope` list (`All entries`, `Standing orders`, `Decisions`, `Requirements`), a `Project` list (`All projects` or one project key) and the form `Add to the ledger`.

- **Entries**: each entry shows a badge (`Decision`, `Requirement` or `Standing order`), the project key (or `every project`), the card reference when it came from a card, who wrote it (a person, or the agent who answered), the date and time, the title and the text. Newest first, at most 200. Entries from answered questions have the text "Question: … Answer: …".
- **Search**: finds entries by words in the title and text (word forms count, so "session" also finds "sessions"); with a search the best match comes first. With no match the page says *"No entries match."*; an empty ledger says *"The ledger is empty. Answers on the desk are recorded here."*
- **Project filter**: shows that project's entries plus the entries for every project.
- **Add to the ledger form**: the same form as on the desk (Kind, Project, Title, Text). The rules for who may add standing orders are the ones given above.
- **Standing orders**: every agent gets them in the brief of its shifts. Orders for every project apply to all projects; project orders only to that project.

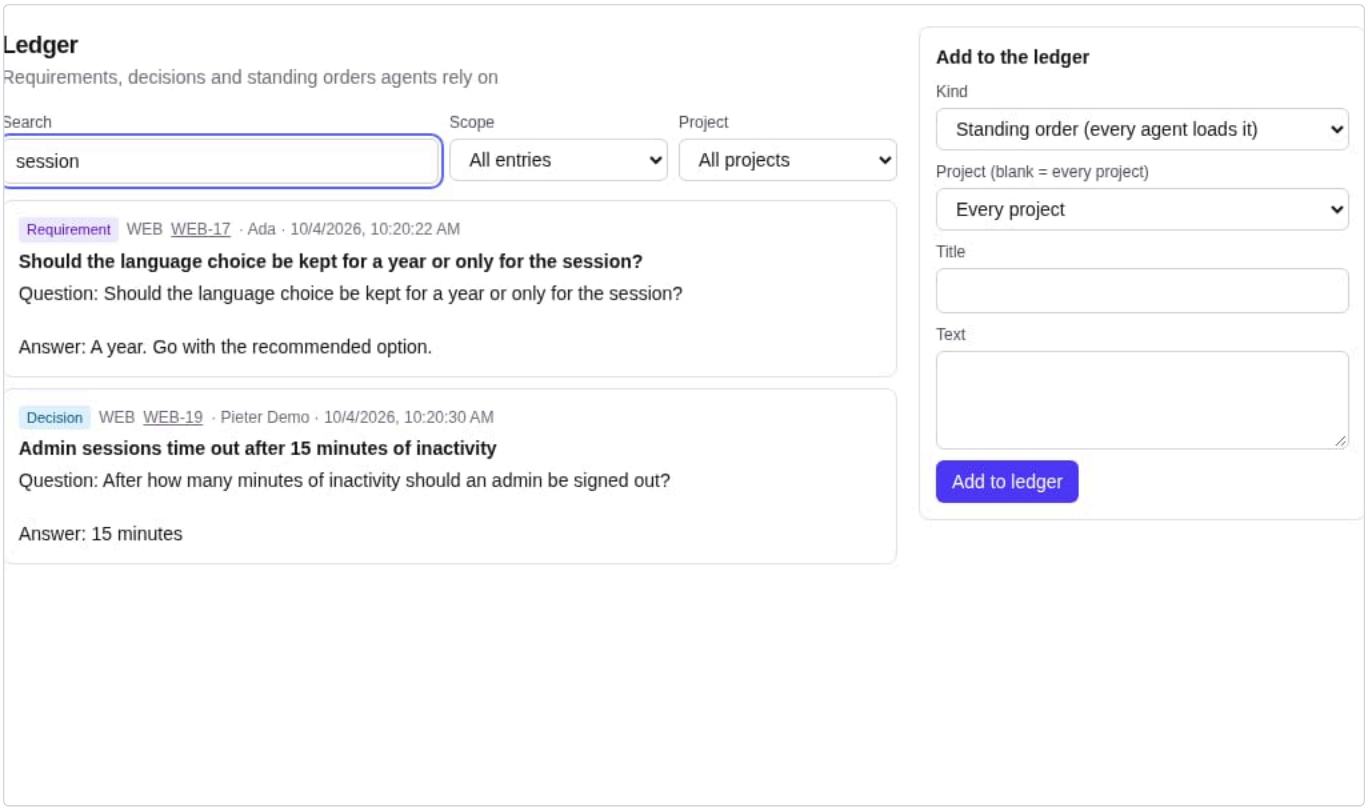


Figure 43 — Ledger searched for "session": two entries found, a requirement (WEB-17, answered by Ada) and a decision (WEB-19) (Pieter Demo)

3.6 Chat with agents

The **Chat** drawer (opened from the decision desk's Chat button) lets the manager talk directly to an agent. You may chat with every agent of your own organization; a manager of a shared project may chat with the agents of that project. Messages go through the agent's workstation and the reply streams in as it is written. Every conversation is saved as a thread.

Starting a chat: choosing an agent

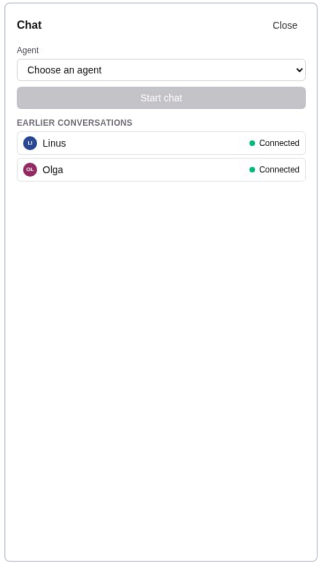


Figure 44 — Chat drawer: the Agent list ("Choose an agent"), the Start chat button (disabled until an agent is chosen) and Earlier conversations with Linus and Olga, each with a green "Connected" dot (Pieter Demo)

Click `Chat` on the decision desk. A drawer opens showing:

- **Agent**: a list of your active agents, each as `Name (role)` (the DevOps engineer as `Name (devops, DevOps)`). Choose one.
- **Start chat**: creates a new conversation with that agent and opens it. A `Back` button returns to this list.
- **Earlier conversations**: your past chats, most recently active first. Each shows the agent's avatar, name and a dot with the text `Connected` (green) or `Offline` (grey), meaning whether the agent's workstation is connected. Click one to continue that conversation.

Sending a message and streaming reply

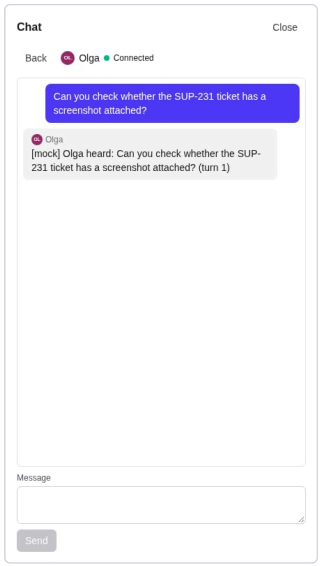


Figure 45 — Chat with Olga: the manager's message in purple, Olga's reply "[mock] Olga heard: … (turn 1)", and the empty Message box with Send (Pieter Demo)

Type a message in the `Message` box and press `Send` (or Ctrl+Enter). The reply streams in below while the agent writes it. While the agent is answering, the box is locked and a `Stop` button appears; a second message is accepted only when the reply is done. The demo uses the MOCK runtime, so replies read `[mock] Olga heard: ...`, an echo of your message; a real Claude Code agent writes real replies.

Connection status and offline workstations

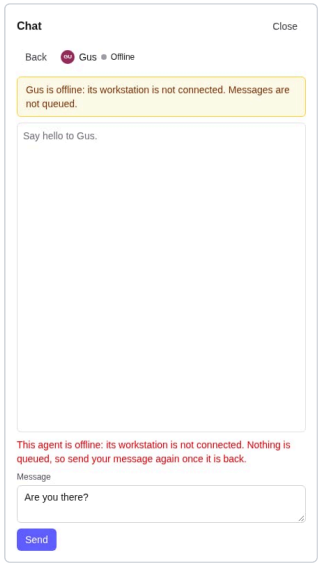


Figure 46 — Chat with Gus whose workstation is offline: grey "Offline" dot, amber banner "Gus is offline: its workstation is not connected. Messages are not queued.", and the red error after pressing Send, with the text put back in the Message box (Pieter Demo)

A green dot with `Connected` means the agent's workstation is connected; a grey dot with `Offline` means it is not. For an offline agent an amber banner reads *"Gus is offline: its workstation is not connected. Messages are not queued."* `Send` is not disabled. If you send anyway, the message is not delivered: a red error says *"This agent is offline: its workstation is not connected. Nothing is queued, so send your message again once it is back."* and your text returns to the Message box. Nothing is stored for later.

Saved threads and visibility

Every conversation is saved as a thread. The address `/chat/<threadId>` opens one as a full page ("Chat with Olga", with a link `Back to the desk`), which suits a phone. A chat started from the desk drawer is not tied to a project and is listed only for the person who started it. A chat that is tied to a project shows the note *"This conversation is visible to this project's managers."*

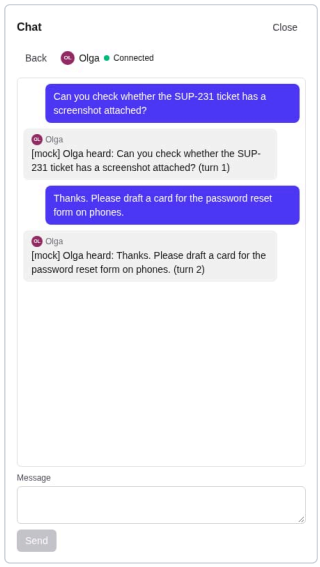


Figure 47 — The same conversation with Olga continued: a second message and the reply "(turn 2)" (Pieter Demo)

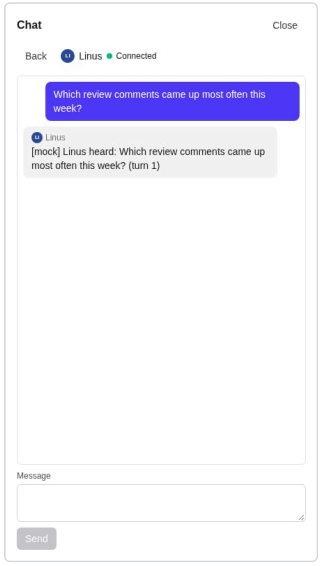


Figure 48 — Chat with Linus (reviewer): a question about review comments and the reply "[mock] Linus heard: … (turn 1)" (Pieter Demo)

3.7 The HR board

The **HR board** shows every agent as a card: role, model, status, current card and six stats against the team. Above the cards are the recommendations. From this page the manager hires, fires and re-roles agents, changes models and applies one-click recommendations.

Page structure: subtitle, filters, recommendations, agent cards

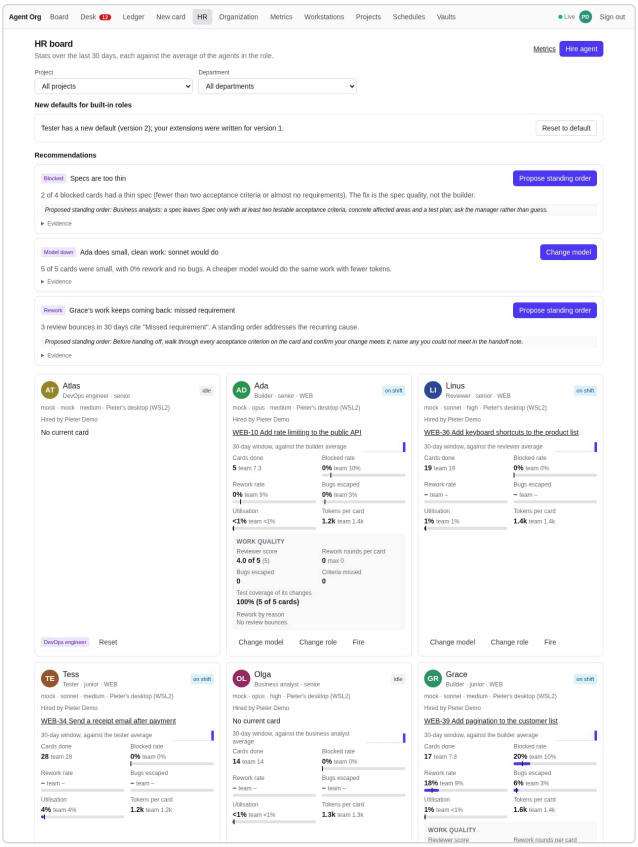


Figure 49 — HR board: subtitle with the 30-day window, Project and Department lists, the "New defaults for built-in roles" banner, three recommendations, then agent cards in three columns (Atlas, Ada, Linus; Tess, Olga, Grace) (Pieter Demo)

The page subtitle shows the time window: *"Stats over the last 30 days, each against the average of the agents in the role."* At the top right are a link `Metrics` and the button `Hire agent`. The list `Project` appears only when you belong to more than one project, and `Department` only when the organization has departments. Below come the role-update banner (if any), the **Recommendations** (if any), then the agent cards in a grid of up to 3 columns. Retired agents are listed in a collapsed line `Retired (n)` at the bottom.

New defaults for built-in roles

If the organization owner extended a built-in role (for example Tester) and the app later ships a newer default of that role, a line appears under *New defaults for built-in roles*: *"Tester has a new default (version 2); your extensions were written for version 1."* (plus how many agents are still on the old version). `Reset to default` opens a dialog that lists what will be removed (extra instructions, skills, standing orders). `Reset and drop extensions` removes them; the role's core rules do not change and its agents move to the new version.

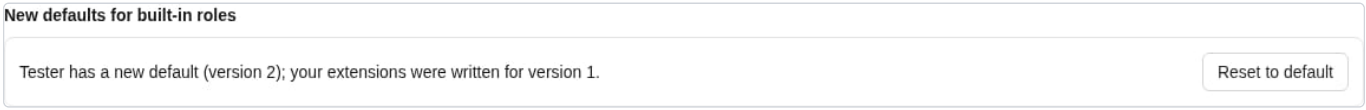


Figure 50 — Role update banner: "Tester has a new default (version 2); your extensions were written for version 1." with Reset to default (Pieter Demo)

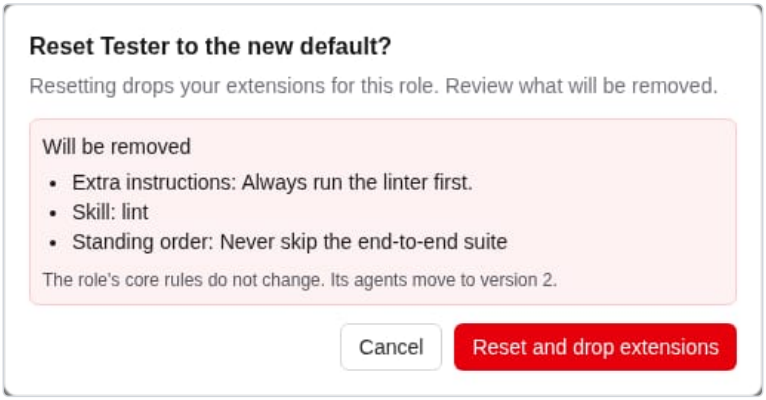


Figure 51 — Reset Tester dialog: will be removed "Extra instructions: Always run the linter first.", "Skill: lint", "Standing order: Never skip the end-to-end suite"; Cancel and Reset and drop extensions (Pieter Demo)

Recommendations: kinds, evidence and one-click actions

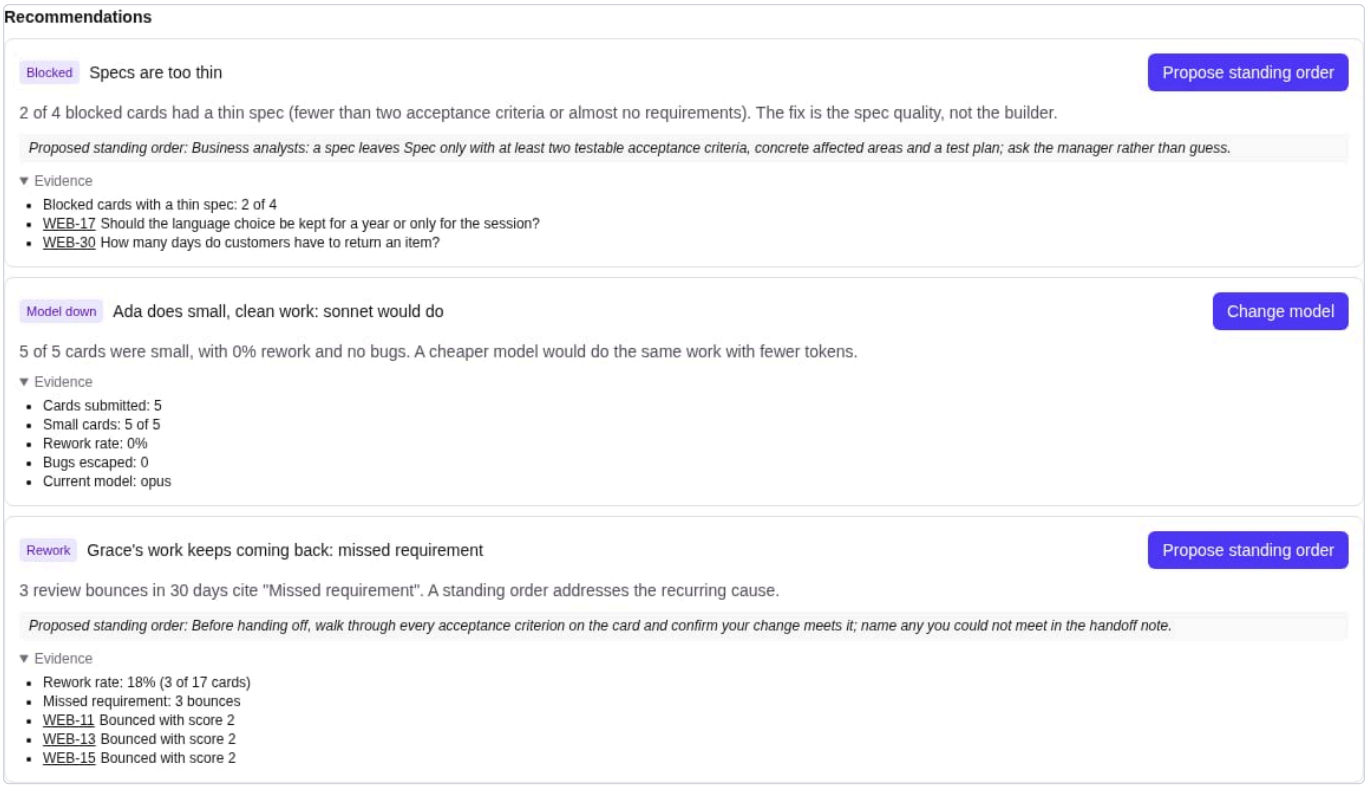


Figure 52 — Three recommendations with Evidence expanded: "Specs are too thin" (Blocked, Propose standing order), "Ada does small, clean work: sonnet would do" (Model down, Change model) and "Grace's work keeps coming back: missed requirement" (Rework, Propose standing order) (Pieter Demo)

Each recommendation has a violet label (`Blocked`, `Rework`, `Model up`, `Model down`, `Split the role`, `Fire or reassign`, `Reviewer`), a title, a summary and an `Evidence` line that expands to the numbers and the cards behind it. All rules look at the last 30 days, ignore the DevOps engineers, and need a minimum number of cards, so a first card never triggers advice. A recommendation that was applied is left out for 30 days. The kinds, what triggers each and its button are:

- **blocked_ledger** (label Blocked): an agent touched at least 3 cards, at least 40% of them raised a question, and at least half of its questions already had a similar ledger entry when asked. Button `Propose standing order` (text: search the ledger before asking).
- **blocked_topic** (Blocked): same blocked-rate test, and at least 3 of the agent's questions share a topic word. Button `Propose standing order`; the text is the last manager answer on that topic, if there is one.
- **blocked_thin_specs** (Blocked): at least 3 cards raised questions and at least half of them had a thin spec (fewer than 2 acceptance criteria, or under 40 characters of requirements). The title names the BA who wrote the specs. Button `Propose standing order`.
- **rework_reason** (Rework): a builder submitted at least 3 cards and at least 3 review bounces cite the same reason (reasons with a standing order: missed requirement, broke convention, no or weak tests, poor structure, changed tests out of scope). Button `Propose standing order`.
- **model_up** (Model up): at least 2 large cards, and at least 50% of them were bounced by review or failed test. Button `Change model` moves the agent one step up.
- **model_down** (Model down): at least 4 submitted cards, rework rate at most 10%, no bugs, and at least 80% small cards (any card that is not large). Button `Change model` moves the agent one step down.
- **split_role** (Split the role): at least 6 submitted cards, at least 2 large and 2 small, and the agent takes both sizes. Button `Split role`: the agent becomes senior on the stronger model and takes large cards only, and a new junior on the lighter model is hired on the same workstation for small cards.
- **fire_quality** (Fire or reassign): at least 3 reviews of its cards, and an average reviewer score of 2.5 or less overall and on large cards and on small cards. Button `Fire`.
- **fire_idle** (Fire or reassign): at least 7 days since hire or window start, on shift less than 2% of the time, no queued work for its role and at least one other agent that could cover the role. Button `Fire`.
- **reviewer_strict** (Reviewer): a reviewer gave at least 4 reviews and at least 80% asked for changes. Advice only (badge `Advice only`, no button).

The button acts at once and the line then shows `Applied`. A standing-order proposal does not change the ledger by itself: it creates a desk item (`Add a standing order: ...`) with an editable text, and the standing order exists only after the owner approves it there. Standing-order proposals can be applied only by the organization owner; the other buttons by the owner and by the manager who hired the agent. Anyone else sees the badge `Not yours to apply`.

Model advice uses the ladder haiku, sonnet, opus and matches by name (a model name containing "sonnet" is on the second step). There is no step above opus or below haiku, and a model whose name contains none of the three gets no model advice.

Agent cards: avatar, role, seniority, model, status, hired by, current card

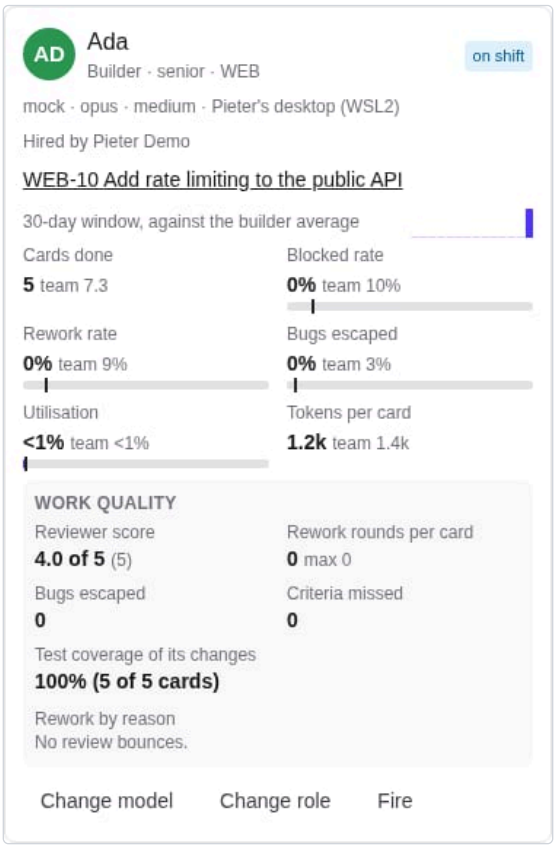


Figure 53 — Agent card of Ada: Builder, senior, WEB; "mock · opus · medium · Pieter's desktop (WSL2)"; hired by Pieter Demo; on shift on WEB-10; six stats against the builder average; Work quality panel; Change model, Change role and Fire (Pieter Demo)

Each card shows:

- **Avatar and name**: the agent's name, then role, seniority (junior or senior) and the project key for an agent of a shared project (for example `Builder · senior · WEB`).
- **Model line**: runtime, model, effort and workstation (`mock · opus · medium · Pieter's desktop (WSL2)`). If the agent runs without an OS sandbox, an amber warning appears below it.
- **Hired by**: the manager who hired the agent.
- **Status badge**: `on shift` (it holds a card), `blocked` (it has no active card and left a card blocked on a question for the manager), `needs rehome` (its hiring manager was removed from the project; a `Re-home` button lets you move it to one of your own workstations, keeping name and history) or `idle`.
- **Current card**: a link to the card it works on, the blocked card marked `(blocked)`, or `No current card`.
- **Six stats**, each with the value, the team value (`team 7.3`) and, for the three rates and utilisation, a bar with a tick at the team value. A small chart at the top right shows cards done per day over the last 14 days. The team value is the average over the agents of the same role. Hover over a stat to see a short meaning. How each is computed, over the last 30 days:
  - **Cards done**: number of cards for which the agent handed off its stage in the window.
  - **Blocked rate**: share of the cards it touched that raised a question.
  - **Rework rate**: share of the cards it submitted to review that were sent back by review at least once.
  - **Bugs escaped**: share of the cards it submitted that later failed a test (or were traced to a later full-suite failure).
  - **Utilisation**: shift time divided by the time available (since hire or since the window start, at least one hour), at most 100%.
  - **Tokens per card**: input and output tokens of its shifts on cards it finished, divided by the cards done.

  A dash means there is no value yet (for example no submitted cards).

Work quality panel for builders

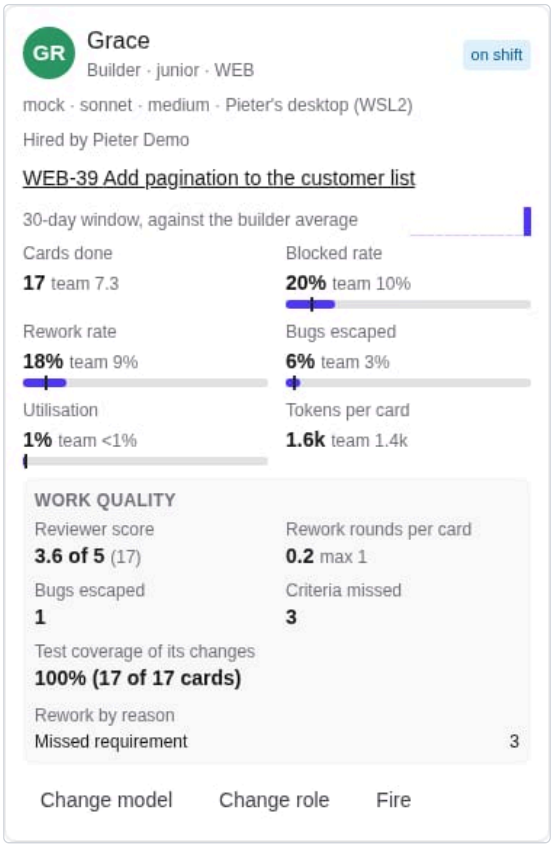


Figure 54 — Grace's builder card with the Work quality panel: Reviewer score 3.6 of 5 (17), Rework rounds per card 0.2 (max 1), Bugs escaped 1, Criteria missed 3, Test coverage 100% (17 of 17 cards), Rework by reason: Missed requirement 3 (Pieter Demo)

Builder cards show a **Work quality** panel below the stats:

- **Reviewer score**: the average review score (1 to 5) of its cards in the window, with the number of reviews in brackets. A dash means no review yet.
- **Rework rounds per card**: average number of review bounces per submitted card, then `max` (the most on one card) and the number of cards with 3 or more.
- **Bugs escaped**: the number (not a share) of test failures and later regressions traced to its cards.
- **Criteria missed**: the number of acceptance criteria the reviewer found unmet when it sent cards back.
- **Test coverage of its changes**: the share of its submitted cards whose change included at least one test file, with the counts (`100% (17 of 17 cards)`). It does not measure line coverage.
- **Rework by reason**: bounce reasons with counts (Missed requirement, Broke convention, No or weak tests, Poor structure, Spec unclear, Changed tests out of scope). A bounce caused only by an unclear spec is marked `(not counted against the builder)`. With no bounces: *"No review bounces."*

DevOps engineer card

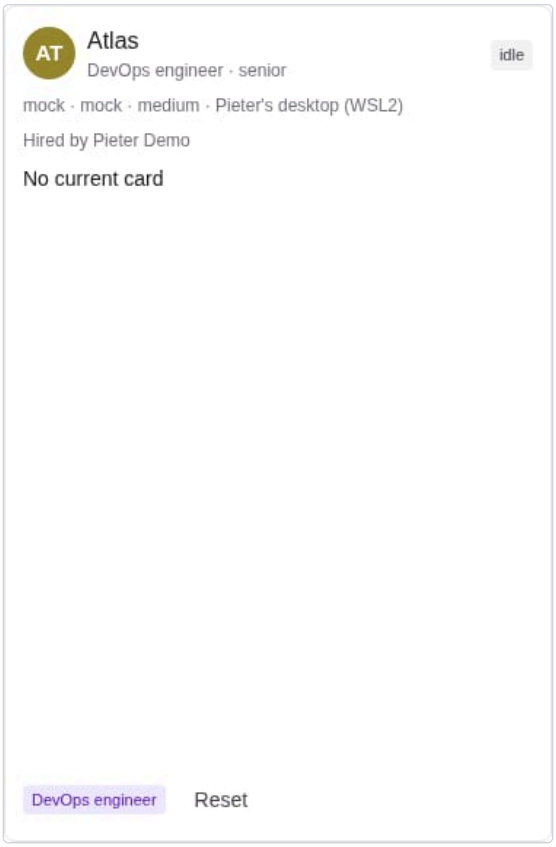


Figure 55 — Lower part of the DevOps engineer card (Atlas): "mock · mock · medium · Pieter's desktop (WSL2)", Hired by Pieter Demo, No current card, the badge "DevOps engineer" and the Reset button (Pieter Demo)

Each paired workstation gets one DevOps engineer automatically. Its card has no stats and no Change model, Change role or Fire buttons: it shows the badge `DevOps engineer` and a `Reset` button (for the owner or the manager who hired it). `Reset` acts at once, without a dialog: it wipes the agent's sandbox like a firing does, but the agent stays. A DevOps engineer cannot be fired and cannot be hired from the Hire form.

Agent actions: Change model, Change role, Fire, Reset

At the bottom of each card are the buttons `Change model`, `Change role` and `Fire`. The organization owner may use them on any agent; a manager of a shared project only on agents they hired themselves. On other agents the card says *"Hired by another manager (read only)"*.

Change model opens a dialog with a free-text field `Model` (the current model is the placeholder) and a list `Effort` (Default, low, medium, high). The change applies from the agent's next shift; it keeps its name, role and history.

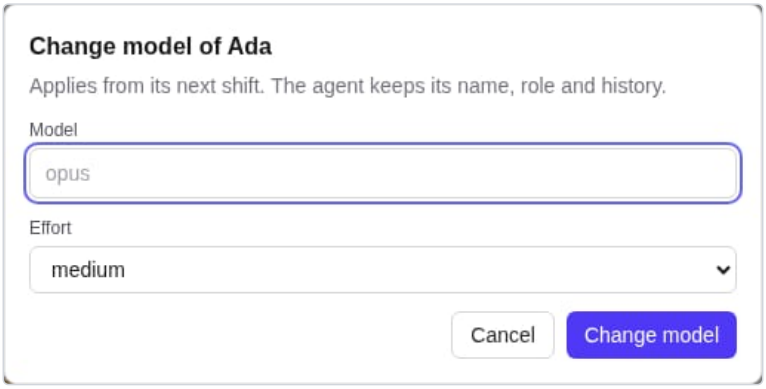


Figure 56 — Change model of Ada: the Model field (placeholder "opus"), Effort "medium", Cancel and Change model (Pieter Demo)

Change role opens a dialog with the lists `Role` and `Seniority`. The agent keeps its name and track record and claims cards of the new role from its next shift.

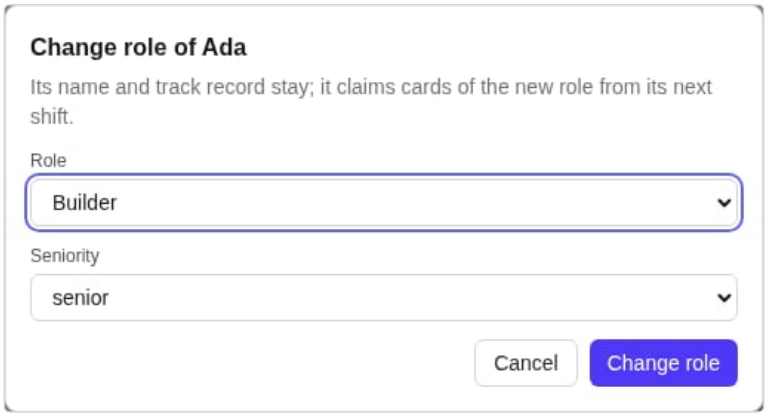


Figure 57 — Change role of Ada: Role "Builder", Seniority "senior", Cancel and Change role (Pieter Demo)

Fire opens a confirmation dialog that lists the cards that will be released. Confirming does this: (1) every card the agent holds goes back to the queue with its last handoff attached and a message in the card thread, (2) the agent is marked retired and is listed under `Retired (n)`, (3) its sandbox is wiped on the workstation, (4) its name is never reused in the organization.

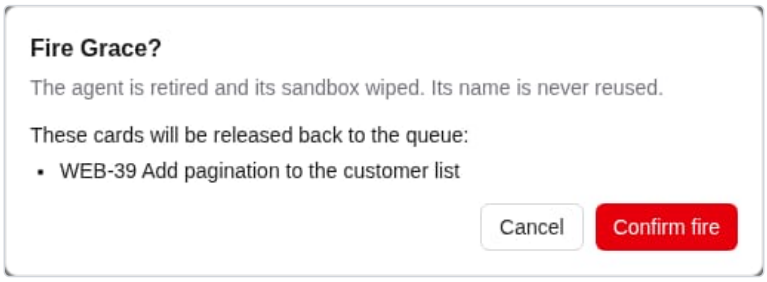


Figure 58 — Fire Grace?: "The agent is retired and its sandbox wiped. Its name is never reused.", the card to be released (WEB-39), Cancel and Confirm fire (Pieter Demo)

Reset (DevOps only): see the DevOps engineer card above.

The Hire agent form: role, model, name, skills, project

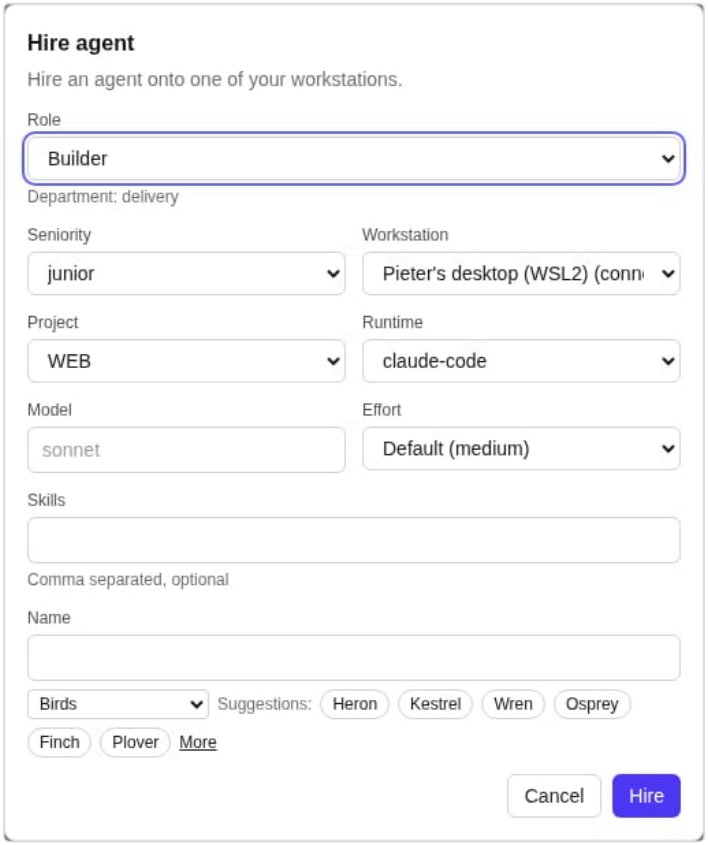


Figure 59 — Hire agent form: Role "Builder" (hint "Department: delivery"), Seniority, Workstation, Project, Runtime, Model, Effort, Skills, Name with the theme list "Birds" and suggestions Heron, Kestrel, Wren, Osprey, Finch, Plover and More (Pieter Demo)

Click `Hire agent` to open the form ("Hire an agent onto one of your workstations."). Fields:

- **Role**: the roles of the catalogue, built-in ones (Business analyst, Builder, Reviewer, Tester, Technical writer, Researcher, Fact checker, Scout, Curator) and approved custom roles. The DevOps engineer is not listed. A hint under the list names the role's department.
- **Seniority**: the levels the role offers (junior or senior).
- **Workstation**: one of your own paired workstations, marked connected or offline.
- **Project**: the project the agent works for (the last project you used is preselected).
- **Runtime**: the role's default runtime and the runtimes available on the chosen workstation.
- **Model**: free text; the placeholder shows the role's default model. Leave it empty to use the default.
- **Effort**: `Default` (shows the role's default, for example medium), low, medium or high.
- **Skills**: optional, comma separated.
- **Name**: required (*"A name is required"*), at most 60 characters, unique in the organization; retired agents keep their names. Under the field a list of name themes (Pioneers, Birds, Trees, Rivers, Planets and moons) and suggestion chips; click a chip to fill the field, or `More` for new suggestions. Suggestions skip names in use and retired names.

There is no hire limit for first runs any more; the HR recommendations replace it. Other messages: *"Choose a role"*, *"Pair a workstation first"*. When the chosen workstation and runtime run without an OS sandbox, the form shows an amber warning and a checkbox `I understand this agent runs without an OS sandbox on ...`. Without the tick, `Hire` shows *"Confirm that you accept hiring onto an unsandboxed workstation"*. From the desk's `Hire tester` button the form opens with the role Tester and the project of the alert preselected.

What Maya sees on the HR board

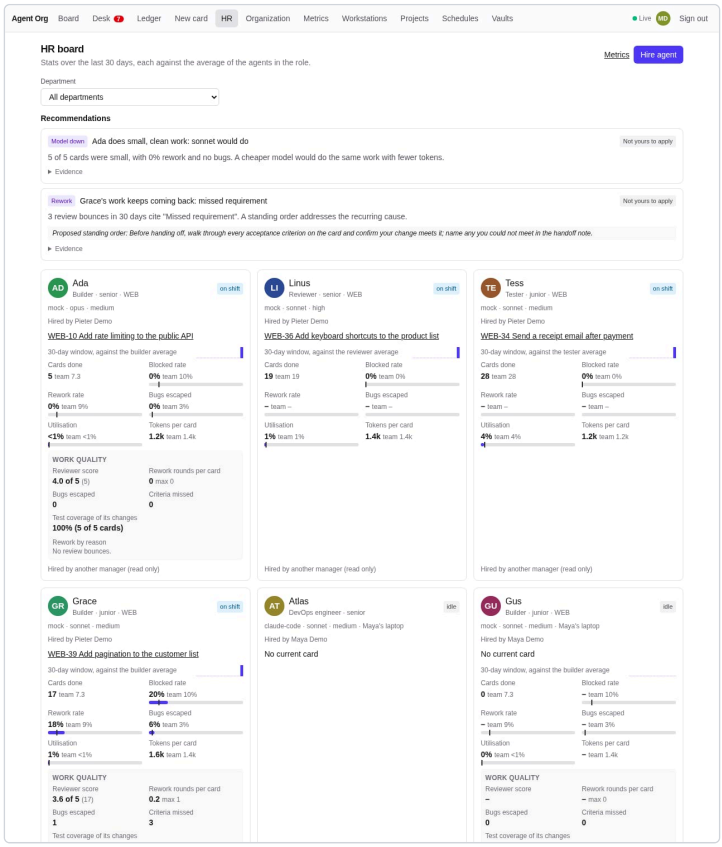


Figure 60 — HR board as Maya sees it: only the Department list; two recommendations about Pieter's agents with the badge "Not yours to apply"; Ada, Linus, Tess and Grace read "Hired by another manager (read only)"; Atlas and Gus were hired by Maya (Maya Demo)

Maya, a manager of the shared project but not its owner, sees the agents of that project. Pieter's agents show *"Hired by another manager (read only)"* and the recommendations about them show `Not yours to apply`. Maya's own agents (Gus, and Atlas, the DevOps engineer of her laptop) have the usual buttons. A manager of a shared project can hire agents for that project onto her own workstations, using the owner's role catalogue. Standing-order proposals are for the organization owner only.

3.8 Metrics

The **Metrics** page displays what the team delivered, where it waited on the manager, and where quality slipped. Metrics are computed from the event log, so rebuilding the projections gives the same numbers.

Page structure: filters and tiles

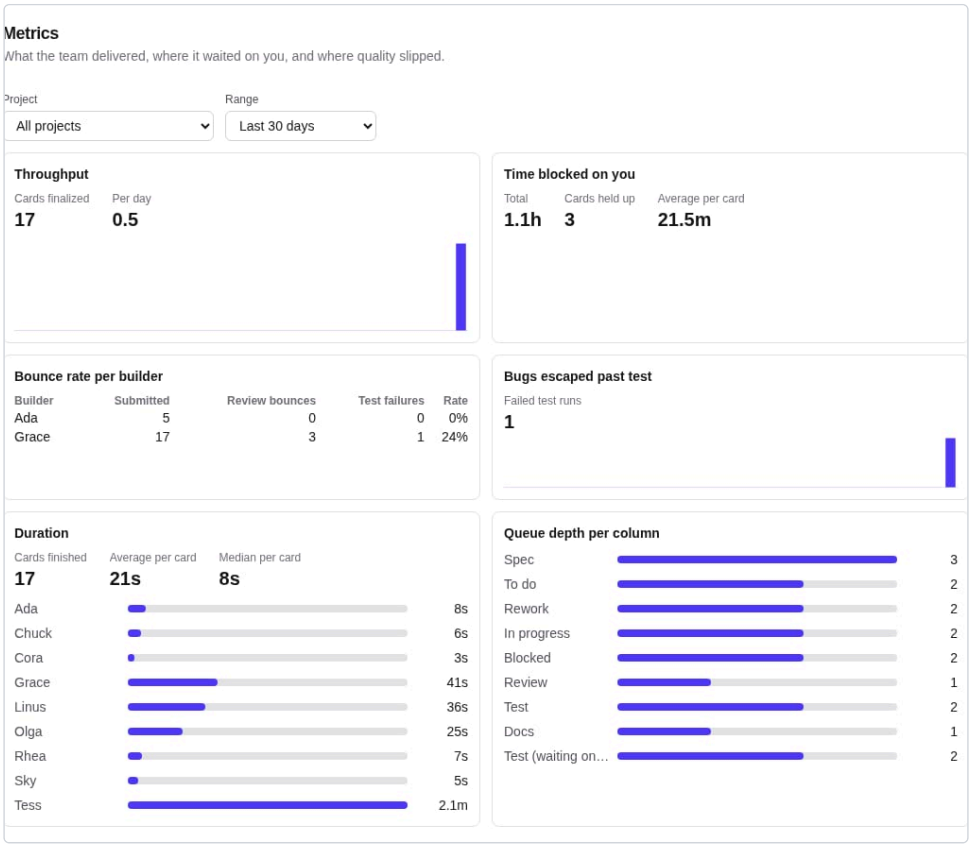


Figure 61 — Metrics dashboard: Project and Range lists ("All projects", "Last 30 days"); tiles Throughput (17 finalized, 0.5 per day), Time blocked on you (11.9m, 3 cards), Bounce rate per builder (Ada, Grace), Bugs escaped past test (1), Duration and Queue depth per column (Pieter Demo)

Two lists filter the page: `Project` and `Range`. Below are six tiles in two columns.

Range selector: 7, 14, 30 or 90 days

The `Range` list sets the time window: Last 7 days, Last 14 days, Last 30 days (the default) or Last 90 days. It applies to all tiles except Queue depth, which is always "now".

Project filter

The `Project` list appears only when you belong to more than one project. `All projects` covers every project you are an active member of.

Throughput

Cards finalized per day over the range. Shows:

- **Cards finalized**: total in the range.
- **Per day**: the total divided by the number of days in the range.
- **Chart**: a bar per day (UTC) in the range; every day appears even if zero.

Only cards with status Done count, and regression cards are left out.

Time blocked on you

How long cards sat in Blocked status, clipped to the range. Time that only waited for a senior agent or the BA (a question answered inside the escalation chain, or still with an agent) is not counted; time with an item on the desk is.

- **Total**: the sum of the blocked time.
- **Cards held up**: how many different cards were blocked.
- **Average per card**: total divided by cards held up.

Bounce rate per builder

For each agent that submitted a card to review in the range, a table shows:

- **Builder**: agent name.
- **Submitted**: cards handed to review.
- **Review bounces**: how many times review sent cards back (a card bounced twice counts twice).
- **Test failures**: how many test failures were traced to its cards.

- **Rate**: the share of submitted cards that were bounced or failed test at least once.

Sorted by name. When nobody submitted a card the tile says *"No builder submitted a card in this range."*

Bugs escaped past test

Test runs that failed (feature E2E, full suite, or regression). Shows:

- **Failed test runs**: total in the range.
- **Chart**: failed test runs per day (a bar per day; zero days are included).

Each failed run counts once, however many tests failed in it.

Duration

How long cards take from creation to Done (finalized), and how much shift time each agent used.

- **Cards finished**: cards finalized in the range (regression cards left out).
- **Average per card**: mean time from creation to finalized.
- **Median per card**: the middle value (half are faster, half slower); better than the mean when a few cards take much longer.
- **Per agent**: bars titled "Shift time per agent": the total length of each agent's ended shifts in the range, sorted by name.

Queue depth per column

Right now, how many cards are in each board column. Only open cards count (not Done or Cancelled; drafts count in Spec). Shows:

- **Column name**: Spec, To do, Rework, In progress, Blocked, Review, Test, Docs, Finalize, and `Test (waiting on bug)`, plus the columns of other workflows.
- **Chart bar**: proportional to the count; the number is on the right.

Code-board columns come in board order, `Test (waiting on bug)` and the columns of other workflows after them. A column with no cards does not appear; with no open cards the tile says *"No open cards."*

> **Good to know about the demo and "waiting on"**
> All agents in the demo run on the MOCK runtime, so chat replies, shift logs and notes are scripted (`[mock]`, "mock peer", "mock ba"); real agents write real text. The large-risk card WEB-35 is at Finalize with status waiting, because it waits for the manager's approval of the merge. The board puts every waiting card in the column `Test (waiting on bug)`, and the Queue depth tile uses the same column name, although WEB-35 waits for the manager and not for a bug.

4 Intake, outbox and external systems

Work arrives from tickets in Jira or GitHub. The BA reads the ticket through the integration, drafts cards with full requirements in a conversation with the manager, and posts updates back. Nothing leaves the organization without the manager's approval.

4.1 Intake from a ticket

Open **Intake** from the desk and type a Jira key, a GitHub issue (URL or `owner/repo#12`) or a free-text description of the need in the field **Ticket or need**, then press **Start intake**. The BA reads the ticket, talks to the manager in a conversation, drafts cards with all required fields and a completion action, and specifies them so they enter the build queue (the To do column).

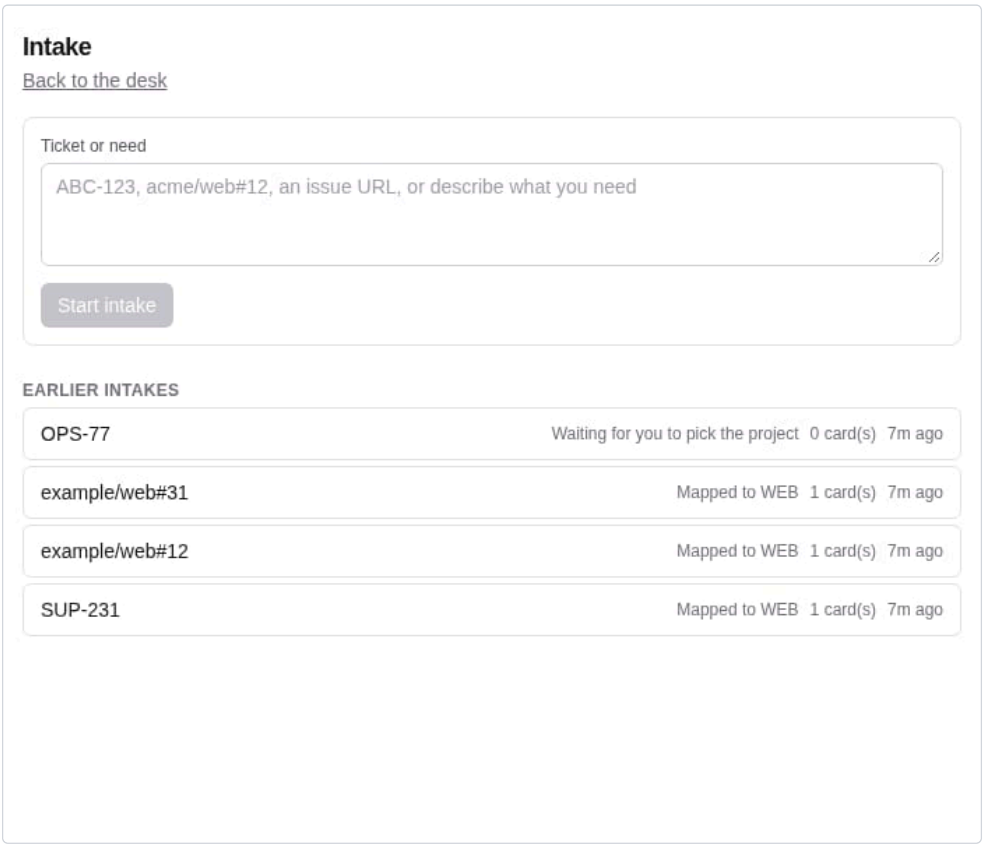


Figure 62 — The Intake page: the empty form with Start intake, and the list of earlier intakes (OPS-77 waiting for a project, example/web#31, example/web#12, SUP-231) (Pieter Demo)

As you type, a hint line under the field says how the input is read: "Jira ticket SUP-231: the BA reads it on your workstation (read-only)", the same for a GitHub issue, or "Free text: the BA works from your description." A free-text intake has no ticket, so its cards carry no external reference and no updates can be posted for them (see 4.3).

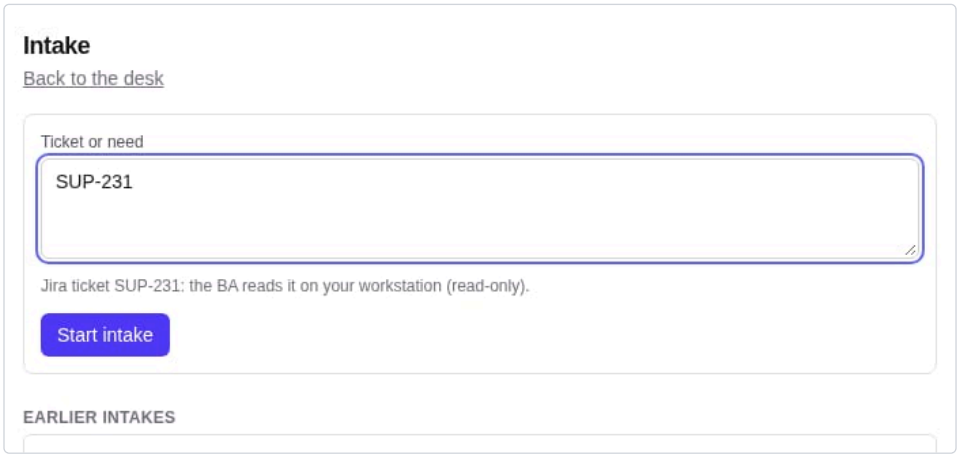


Figure 63 — Typing the Jira key SUP-231: the hint line names it as a Jira ticket (Pieter Demo)

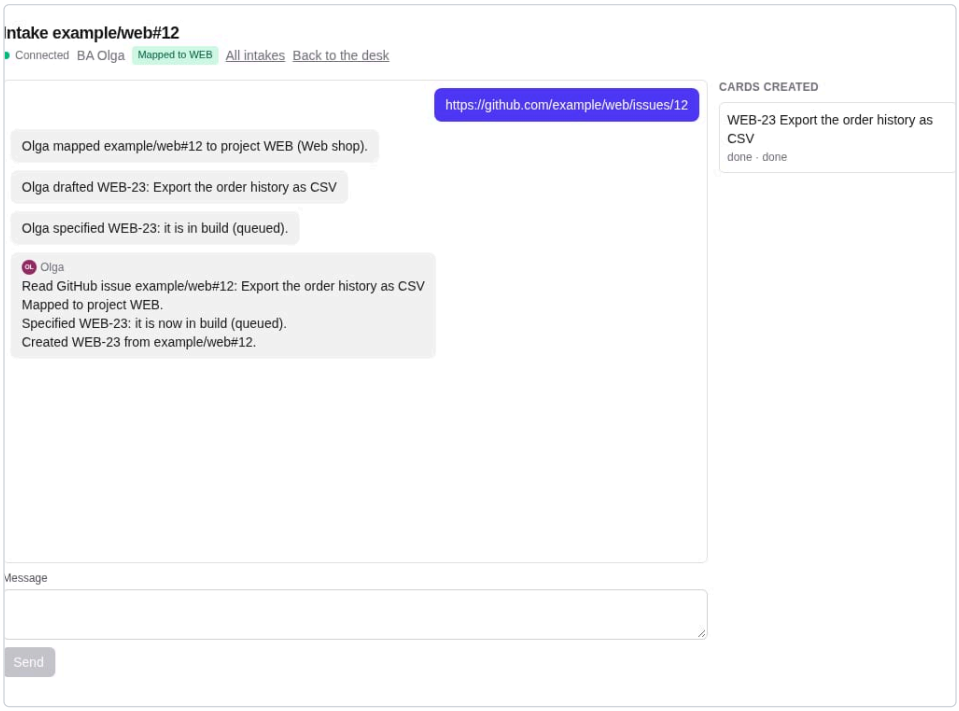


Figure 64 — Intake of the GitHub issue example/web#12: the pasted issue URL, mapped to WEB, card WEB-23 created (Pieter Demo)

A pasted GitHub issue URL, `owner/repo#12`, a Jira browse URL or a Jira key such as `SUP-231` is recognized as the ticket. The BA's own messages in the figure ("Read GitHub issue ...", "Mapped to project WEB.") are scripted by the mock runtime; a real BA writes its own.

What the BA does, in order:

Intake workflow

1. **Read the ticket:** a Jira ticket is read by the BA itself with the Jira skill and the Jira tool of its runtime (read-only: it never creates, edits, comments on or transitions a ticket). A GitHub issue is read with `aom source read`, which uses the `gh` login on the workstation. Free text needs no reading.
2. **Map to a project:** the BA picks the project from your project list (the Jira project key, repositories and names help, and it searches the ledger for earlier mapping decisions). If no project fits, it does not guess: it raises an unmapped-ticket question on the desk (see 4.2) and the conversation continues after you answer.
3. **Investigate and ask:** the BA reads the project's PROJECT.md, the repository (read-only) and the ledger, and asks you in the conversation only what these sources cannot answer.
4. **Draft cards:** one card per independently deliverable change, with the fields a builder needs: Context, Requirements, Acceptance criteria, Affected areas, Constraints, Test plan, a Completion action (pr, push_branch or merge), Docs needed, Priority and the ticket as external reference. The card type is feature, bug, support, refactor or docs.
5. **Specify:** the BA runs `aom card specify`. The server refuses a card with missing fields and says what is missing; the BA fixes it and tries again. A specified card enters the build queue and the BA tells you which cards it created.

The intake view has the header "Intake" plus the ticket key, a connection dot, "BA" and the BA's name, the status badge, and links **All intakes** and **Back to the desk**. Below it the conversation with the BA is on the left (with a Message field and **Send**) and the list **Cards created** is on the right: each entry shows ref, title, stage and status. Until the BA has drafted a card it reads "None yet. The BA drafts them as you talk."

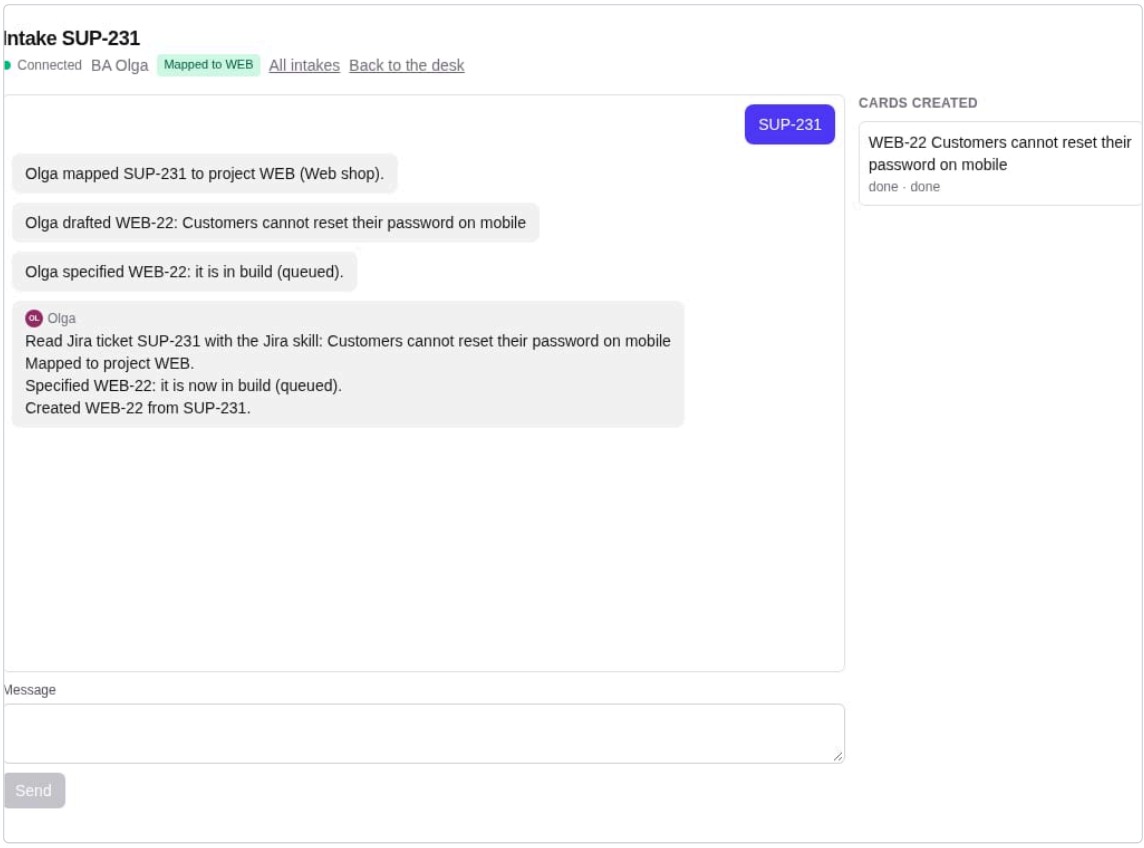


Figure 65 — Intake SUP-231, mapped to WEB: the conversation with Olga (BA) and the card WEB-22 it created (Pieter Demo)

The status badge shows one of three texts:

- **Mapped to WEB** (green; the key of the project): the BA found the project and is drafting or has drafted cards.
- **Waiting for you to pick the project** (amber): the BA could not map the ticket and asked on the desk. An amber banner under the header says so and its link **Pick the project** goes straight to the desk question. Once you pick, the status changes to mapped and the BA carries on in the same conversation.
- **Open**: the intake has been started and the BA has not mapped it yet.

Earlier intakes appear on the Intake page showing the ticket key ("No ticket" for free text), the status text, the number of cards and the age. Clicking a row opens that intake's conversation.

> **How the BA reads the ticket**
> **Jira:** the BA reads the ticket with the Jira skill and whatever Jira tool its runtime offers (an Atlassian MCP server, a CLI or the REST API). The credentials are in that tool's configuration on the workstation; the runner holds no Jira credentials for reading. **GitHub:** the runner runs `gh issue view` with the `gh` login of the workstation. **Free text:** you type the need directly; no ticket is read. In all cases nothing is written to the ticket during intake.

4.2 Tickets that match no project

When the BA cannot determine which project a ticket belongs to, it raises an unmapped-ticket question on the desk ("Which project does OPS-77 belong to?"). You pick a project and press **Map to project**. The choice is always saved in the ledger as a decision.

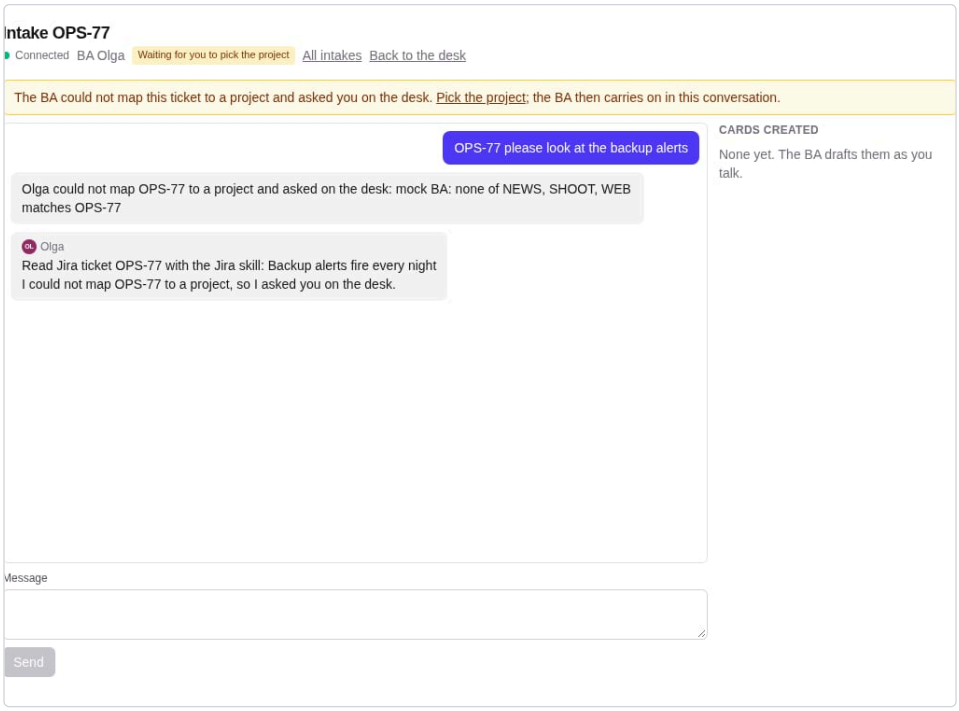


Figure 66 — Intake OPS-77 waiting for the manager to pick a project: amber banner with the link Pick the project, no cards yet (Pieter Demo)

On the desk the item carries the violet badge "Unmapped ticket" and "Non-blocking". It shows the asking agent (Olga, BA), what it was doing, the question, a link **open the intake**, and one radio button per project (here NEWS, SHOOT and WEB). Buttons: **Map to project** and **Dismiss**. If you press Map to project without choosing, it says "Pick the project this ticket belongs to".

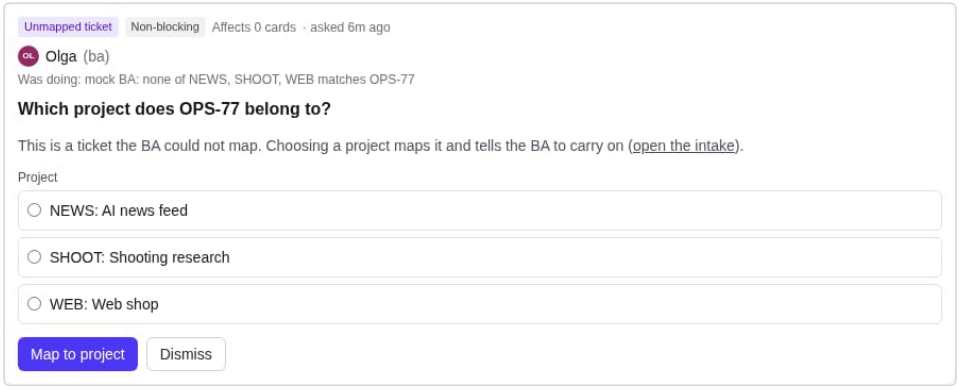


Figure 67 — Unmapped ticket desk item for OPS-77: choose NEWS, SHOOT or WEB and press Map to project (Pieter Demo)

> **Mapping rule**
> Map to project tells the BA which project to use and records a ledger decision titled "<ticket> belongs to <project>". There is no separate choice to make. When the BA meets a similar ticket later it searches the ledger for earlier mapping decisions and can use them; the ledger entry is a record the BA reads, not an automatic rule that skips the question.

4.3 The outbox: updates back to a ticket

When a card that came from a ticket reaches Finalize, the BA drafts an update for that ticket before it finalizes the card (one draft per card and kind). The draft lands on the desk for approval. Only the card's owner can approve it, and only an approved draft is posted, by the owner's own workstation with the owner's own credentials. You can also write an update yourself in the Updates panel of the card (see 4.4); it goes to the desk like any other draft. A card with no ticket reference cannot have updates.

Kinds of outbound update:

- **Comment:** a comment on the ticket (Jira or GitHub).
- **Resolution:** the work resolves the ticket. On GitHub the issue is closed with the text as its closing comment. On Jira the text is posted as a comment only; no transition is sent, so you close the ticket in Jira yourself.
- **Design decision** (kind `decision`): a design decision you made. It is posted as a comment, on both systems.

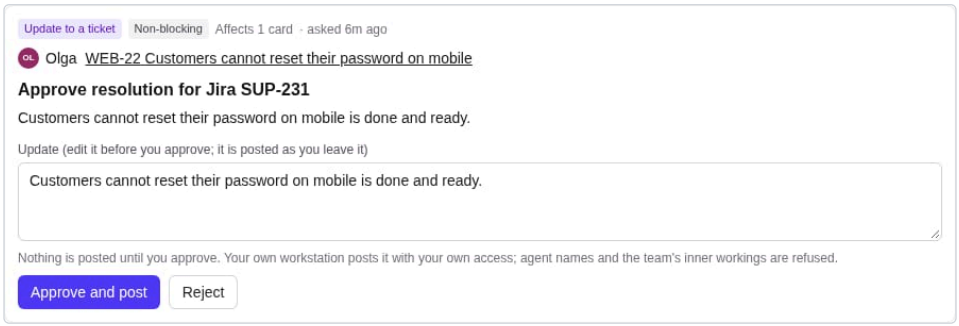


Figure 68 — Outbound approval: Olga's draft "Approve resolution for Jira SUP-231" for card WEB-22, with the buttons Approve and post and Reject (Pieter Demo)

The approval item on the desk has the violet badge "Update to a ticket" and shows the title "Approve <kind> for Jira|GitHub <key>", the card link and the draft in an editable text area. You may approve it as it is (**Approve and post**), edit it and approve (**Approve and post my edit**), or reject it (**Reject**).

What happens after you approve, in order

1. The server sends the approved text to one of your own workstations (a connected one first). Only the card owner's workstation is used, so the credentials are always the owner's.
2. The runner scrubs the text once more (see 4.4). A draft that leaks is refused here even if it was approved.
3. The runner posts it exactly as approved. GitHub: `gh issue comment`, or `gh issue close --comment` for a resolution, with your `gh` login. Jira: a comment through the Jira REST API with the API token in the runner's `config.json` (`jira: baseUrl`, `email`, `apiToken`).
4. On success the card's Updates panel shows "Posted" with a link to the result and the thread gets a message. If the same text is already on the ticket (a retry after an unclear failure), it is not posted twice.
5. On failure the desk gets a non-blocking "Posting failed" item with the error, the thread gets a message, and the approved text is kept. The owner presses **Retry posting** on the card's Updates panel once the cause is fixed.

4.4 Edit, reject, failed posts and blocked drafts

Edit the draft: the text area is editable and the update is posted as you leave it. When you change it, a **Your changes** section appears showing the word diff (added words in green, removed words in red). The button changes to **Approve and post my edit**. An edit goes through the scrubber again; if it names internal details the approval is refused with the reason.

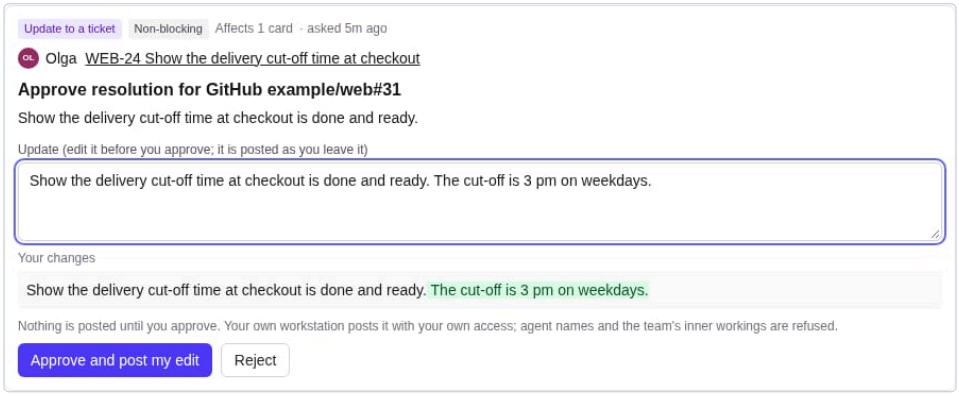


Figure 69 — Approval for GitHub example/web#31 (card WEB-24): the manager added a sentence; the diff shows it in green and the button reads Approve and post my edit (Pieter Demo)

Reject: click Reject and an optional field "Why not (optional, kept on the card)" appears, with the button **Reject, never post**. The draft is never posted and the card's Updates panel shows it as "Rejected, never posted" with your reason.

Posting failed: if the post does not work (no `gh` or Jira credentials on the workstation, the system answers with an error, a network problem), a non-blocking desk item with the badge "Posting failed" appears. Its title is "Could not post the update to GitHub example/web#40" (or Jira) and it shows the error. The item has the buttons **Acknowledge** and **Dismiss**; the retry is not on the desk but on the card: the **Retry posting** button in the card's Updates panel, which only the card owner sees.



Figure 70 — Posting failed for card WEB-28: "gh issue comment failed: HTTP 502: Bad Gateway", with Acknowledge and Dismiss; the text says to retry from the card's updates (Pieter Demo)

Blocked drafts: a scrubber (a leak check) runs on every draft before it reaches the desk, on every edit you make before approving, and once more on the runner before posting. A draft by the BA that names internal details is recorded as blocked, never reaches the desk, and the BA gets the reason and is told to rewrite it without them. The card's thread and the Updates panel show the blocked draft. If you write a draft yourself and it is refused, you see the reason on the spot and nothing is saved.

What the scrubber refuses

- The name of any agent of the organization, retired agents included (whole word, case-insensitive; names that are also everyday words, such as Grace or Sky, only when written with a capital letter).
- The words agent or agents.
- Review and bounce talk: "bounce", "review comments", "reviewer feedback", "changes requested", "rework".
- Decision desk, "desk item", "the desk", "question thread", escalation, ledger, standing orders.
- Role titles: builder, reviewer, business analyst, BA, scout, curator.
- The product name (Agent Org Manager, AOM) and internal branch names starting with `aom/`.

The check is pattern based: it looks for these words and phrases, it does not judge meaning. Write the update in your own voice and say what was done, not how the team did it.

Updates panel on the card: a card that came from a ticket (or has drafts) shows a panel **Updates to <ticket> (n)** with the history of every draft. Each entry shows a status badge, the kind and the system and ticket key ("Resolution for GitHub example/web#12"), who drafted it, who approved it, and an Edited badge when the manager changed the text. An edited update is shown as the diff and "Posted as edited"; a posted update shows "Posted:" with a link; a failed one shows the error and **Retry posting**; a blocked one shows the words that were refused. The panel ends with a form "Write an update yourself" (kind and text, button **Draft update**); it goes to the desk like any other draft.

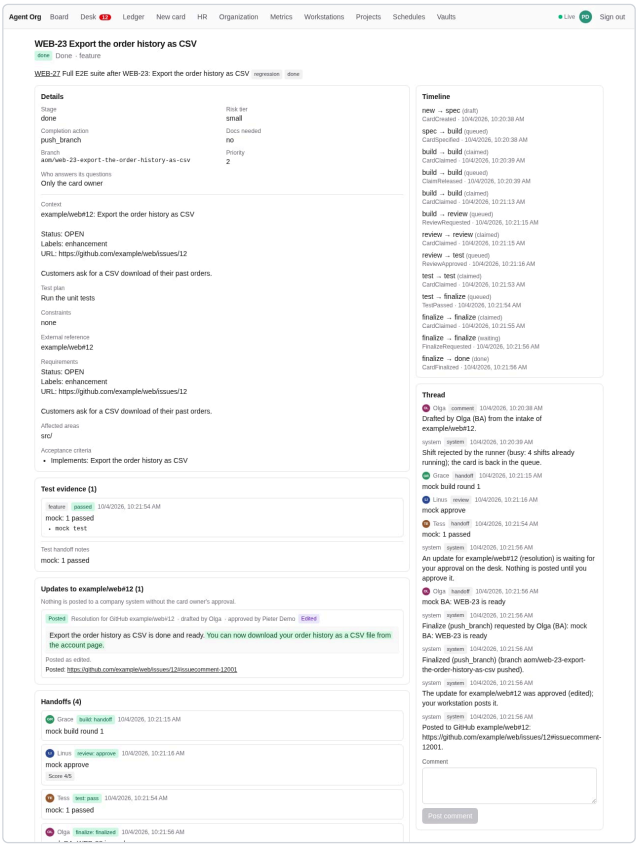


Figure 71 — Card WEB-23 (from GitHub issue example/web#12): the Updates panel shows the resolution posted as edited, with the added sentence marked, and the link to the GitHub comment (Pieter Demo)

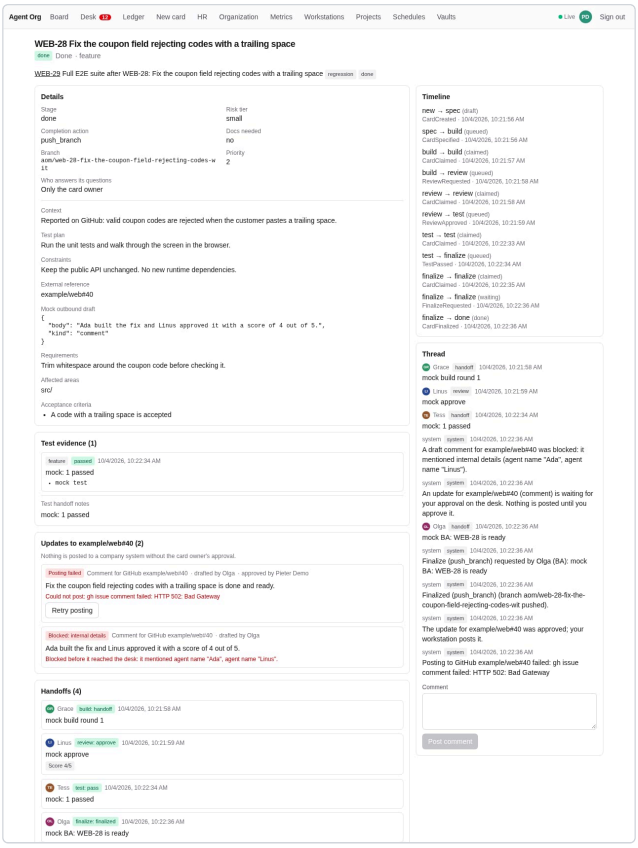


Figure 72 — Card WEB-28 (example/web#40): the Updates panel shows a failed post with Retry posting and a draft blocked because it named Ada and Linus (Pieter Demo)

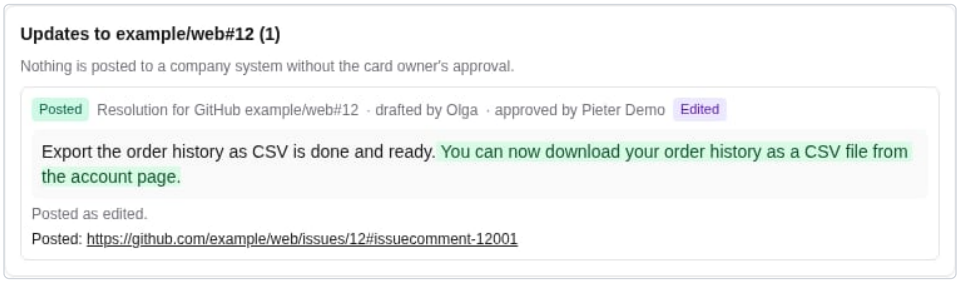


Figure 73 — Updates panel of WEB-23: one posted, edited resolution (Pieter Demo)

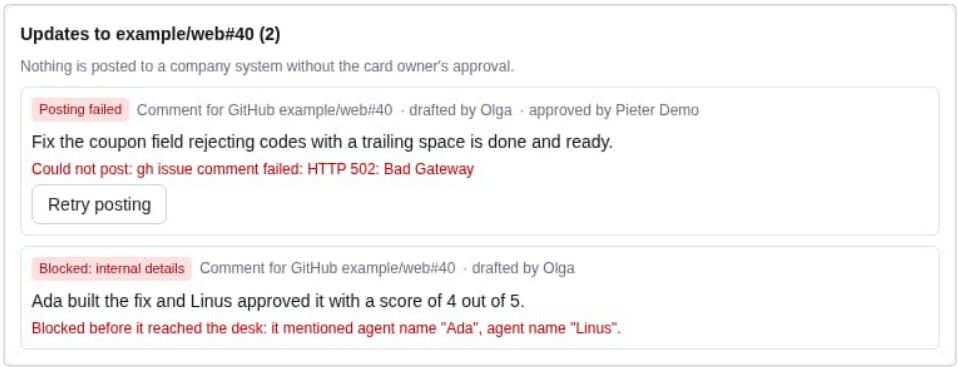


Figure 74 — Updates panel of WEB-28: "Posting failed" with the error and Retry posting, and "Blocked: internal details" with the refused agent names (Pieter Demo)

Status badge	Meaning
Waiting for approval	On your desk, not yet decided.
Approved, waiting to be posted	Approved; the workstation has not reported the result yet.
Rejected, never posted	You rejected it. The reason, if any, is shown.
Posted	Sent to the external system; the reference or link is shown.
Posting failed	Approved but the post failed. The error is shown; the owner can press Retry posting.
Blocked: internal details	The scrubber refused the draft before it reached the desk. The refused words are shown.

4.5 Guardrails and credentials

What never leaves without approval is enforced in three places:

- **Scrubber:** blocks drafts with internal details on the server (every draft and every edit) and again on the runner before posting.
- **Manual approval:** the card owner approves every draft on the desk before it is posted. Nobody else can approve it.
- **Read-only intake:** during intake the BA only reads tickets; it never writes to Jira or GitHub.

Credentials stay on the workstation:

- Reading a Jira ticket uses the Jira tool configured in the agent's runtime on the workstation.
- Posting to Jira uses the API token in the runner's `config.json` (`jira`: `baseUrl`, `email`, `apiToken`, file mode 0600).
- GitHub reading and posting use the `gh` login of the workstation.
- The server holds none of these secrets and makes no calls to Jira or GitHub. Only the workstation that posts an update uses them, and only after the owner approved it on the desk.

Rule: the credentials of the card owner's workstation are the ones used. The ticket shows the owner's own GitHub login or Jira account as the poster, not a shared bot account.

> **Verified**
> Every outbound draft is approved by the card owner before posting. A failed post appears on the desk with the error, and the approved text is kept so the owner can retry it from the card.

5 Testing, review and integration

Code stays quarantined in its card's worktree until a reviewer approves it and a tester confirms it works. Each stage has its own rules and gates. Failures loop back as new work without blocking anyone.

5.1 Risk tiers and review

Every card that is handed off for review gets a risk tier, **small** or **large**. The runner computes it from the builder's diff against the working branch when the builder hands off. The tier decides how many reviewers must approve and whether the merge needs your approval.

Tier	Trigger	Review requirement	Merge approval
Small	Fewer than 200 changed lines (added plus removed), no changed path matching the project's architectural globs, and the card type is not refactor	One senior reviewer	None: the merge runs without a desk approval
Large	200 or more changed lines, or any changed path matching an architectural glob, or card type refactor	Two different senior reviewers	You approve the merge on the desk (only when the completion action is merge)

The **architectural globs** are a setting of the project (Project settings, "Architectural globs", one glob per line). One matching file makes the card large, whatever the size of the diff. Bug cards created by a failed test use a shorter route: one reviewer approval, whatever the tier.

Overriding the tier: the tier can be set on a card by its manager through the card API (`PATCH` with `riskTier`); a tier set this way is kept and the runner's computation no longer replaces it. The web page shows the tier in the Details panel and on the card tile but has no control to change it.

Review flow: when a card reaches Review it is claimed by a senior reviewer agent (reviewers are always senior). The reviewer reads the card, the diff, the tests and the acceptance criteria, and reports an outcome with `aom review approve` or `aom review changes`. Both carry a score from 1 to 5; a request for changes also carries one or more reasons (missed requirement, broke convention, weak tests, poor structure, spec unclear, out of scope test change) and the criteria that were missed. There are no Approve or Request changes buttons in the web UI: reviewers are agents.

- **Changes requested:** the card goes back to Build and shows in the Rework column with a Rework counter increased by one. The original builder is preferred for the rework.
- **Approved, small:** the card goes to Test.
- **Approved, large:** the first approval puts the card back in the Review queue; a second, different reviewer must approve before it goes to Test. A request for changes from either reviewer sends it back to Build and the approval count starts again at zero.

If a builder changed existing unit tests outside the card's scope, the diff is flagged and the server refuses an approval: the reviewer has to request changes with the reason "out of scope test change".

> **Review score and work quality**
> The reviewer scores the card from 1 to 5. Scores and rework reasons feed the work quality figures on the builder's HR card and the HR board's recommendations, for example a standing order when the same reason keeps coming back, or a stronger model when large cards often bounce.

5.2 The tester loop and evidence

Before the tester starts, the runner rebases the card branch onto the working branch (see 5.4) and checks the test environment (see 5.8). If the environment is not ready, the shift does not start and the DevOps engineer of that workstation is asked to fix it.

Tester works the card. A tester claims the card at Test. It writes Playwright end-to-end tests for the new behaviour in the project's E2E paths, runs them together with the existing E2E tests the change can affect, and commits them on the card branch. The test plan on the card is the contract: every acceptance criterion needs at least one check that was run and passed.

Test evidence: the tester reports `aom test pass` with the evidence (what ran, the commands, an output excerpt) or `aom test fail` with a bug title, reproduction notes and evidence. These are agent commands, not buttons in the web UI. On a pass the card goes to Docs (if Docs needed is set) or to Finalize. On a fail a bug card is created (see 5.3).

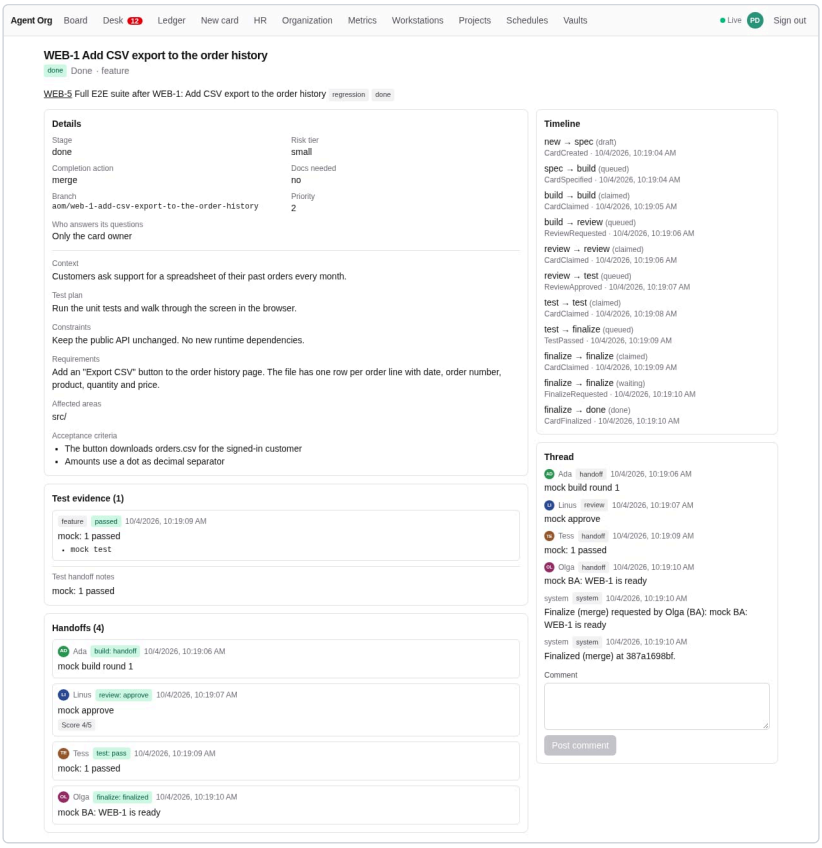


Figure 75 — Finished card WEB-1: the Test evidence panel shows one feature run that passed, with its command, and the tester's handoff note; the Handoffs panel lists Ada, Linus (score 4/5), Tess and Olga (Pieter Demo)

Evidence panel on the card: "Test evidence (n)" lists every test run with a badge for the kind (feature, full suite or regression), a platform badge when the run has one (suite and regression runs), the result (passed or failed), the time, a summary, the commands that ran and an output excerpt. Below that, "Test handoff notes" shows the tester's notes. In the demo the texts are scripted by the mock runtime ("mock: 1 passed", command "mock test").

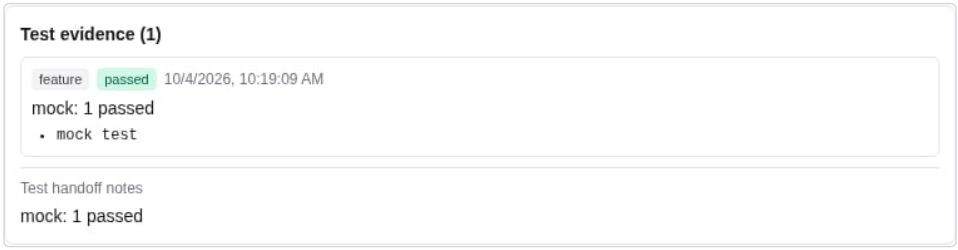


Figure 76 — Test evidence panel of WEB-1: one feature run, passed, and the test handoff note (Pieter Demo)

> **Tester scope guard**
> A tester may only change files that match the project's **E2E globs** (Project settings, "E2E globs", one glob per line). If none are configured, a tester may change nothing. When the tester reports, the runner's post-shift guard checks the worktree; if anything outside the E2E paths changed (application code, unit tests, configuration), the report is rejected with the guard's reason and the out-of-scope edits are discarded instead of committed. The same guard stops builders from writing E2E tests and reviewers from changing any file.

5.3 Bug cards and waiting on a bug

When a tester reports `aom test fail` (a genuine product bug, not a flaky test of its own, which it must fix itself), the server creates the bug card automatically from the tester's title, notes and evidence. The tester does not build it. The bug card uses the internal "Code bugfix" workflow: it starts in Build, shares the parent's worktree and branch, and the original builder is preferred for it (any free builder can take it if the original is busy). It needs one reviewer approval, then it is done. The parent card gets the status **waiting**, stays in the Test stage and shows in the board column "Test (waiting on bug)".

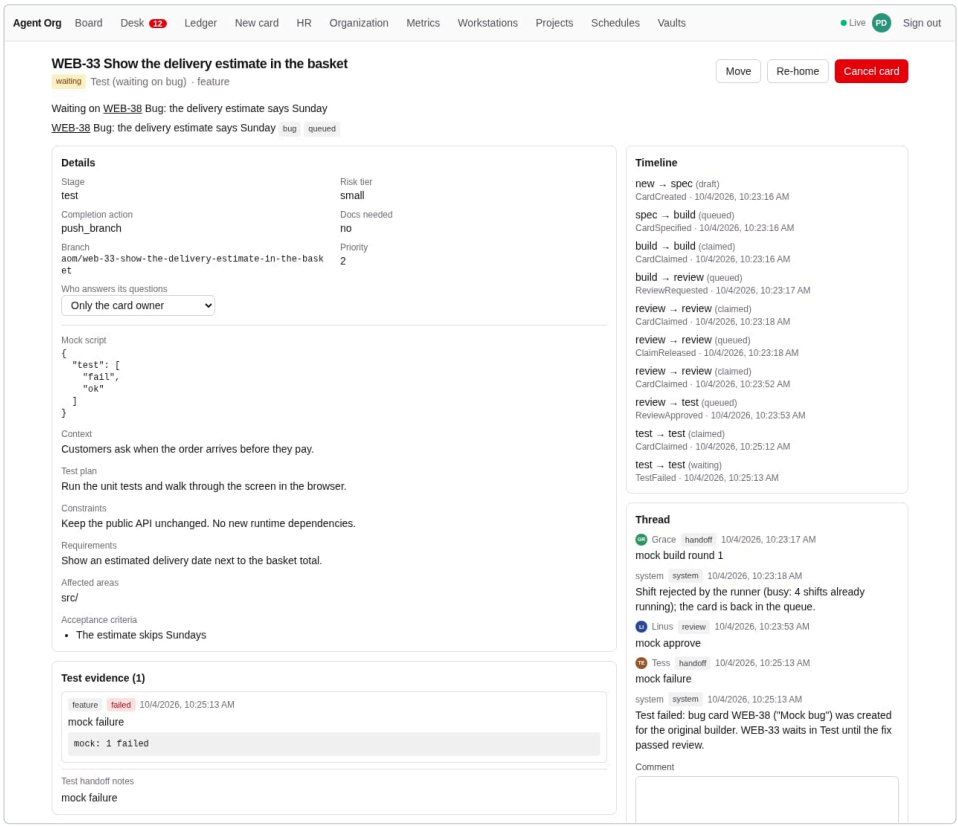


Figure 77 — Card WEB-33 waiting: the line "Waiting on WEB-38 Bug: the delivery estimate says Sunday", the failed test run in the evidence panel, and the system message that WEB-38 was created for the original builder (Pieter Demo)

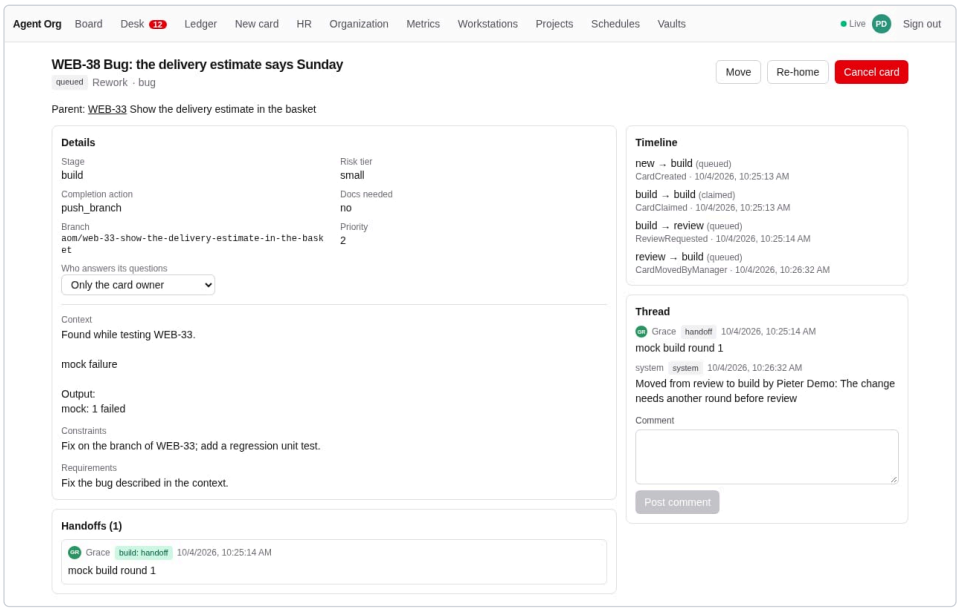


Figure 78 — Bug card WEB-38: type bug, a "Parent: WEB-33" link, and the bug description from the failed test in its Context (Pieter Demo)

The waiting card shows a line **Waiting on WEB-xx** with a link to the bug, and the bug card shows **Parent: WEB-xx**. When the bug card is approved in review, the server puts the parent back in the Test queue ("WEB-33 is back in the Test queue") for a tester to run again. A bug card in the Build stage is shown in the Rework column because its type is bug.

> **Good to know**
> In the figure the thread of WEB-38 has a message "Moved from review to build by Pieter Demo": a manager moved the bug card back by hand, so it sits in Build again and the board shows it in the Rework column. Without such a move the bug card goes from Build to Review to Done.

> **Why bugs are cards**
> A bug card is ordinary work on the board: it has a builder, a review and a ref of its own. This keeps the fix visible and lets you see and prioritize it like any other card.

5.4 Rebase conflicts

Before a tester shift starts and again before the completion action runs, the runner does `git fetch` and rebases the card branch onto the working branch. If this conflicts, the runner lists the conflicted files, aborts the rebase (it never leaves a half-done rebase behind) and never resolves the conflict itself. The card goes back to Build, shows in the Rework column with the Rework counter increased by one, and the conflicted files are listed in a system message on the thread.

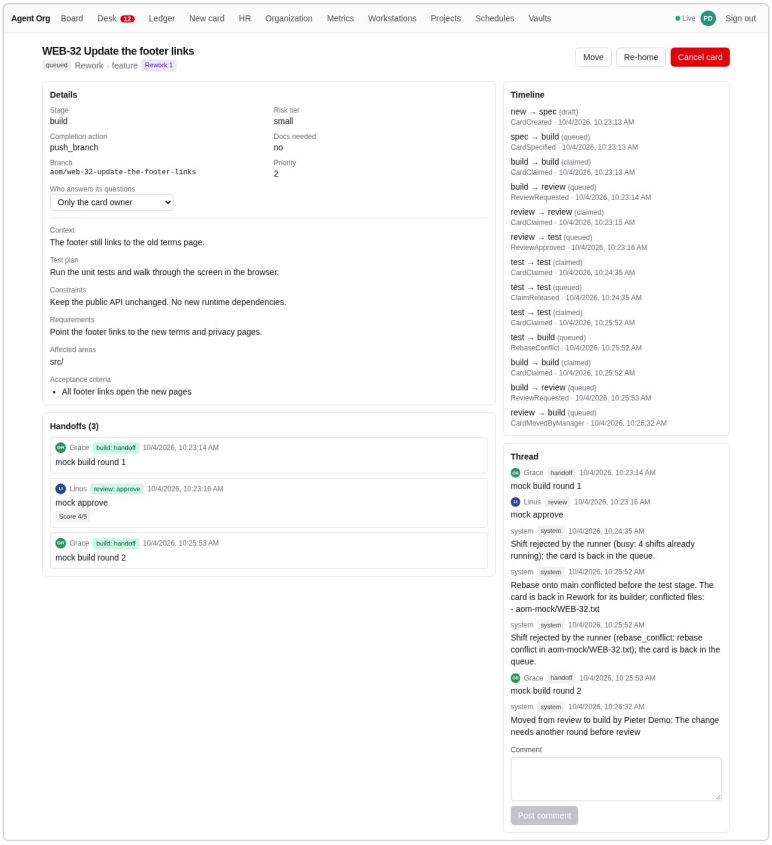


Figure 79 — Card WEB-32 after a rebase conflict: Rework 1, the thread message "Rebase onto main conflicted before the test stage … conflicted files: aom-mock/WEB-32.txt", and Grace's second build round (Pieter Demo)

Conflict handling

1. The runner detects the conflict in the pre-test or pre-finalize rebase (a merge conflict while merging at Finalize is treated the same way), aborts the rebase and reports the conflicted files.
2. The card moves to Build (Rework counter plus one) and a system message on the thread names the stage and the conflicted files.
3. A builder claims the card, brings the branch up to date and resolves the conflicts, and hands off again.
4. The card goes through Review and Test again like any reworked card, and the rebase is tried again before the test and before the completion action.

5.5 Finalize and completion actions

Finalize is claimed by the BA (always a senior agent). If the card came from a ticket, the BA first drafts the update for the ticket (see 4.3). It then checks that the completion action still fits the card (it may change it and says why in the note) and reports `aom finalize --action ...`. The action itself is not done by an agent: the server sends it to the card's workstation and the runner executes it as fixed code. The runner first rebases the branch onto the working branch (5.4), then:

- **pr (Open a pull request):** pushes the card branch to `origin` and opens the pull request with `gh pr create`, base = the working branch. Without `gh` the branch is still pushed, the card finalizes, and a desk notice says to open the pull request by hand.
- **push_branch (Push the branch):** pushes the card branch to `origin` with `git push -u`. Nothing else happens; someone else integrates it.
- **merge (Merge into the base branch):** in a throwaway worktree on the working branch the runner merges the card branch with `git merge --no-ff`, pushes the working branch to `origin` (when the repository has a remote) and moves the local working branch forward when that is safe. A merge conflict is reported like a rebase conflict (5.4). For a large card the manager must approve first (5.6).

The pr and push_branch actions need an `origin` remote; without one the action fails. As soon as the action succeeds the card moves to **Done**, its worktree is removed (the branch is deleted only after a merge) and a separate full-suite card is queued for a tester (see 5.7). The full suite does not hold the card back.

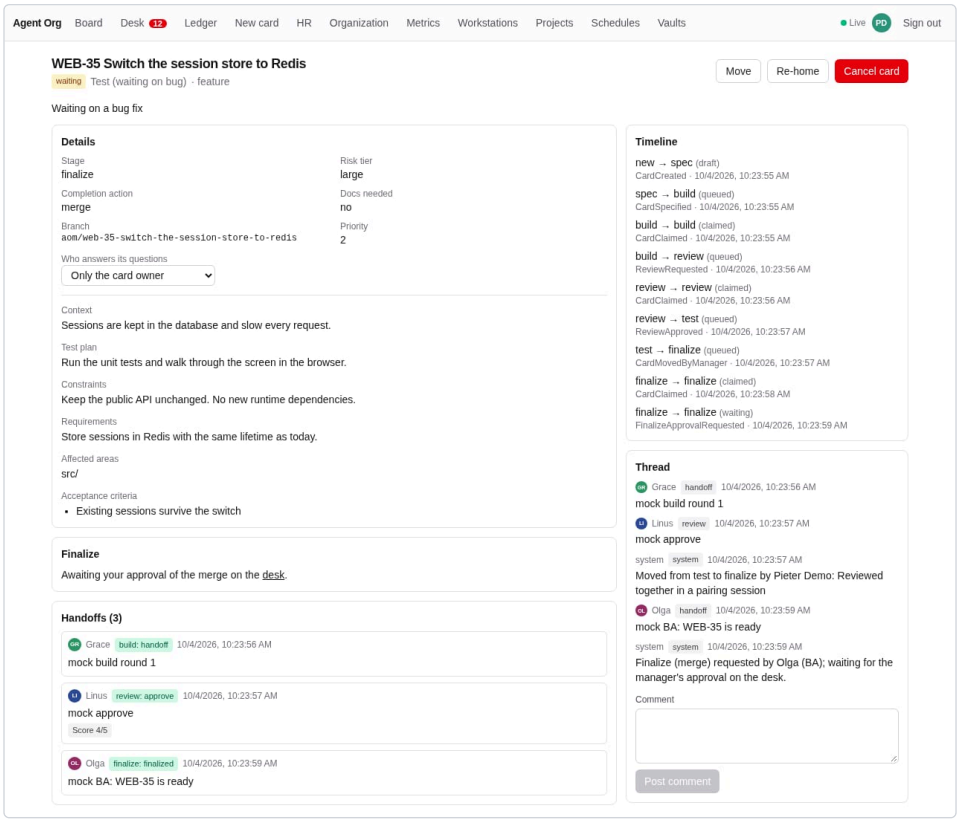


Figure 80 — Card WEB-35, risk tier large, completion action merge, waiting at Finalize: the Finalize panel says "Awaiting your approval of the merge on the desk" (Pieter Demo)



Figure 81 — The Finalize panel of WEB-35 while the merge awaits approval (Pieter Demo)

The Finalize panel has two states. While a card waits unclaimed at Finalize, the panel shows a selector **Completion action** (Open a pull request, Push the branch, Merge into the base branch) and a **Finalize** button with the note "A BA, if you have one, finalizes cards waiting here by itself; this is your override." Once a merge awaits approval (as in the figure), the panel only shows the awaiting-approval line with a link to the desk. While the runner works it says "Finalizing: the … is running."

> **Finalize and completion action, in order**
>
> 1. The card reaches Finalize and the BA claims it.
> 2. The BA drafts the ticket update if the card came from a ticket, and picks or confirms the completion action (pr, push_branch or merge).
> 3. If the action is merge and the card's risk tier is large: a desk item "Approve merge of …" is created and the card waits. Nothing is merged until you approve. For every other combination the action is sent to the workstation at once.
> 4. The runner rebases, then pushes, opens the pull request or merges, as described above.
> 5. Success: the card moves to Done and a full-suite card is queued. A rebase or merge conflict sends the card back to Build (5.4).
> 6. Any other failure (branch not found, no remote, push refused, uncommitted changes in the worktree): see below.

Finalize failed: the card returns to the Finalize queue, the Finalize panel shows "Finalize failed: <error>. Fix the cause and press Finalize again.", and a pinned, non-blocking desk item "Finalize of WEB-xx failed (action)" carries the error. After fixing the cause, press **Finalize** on the card or let the BA try again.

Docs-needed cards: if the card has **Docs needed** set, it goes to the Docs stage after testing instead of straight to Finalize. A technical writer claims it, updates the documentation and hands it on to Finalize, where the BA finalizes it with the rules above. A writer may only change `docs/**` and Markdown files; the runner's guard rejects anything else.

5.6 Large merges need your approval

When a card with risk tier large reaches Finalize with the completion action **merge**, the merge is not executed until you approve it on the desk. The same holds when you press Finalize yourself. The completion actions pr and push_branch never need an approval, whatever the tier.

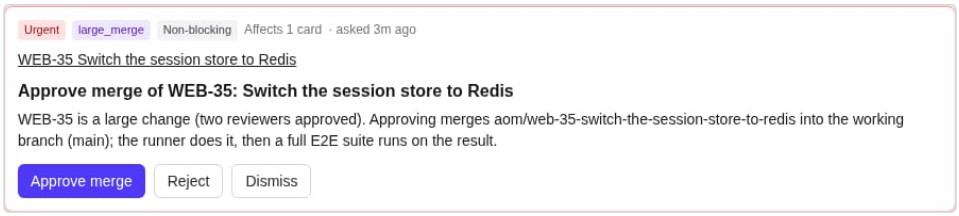


Figure 82 — Large merge approval for WEB-35: badges Urgent, large_merge and Non-blocking, and the buttons Approve merge, Reject and Dismiss (Pieter Demo)

The item is pinned (shown as Urgent) and non-blocking. Its title is "Approve merge of WEB-35: <title>" and its text says that the card is a large change, which branch will be merged into which working branch, that the runner does the merge and that a full E2E suite runs on the result. **Approve merge** sends the merge to the workstation. **Reject** ends the attempt: the card is marked "Finalize failed: Merge not approved" and returns to the Finalize queue.

> **Good to know**
> While the merge awaits approval the card has the status **waiting**, so the board shows it in the column "Test (waiting on bug)" and the card page says "Waiting on a bug fix", although no bug is involved: it is waiting for you. The Details panel shows the stage Finalize. In the demo, WEB-35 was moved to Finalize by hand ("Reviewed together in a pairing session"), so its thread shows one review although the desk text says two reviewers approved.

> **Why large merges need approval**
> A large card is one with a big diff, a refactor or a change in an architectural area. Approving the merge is your last chance to stop it before it reaches the working branch.

5.7 Full suite and regression runs

Full suite: every time a card is finalized (pr, push_branch or merge), the server queues a card "Full E2E suite after WEB-xx: ..." (type regression, platform web) in the Test stage. It is a scheduled-style card, so it jumps the queue. A tester runs the whole Playwright E2E suite on the branch the action produced: the working branch after a merge, the pushed card branch after pr or push_branch. This never blocks anything.

Regression runs: you start one with the **Run regression** button in the header of the board. The dialog "Run regression" has a Platform select (Web, Mobile or Both) and the buttons Close and **Run**. It queues one regression card per platform in the Test stage ("Both" makes two cards) and lists the cards it created. A tester runs the full E2E suite for that platform on the working branch, not on a feature branch. Regression runs can also be scheduled (a schedule of kind regression emits the cards), but the Schedules page in the web UI only creates feed schedules; a regression schedule has to be created through the API.



Figure 83 — The Run regression dialog: the Platform select (here Web) and the buttons Close and Run (Pieter Demo)

If a suite fails, the tester that runs it decides what the cause is. A test-side cause (a flaky test, test data, the environment) it fixes itself in the E2E tests and commits. A genuine product bug it reports with `aom test fail`: the server creates a bug card as a draft in the Spec stage, for the BA, and a non-blocking desk notice "Full-suite failure: ..." (or "Regression failure: ...") that names the bug card. Nothing is blocked; you decide the priority of the fix. A passing suite ends its card.

5.8 The tester bottleneck alert and environment check

Tester bottleneck alert: when a card has been waiting in a tester queue for more than 5 minutes and every tester of the project is busy or offline, you get one alert on the desk per project. It is pinned (shown as Urgent) and non-blocking. The alert is only raised for projects that have at least one tester; a project without testers never gets it. It closes itself when no card waits for a tester any more. Its buttons are **Hire tester**, **Acknowledge** and **Dismiss**; Hire tester opens the Hire form with the Tester role already chosen.

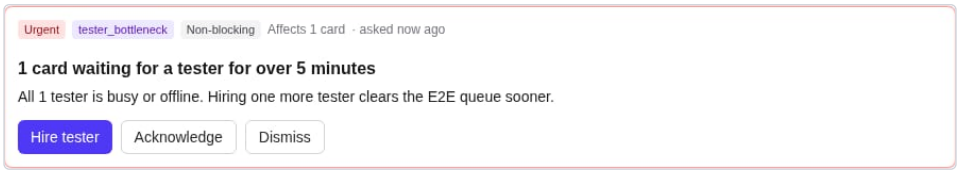


Figure 84 — Tester bottleneck alert: "1 card waiting for a tester for over 5 minutes", with Hire tester, Acknowledge and Dismiss (Pieter Demo)

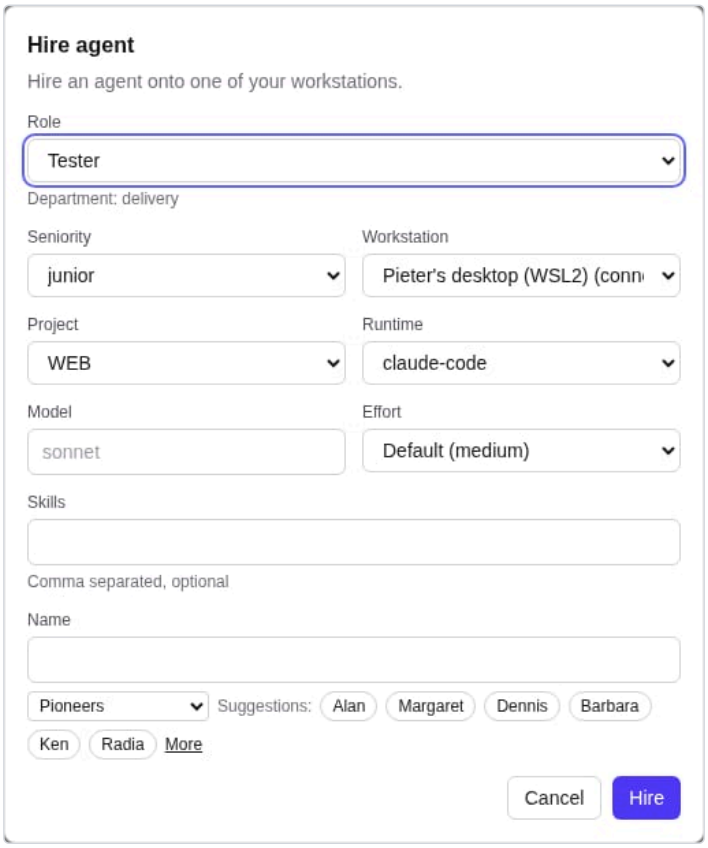


Figure 85 — The Hire agent form opened from the alert: Role is Tester, Project is WEB, Seniority junior (Pieter Demo)

Test environment check: a tester shift on a card is only started when the runner has checked the environment on that workstation. The check has two parts:

- Docker answers (`docker info`).
- If the repository has a file `.aom/test-env.json` with a `healthUrl`, that URL answers with a success status within 5 seconds. Without that file the application is not checked.

If the check fails, the shift is not started. The server asks the DevOps engineer of that workstation (a DevOps request: install Docker, or start the application) and parks the card: it gets the status waiting, so it also shows in the board column "Test (waiting on bug)", with a notice that it is parked on a DevOps request. When the request is done or has failed, the card returns to the Test queue and the check runs again.

> **Hard stop**
> The card is not parked but stopped with a blocking desk item "WEB-xx cannot be tested: ..." when Docker is missing and the DevOps engineer is not allowed to install Docker on that workstation (it needs both the install and the Docker access), when the DevOps engineer already tried twice, or when the workstation has no DevOps engineer. Fix the environment or grant the access, then resolve the item and the card returns to the Test queue. Separately, the DevOps engineer itself asks for your approval (a "devops" item on the desk) before it does anything destructive or outside its granted access.

6 Research, feeds and schedules

Research and feed projects run facts and news on the same kind of board as code. A researcher writes and a checker verifies and publishes; scouts find candidates and curators filter them. You rate the digest items to teach the system what matters. The demo runs on the MOCK runtime: the notes, sources and items in the figures are scripted ("Source 1", "mock scout found 3 candidate(s)"); real agents open real pages.

6.1 Research projects: question to published note

Research projects use the built-in Research workflow to answer questions and publish fact-checked notes into the project's vault. A project has one board with a card per question.

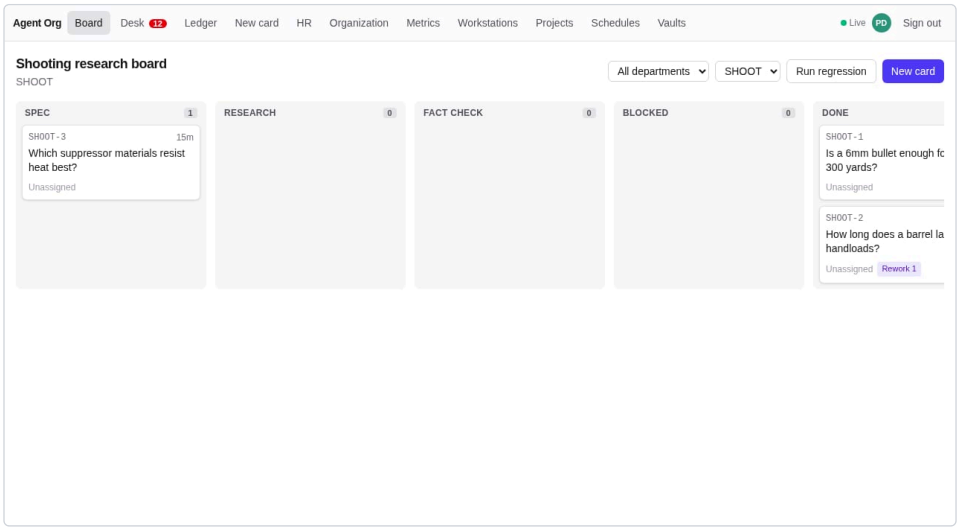


Figure 86 — Board of the research project SHOOT: the columns Spec, Research, Fact check, Blocked and Done. SHOOT-3 waits in Spec; SHOOT-1 and SHOOT-2 (Rework 1) are Done (Pieter Demo)

Research cards carry these fields: **Question**, **Scope** (free text), **Depth** (quick, standard or deep), **Sub-category** (the folder inside the project's vault), and **Preferred sites**. A **researcher** agent finds and writes; a **checker** agent verifies the claims and publishes. Nobody clicks a "publish" button: both agents report with their own commands.

Research workflow:

> **Research: Question, Research, Fact check, Done**
>
> 1. **Question** (the stage shows in the board column "Spec"): you create and specify the card with its question and depth. A research card needs Question and Depth before a researcher can take it.
> 2. **Research:** the researcher claims the card. It searches generally, searches the preferred sites first, and matches its effort to the depth (quick is a few good sources, deep is broad and cross-checked). It opens every source it relies on, writes down each claim with the page that supports it, and hands over a note whose claims carry numbered references [n] and a list of references. It says plainly what it could not establish. If it needs an answer from you, it asks a question on the desk like any other agent.
> 3. **Fact check:** the checker claims the card and opens every cited page to check that it really says what the note says. If any claim is unsupported it sends the card back to Research (outcome "Unsupported claims", Rework counter plus one). If all hold, it publishes.
> 4. **Done:** publishing is the end of the Fact check stage, there is no separate Publish stage. The runner writes the note into the project's vault at `projects/<project folder>/<sub-category>/<title>.md` with front matter (title, card ref, sub-category, publication date and the citations with url, title and access time), and the card moves to Done.

6.2 Citations and the fact check

No citations, no publication: the publish step itself refuses a note that has no citations, and it refuses a source the checker did not open in that shift. The checker has to fix what is reported and publish again. A claim that the cited page does not support never gets that far: the checker sends the card back to Research with the outcome "Unsupported claims" and a note that names each claim and why. The researcher then supports the claim with a better source or removes it.

Fact check outcome: the checker reports either "published" or "unsupported". These are agent commands, there are no Fact check passed or Request rework buttons in the web UI. The note of the checker explains each unsupported claim and shows in the card's thread and handoffs.

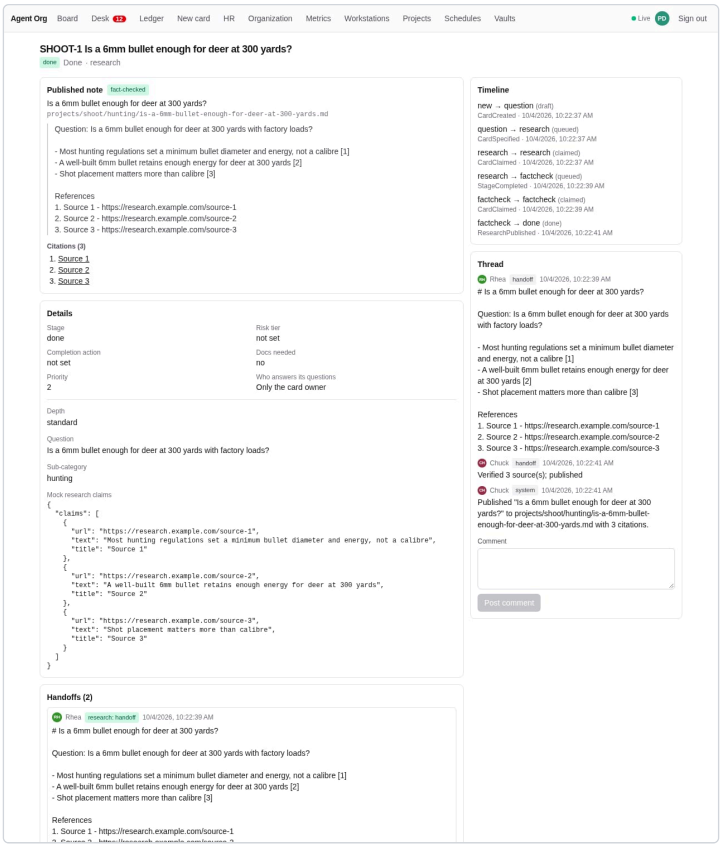


Figure 87 — SHOOT-1 "Is a 6mm bullet enough for deer at 300 yards?": the Published note panel with the vault path, the preview and three citations; Rhea's and Chuck's handoffs in the thread (Pieter Demo)

Published note panel: the heading "Published note" with a green "fact-checked" badge, the title, the vault path in monospace (`projects/shoot/hunting/is-a-6mm-bullet-enough-for-deer-at-300-yards.md`), a quoted preview of the start of the note, and "Citations (n)" as a numbered list of links.

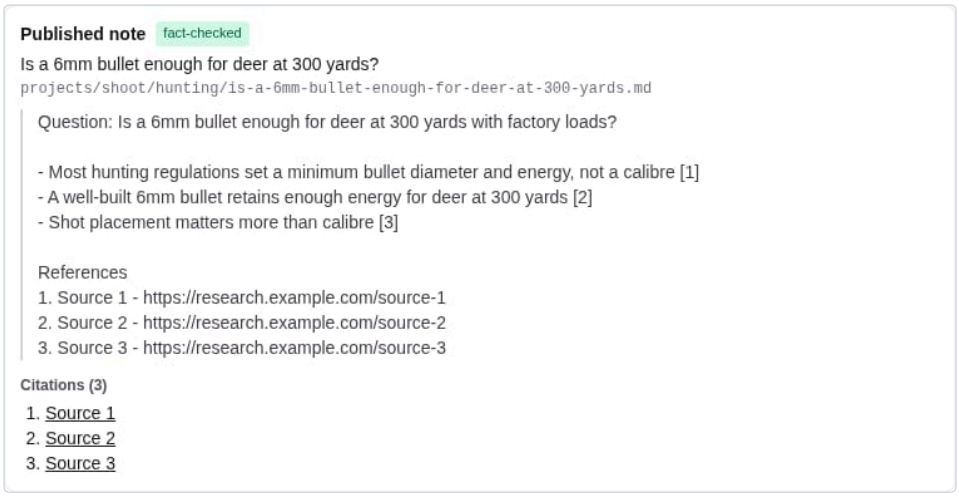


Figure 88 — The Published note panel of SHOOT-1: title, vault path, preview with its references, and Citations (3) Source 1 to Source 3 (Pieter Demo)

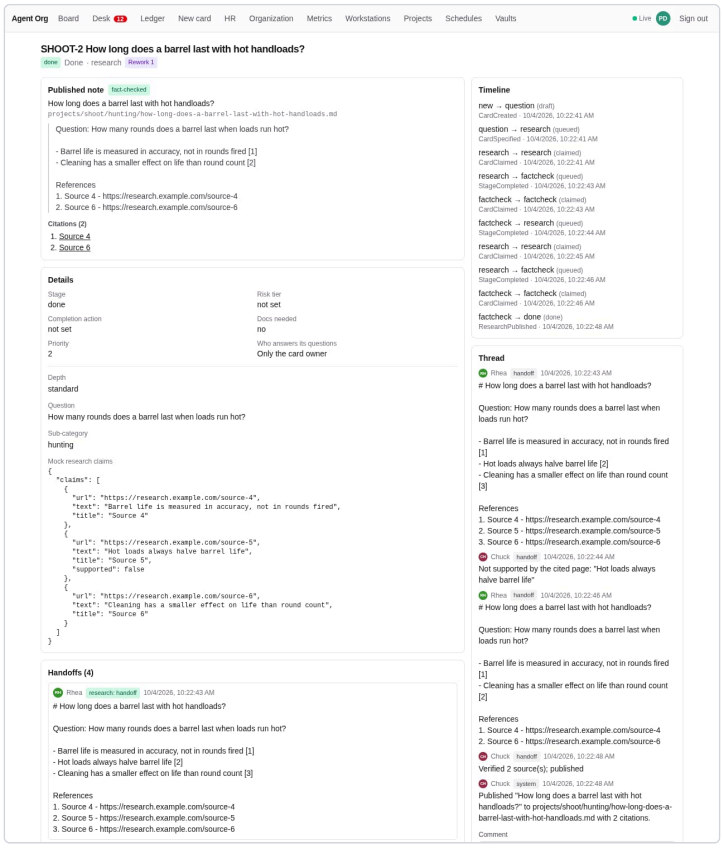


Figure 89 — SHOOT-2 "How long does a barrel last with hot handloads?": Rework 1. The checker (Chuck) rejected "Hot loads always halve barrel life" as not supported by the cited page, Rhea dropped it, and the note was published with 2 citations (Pieter Demo)

Fact check rules

- The checker opens every cited page and checks numbers, dates, names and quotations against it.
- Any unsupported claim sends the whole card back to Research as Rework; the checker does not publish a partial note.
- The checker does not rewrite claims; the researcher fixes the draft. The published text is the researcher's draft, corrected only for wording.
- A note must carry at least one citation, and every citation must be a page opened in that shift.
- Open questions that agents cannot answer go to the desk like any other card question.

Good to know

The card form and the Details panel show a field "Mock research claims" (JSON). It is a test aid of the mock runtime: it scripts the claims the pretend researcher "finds" and whether the pretend checker can verify them. Real users leave it empty. In SHOOT-2 it is the claim with `"supported": false` that caused the rework round.

6.3 Feed projects and digests

Feed projects use the built-in Feed workflow to collect the latest items on a topic. Every run is a card that goes through the stages Scan and Curate to Done. The board header of a feed project has two extra buttons: **Feed** (opens the feed project page) and **Run now** (starts a run immediately).

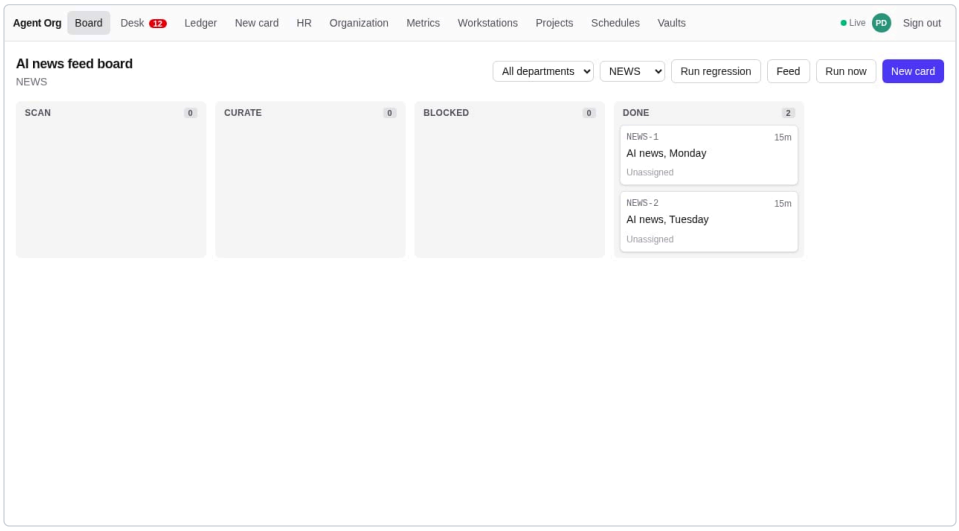


Figure 90 — Board of the feed project NEWS: columns Scan, Curate, Blocked and Done with the finished runs NEWS-1 and NEWS-2, and the header buttons Feed and Run now (Pieter Demo)

Feed cards are created by a schedule or by Run now. Each carries a **Topic**, a **Window (hours)** (default 24: the scout looks for items from the last 24 hours), **Preferred sites** and **Max per kind** (default 5 articles and 3 videos). A **scout** collects candidate pages and a **curator** picks the best of them. Run now uses the template of the project's schedule; a feed project without a schedule gets a default card whose topic is the project name.

Feed workflow:

> **Feed: Scan, Curate, Done**
>
> 1. **Scan:** the scout searches the web for the topic, the preferred sites first. Every candidate must be a real page the scout opened in that shift, with kind, URL, title, source, publication date and a summary written from the page; the runner rejects the scout's report if a URL was not opened. Items already on the project's seen-list are skipped, and a story the scout already has from another source too. It may hand over a few more candidates than the maximum so the curator can choose.
> 2. **Curate:** the curator reads the project's `taste.md` and picks the best of the scout's candidates, at most the maximum per kind (it may pick fewer, or none). It cannot add an item that was not a candidate. The server drops duplicates (same URL or same story as an item already listed) and anything over the maximum per kind.
> 3. **Digest posted, card Done:** posting the digest ends the Curate stage and the card moves to Done at once. The new items are appended to the running list `feed.md` in the project's vault, and a "digest" item for you to rate appears on the desk (see 6.4). If nothing qualifies, nothing is logged and no desk item is created.

The running list is the file `projects/<project folder>/feed.md`: oldest first, entries only ever appended, one line per item (date logged, linked title, kind, source, publication date and summary). Earlier entries are never reordered or dropped. The list and `taste.md` are only written when the project has a vault; without one the ratings and the list still exist on the site.

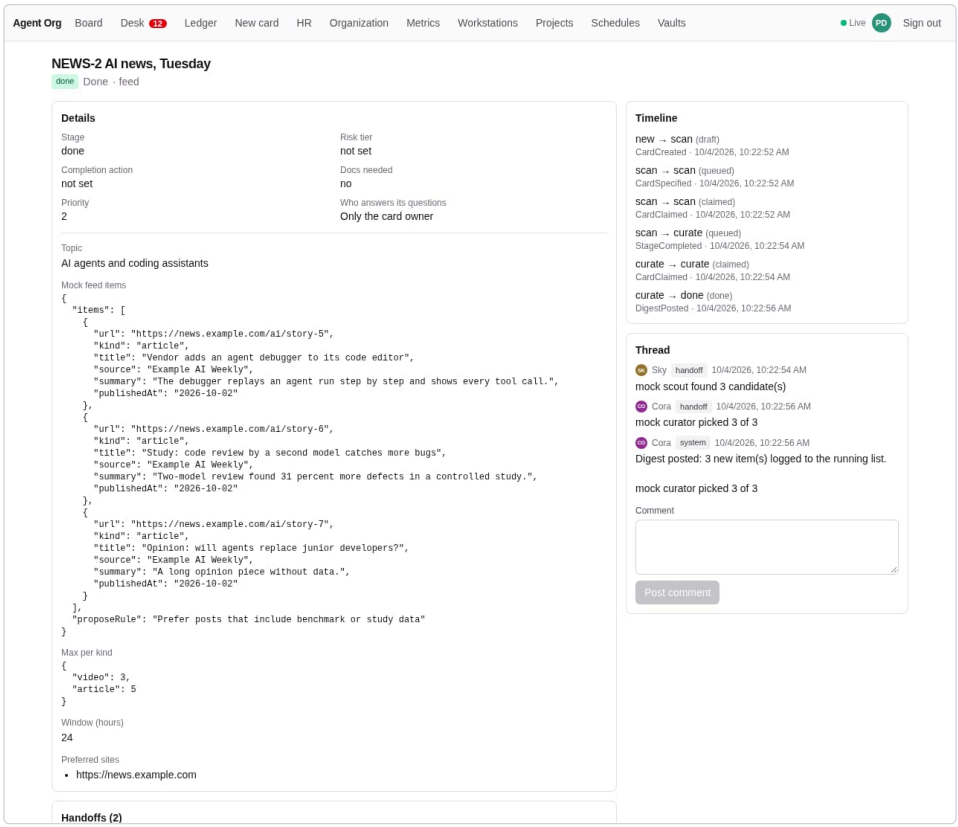


Figure 91 — NEWS-2 "AI news, Tuesday": the feed fields (Topic, Max per kind 5 articles and 3 videos, Window 24 hours, Preferred sites), Sky's scout handoff and Cora's curator handoff "Digest posted: 3 new item(s) logged to the running list" (Pieter Demo)

The feed project page (**Feed** button on the board) has the buttons **Board** and **Run now** and three panels: **Schedules** (name, recurrence, time zone and next run of each), **Running list** (the entries with their rating, and rating buttons for unrated ones) and **taste.md** (a preview of the file, or "No ratings yet.").

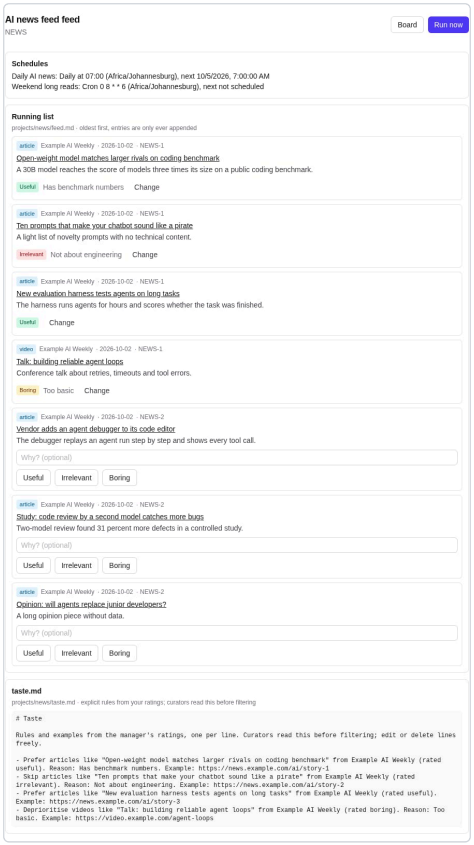


Figure 92 — Feed page of NEWS: two schedules (Daily at 07:00, and Weekend long reads whose next run is "not scheduled"), the running list with four rated entries from NEWS-1 and three unrated from NEWS-2, and the taste.md preview (Pieter Demo)

6.4 Rating digests and taste.md

When a digest is posted, one desk item "Rate the AI news feed digest (3 items)" appears. It has the violet badge "digest", is non-blocking, and names the curator and the feed card. Each item shows its kind, source, publication date, card ref, a linked title and the summary, with an optional field "Why? (optional)" and three buttons: **Useful**, **Irrelevant** and **Boring**. You can also rate from the running list on the feed page, where a rating can be changed with **Change**. When the last item of a digest is rated, the desk item closes by itself; **Acknowledge** or **Dismiss** closes it without rating the rest.

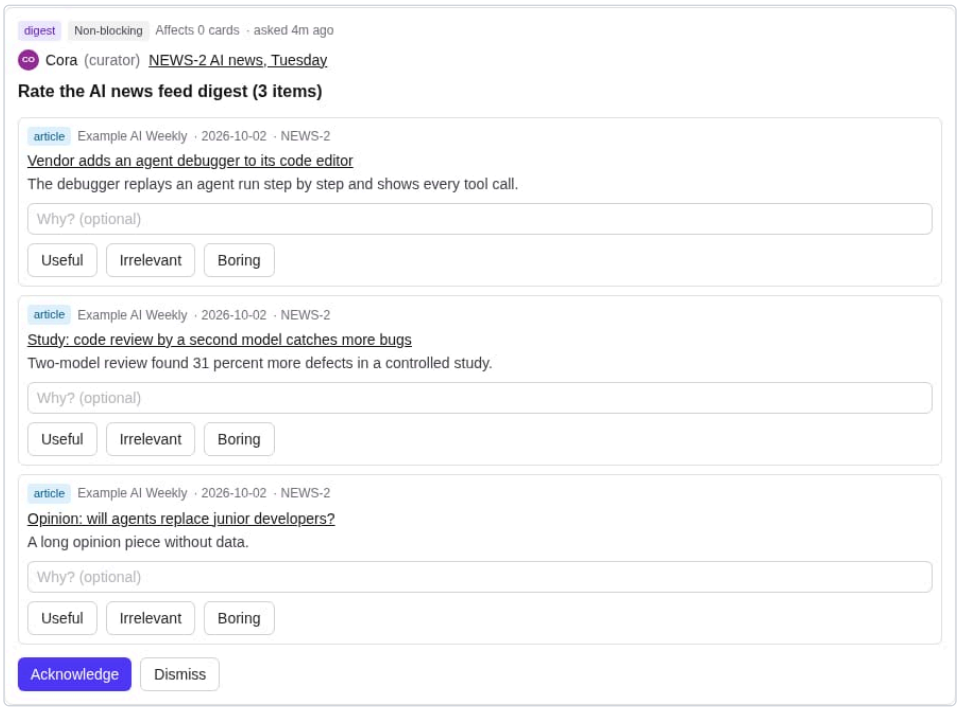


Figure 93 — Digest item on the desk from Cora (curator) for NEWS-2: three articles, none rated yet, each with Useful, Irrelevant and Boring, and the buttons Acknowledge and Dismiss (Pieter Demo)

Every verdict becomes a line in `taste.md`: as soon as you rate an item, a line is appended to `projects/<project folder>/taste.md` in the vault. The line is a readable rule plus the example it came from, never a score. The wording depends on the verdict:

- Useful: `- Prefer articles like "<title>" from <source> (rated useful). Reason: <your reason>. Example: <url>`
- Irrelevant: `- Skip articles like "<title>" from <source> (rated irrelevant). ...`
- Boring: `- Deprioritise videos like "<title>" from <source> (rated boring). ...`

The word before "like" is the item's kind (articles, videos). The Reason part only appears when you typed a reason. If you change a rating, the file keeps both lines. The file is plain text: you can edit or delete lines in the vault freely.

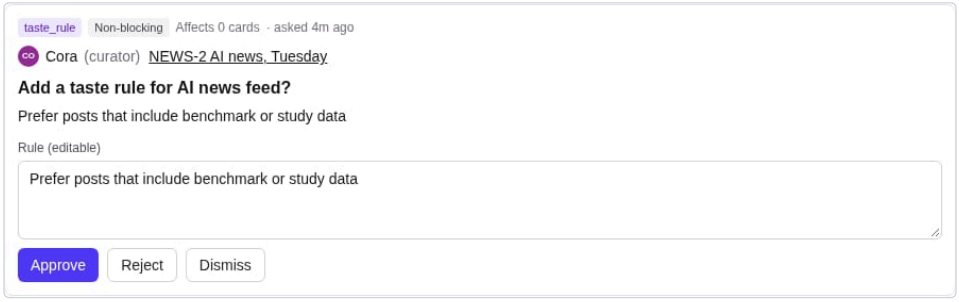


Figure 94 — Taste-rule proposal from Cora (curator) for NEWS-2: "Add a taste rule for AI news feed?" with the editable rule "Prefer posts that include benchmark or study data" and the buttons Approve, Reject and Dismiss (Pieter Demo)

Curator taste-rule proposal: when the verdicts show a pattern that no rule covers yet, the curator may attach a proposed rule (one sentence, with the URLs that are its evidence) to its digest. It appears on the desk as an approval item (badge "taste_rule") with the text editable. **Approve** appends the rule, in your edited wording if you changed it, to `taste.md` as `- Rule: <text>`. **Reject** and **Dismiss** add nothing. The curator never edits `taste.md` itself.

Curators read taste.md before filtering: the next feed run shows the curator the taste rules. It prefers what you rated useful, skips what you rated irrelevant, deprioritises what you rated boring, and follows explicit rules exactly. The rules are plain lines you can read and edit, not a hidden model.

Feedback loop: rating, taste.md, curation

1. You rate digest items: Useful, Irrelevant or Boring, with an optional reason.
2. Each rating is appended to `taste.md` as a Prefer, Skip or Deprioritise line.
3. The curator of the next run reads `taste.md` before picking items.
4. If the curator sees a pattern, it proposes a rule on your desk.
5. You approve (optionally edit) or reject the rule; an approved rule is appended as a "Rule:" line.

6.5 Schedules and Run now

The **Schedules** page (`/schedules`) lists the schedules of the projects you are a member of. Each schedule is a panel with:

- The name, a badge for its kind (feed), a link to the project and the badge Enabled or Paused.
- **When**: a preset name such as "Daily at 07:00" or, for any other expression, "Cron 0 8 * * 6", followed by the time zone.
- **Next run**: the next fire time, or "not scheduled" for a paused schedule. **Last run**: the time, or "never". **Topic** of the feed.
- When a run could not be made: a red badge "Cannot run (n failed tries)" and the error with "Retrying with growing pauses."

Buttons: **Run now** emits a card immediately (a message "Emitted NEWS-3." follows) without changing the schedule's own timing. **Pause** or **Resume** switches it off or on. **Delete** removes it after a confirmation. Only the project owner can create or manage schedules.

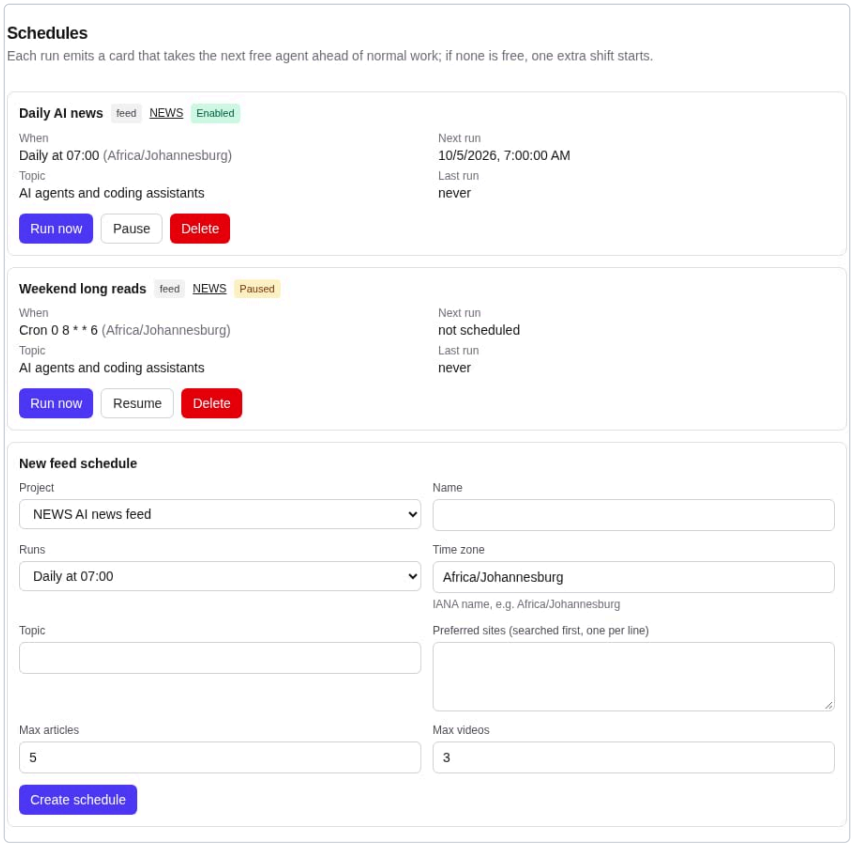


Figure 95 — The Schedules page: "Daily AI news" (Daily at 07:00, Africa/Johannesburg, Enabled, next run 10/5/2026 7:00 AM), "Weekend long reads" (custom cron 0 8 * * 6, Paused, next run not scheduled) and the New feed schedule form (Pieter Demo)

New feed schedule: the form lists the feed projects you own. Fields: **Project**, **Name**, **Runs**, **Time zone**, **Topic**, **Preferred sites** (one per line, or separated by commas) and **Max articles** and **Max videos** (default 5 and 3). Name and Topic are required. Press **Create schedule**.

The presets in **Runs** are: Daily at 07:00, Daily at 18:00, Weekdays at 07:00, Mondays at 07:00, Every hour, and **Custom cron expression**, which shows a field for a 5-field expression (minute hour day month weekday). The **Time zone** is an IANA name such as `Africa/Johannesburg`; it starts as the time zone of your browser, and the cron expression is evaluated in that zone. An invalid expression or time zone is refused. The form only creates feed schedules; the card's Window (hours) is not set here and stays at 24.

Failed schedules with retries: if the server cannot emit the card of a schedule (for example a regression schedule on a project without a repository), the schedule does not retry on every tick. It backs off: the first retry after 1 minute, then the pause doubles each time, up to 6 hours. It shows "Cannot run (n failed tries)" with the error, and one non-blocking desk notice "Schedule ... cannot run" is raised for the whole streak. The notice closes by itself when the schedule fires again; a successful fire, or an edit of the schedule, resets the count. Run now is available at any time.

Schedule firing and queue priority

1. The server checks for due schedules every 15 seconds. A schedule that was due while the server was down fires once, not once per missed time.
2. Each fire creates a scheduled card. Scheduled cards are offered to agents ahead of normal work.
3. If a scheduled card has waited more than 30 seconds in the queue with no free agent of the right role, the dispatcher gives it one extra shift on an agent that is already at capacity (the overflow rule, see 6.6).
4. Only one such extra shift runs at a time per organization; a second late scheduled card waits for the next free agent.

6.6 The overflow rule

Scheduled work (feed runs and regression runs) must not slip just because the agents are busy. The rule, as the dispatcher applies it:

- A scheduled card is offered to agents ahead of normal work. If a free agent of the right role exists, it takes the card in the normal way.
- If the card has been queued for more than 30 seconds and no agent is free, the dispatcher picks an agent of the right role that is at its capacity and starts one **extra** shift for it. The extra shift does not end the shift the agent is already running.
- Only one extra shift may run at a time in the whole organization. While it runs, a second late scheduled card waits in the queue until an agent becomes free; it does not start another extra shift.

This is a limit per organization, not per agent. The runner of a workstation also limits how many shifts run at once (4 by default, set in its config) and refuses more ("Shift rejected by the runner (busy: 4 shifts already running)"); the card then goes back to the queue.

Why the overflow rule exists

A feed that fires late or not at all would break the running list and the digest cycle. One extra shift is an acceptable cost; one extra shift per late schedule would overload the workstations. So the first late scheduled card jumps ahead and gets an extra shift, and later ones wait their turn.

7 Projects, vaults, DevOps and sharing

7.1 Projects

Projects → New project opens a dialog to create a new project. Every project owns a board, repositories and agents. Once created, the project appears on the Projects list with its key, its name and four badges: the workflow key, the department of that workflow (if it has one), the domain (`work` or `personal`) and your role (`owner` or `manager`). When any project has a department, a **Department** filter appears above the list. Click **Board** to see the cards, or **Settings** to configure it.

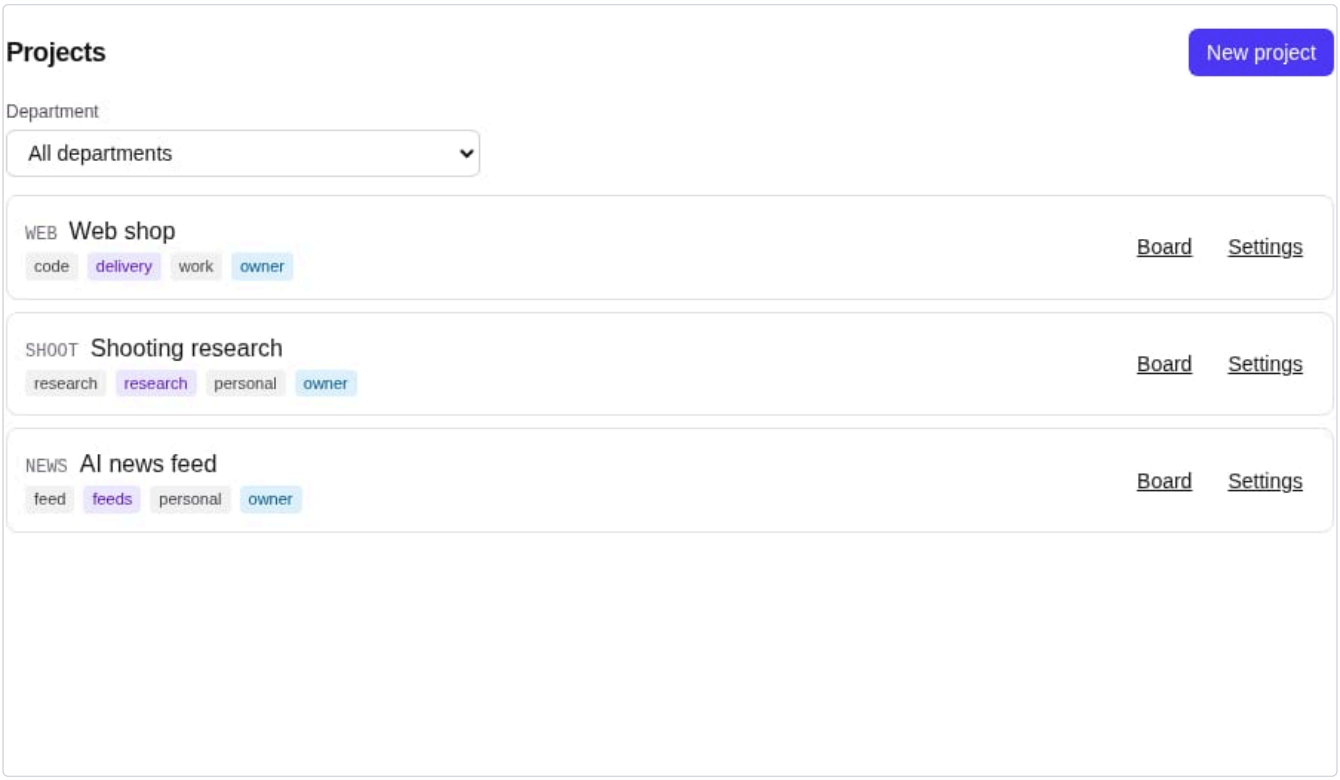


Figure 96 — Projects list: WEB, SHOOT and NEWS with workflow, department, domain and role badges (Pieter Demo)

Project key and name

The project key is a short identifier used in card references, for example `WEB-1` for a card in the WEB project. Keys must be **2 to 10 characters**, start with a letter (A-Z), and contain only uppercase letters and digits (0-9). The name is required (up to 120 characters) and is shown on the Projects list and in the board header. The dialog asks only for Key, Name and Workflow; the domain of a project created here is `work`.

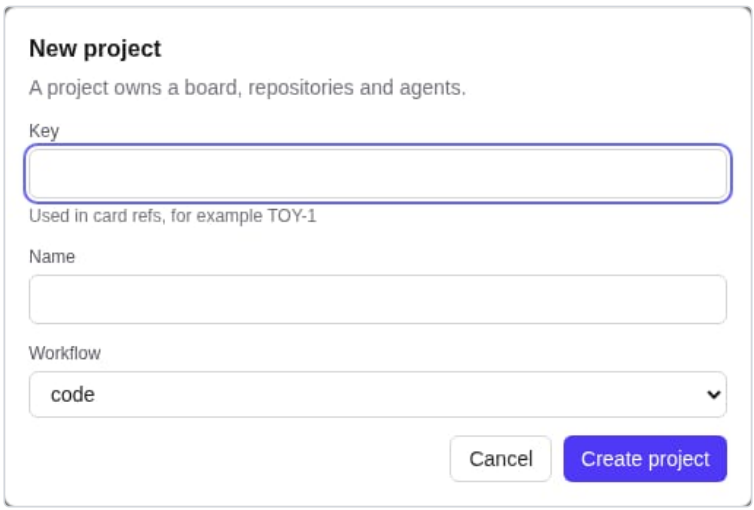


Figure 97 — New project dialog: Key, Name and Workflow (Pieter Demo)

Workflow choice: code, research, or feed

The project's workflow defines the stages cards move through. The Workflow list in the dialog shows every workflow of your organization. Four built-in workflows ship with the system; the table lists three of them. The fourth, `code-bugfix` (Build, Review,

Done), is for bug cards.

Workflow	Stages	Purpose
Code (default)	Spec, Build, Review, Test, Docs, Finalize, Done (the board adds columns such as To do, Rework, Blocked and Waiting)	Software delivery with code review and testing
Research	Question, Research, Fact check, Done	Research questions, fact checking and publication
Feed	Scan, Curate, Done	Automated feeds: a scout scans, a curator posts a digest

Custom workflows can be defined on the **Organization** page (sections 8.3 and 8.4).

7.2 Project settings and repositories

Projects → [Project] → Settings opens the project settings page. It has three panels one below the other: General, Repositories and Members. Only the project owner can change the settings; the server refuses changes from an additional manager.

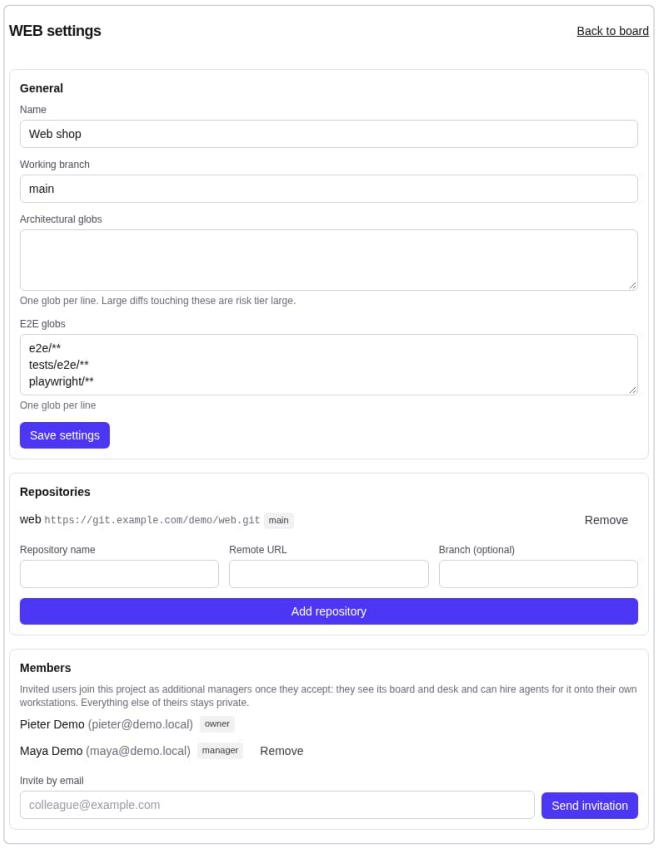


Figure 98 — Project settings of WEB: General (working branch main, E2E globs), Repositories (one repository, web) and Members (Pieter Demo owner, Maya Demo manager) (Pieter Demo)

General settings

The panel has the fields Name, Working branch (default `main`), Architectural globs and E2E globs, and a **Save settings** button. The two glob fields take one glob per line:

- **Architectural globs:** Large diffs touching these files get risk tier **large**. A large change needs the manager's approval before it is merged (see the Finalize panel on the card).
- **E2E globs:** Identify the end-to-end test files. Testers may only change files matching these globs, and builders may not change them.

Repositories

A project's repositories are the git sources the agents clone and work with. Each repository has:

- **Repository name:** Identifies the repository in the project.
- **Remote URL:** The git URL (https, ssh or `git@host:path`; a URL with a password or token is refused).
- **Branch (optional):** The working branch of this repository.

The Repositories panel lists each repository with its URL and branch, and **Add repository** adds a new one. The runner does not clone for you. At the start of a shift it looks for a clone with the same remote URL in the folders you gave it with `aom-runner repos`

`add` (section 10.2); if there is none, the shift is rejected. A workstation card lists the clones the runner found. Use **Remove** to remove a repository from the project (nothing on the workstation is deleted).

7.3 Vaults

The **Vaults** item in the top menu lists the knowledge vaults you can see and their sync status. A vault is a git repository of markdown that your agents read and that the DevOps engineer keeps in sync on every workstation. The site stores only the git URL, the sync interval and the sync status per workstation, never the vault content itself. Approved role and workflow definitions are written to your vault as YAML files (section 8.6).

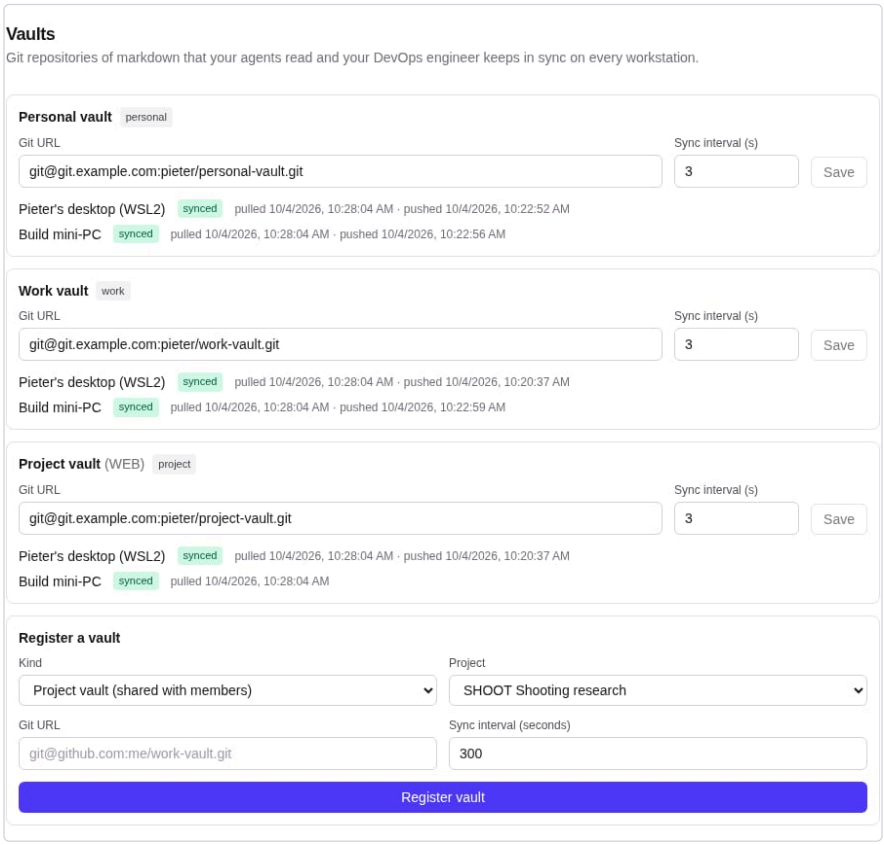


Figure 99 — Vaults page: personal, work and project (WEB) vaults, each synced on both workstations, and the Register a vault form (Pieter Demo)

Three vault kinds

Kind	Sharing	Content and access
Personal vault	Only you	Registered by you for yourself. One per user.
Work vault	Your organization	Not tied to a project. One per organization.
Project vault	Members of the project	Belongs to one project; one vault per project. Only the project owner registers it or edits its URL and interval. Additional managers see it with a shared with you badge and cannot edit it.

Git URL rules

The server checks the URL when you register or edit a vault. The runner uses its own git authentication, so a URL must not carry credentials. Rules:

- Accepted forms: `https://host/path`, `ssh://host/path` and `git@host:path`.
- No passwords or tokens in the URL: `https://token@github.com/me/vault.git` is refused.
- Local paths and `file://` URLs are refused (they exist for tests only).
- Register a vault with the **Register a vault** form: Kind (Work vault, Personal vault or Project vault), the Project for a project vault, the Git URL and the sync interval. You can register one work vault and one personal vault, and one vault per project you own.

Sync interval and status

Each vault has a sync interval in seconds (1 to 86400, default 300 in the register form). The DevOps engineer on each workstation pulls and pushes the vault at that interval. Below the URL, every one of your workstations shows its status:

- synced – pulled and pushed successfully on this workstation.
- waiting for first sync – not yet pulled since the vault was registered.
- error – last pull or push failed; the error details appear below.

- Sync paused: conflict – the sync found a conflict. The conflicting files and the clone path are listed; resolve them in the clone, then click **I resolved it, resume**.

Last pull and push times are shown for each workstation. The demo shows a sync interval of 3 seconds, which is a test setting.

Definition files and late registration

A role or workflow definition is written to your vault when you approve it, as `organization/roles/<key>/v<N>.yaml` or `organization/workflows/<key>/v<N>.yaml`, in your work vault (or another vault that is not tied to a project, if you have no work vault). If you have no such vault at that moment, nothing is written, and registering a vault later does not write the earlier approvals.

7.4 Sharing a project with another manager

In the **Members** panel of the project settings page, the owner types an email in **Invite by email** and presses **Send invitation**. The person must already have an active account; otherwise the server answers that there is no active account with that email. No email is sent. The invitation shows in the Members list with a pending badge and a **Cancel invitation** button. The invited person finds it in the **Project invitations** panel at the top of their **Projects** page and presses **Accept** or **Decline**. After accepting, they are an additional manager (`manager`) of that project only: they see its board and desk and can hire agents for it onto their own workstations.

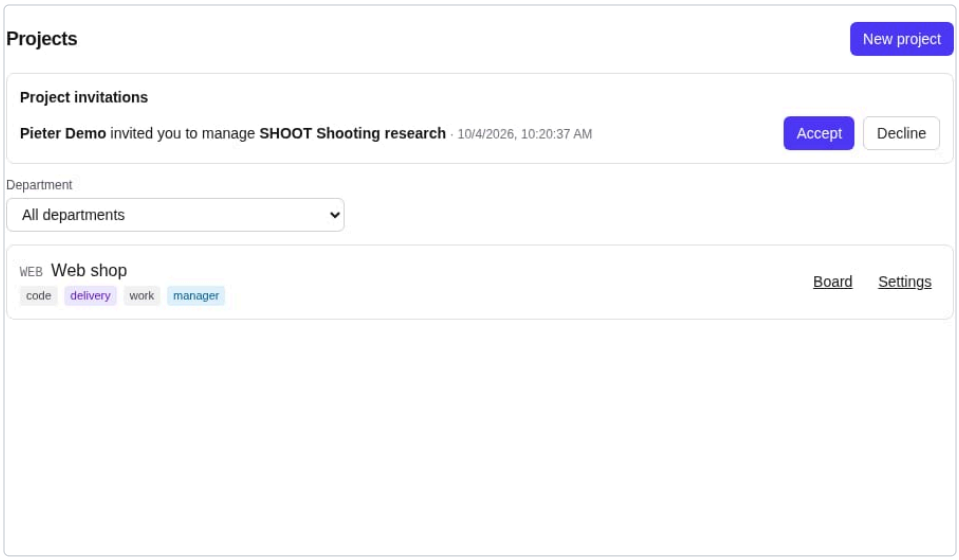


Figure 100 — Projects page as Maya: the Project invitations panel with Pieter Demo's invitation to SHOOT Shooting research, Accept and Decline, above Maya's one project WEB with the role badge manager (Maya Demo)

What an additional manager may and may not do

The project has two roles: `owner` and `manager` (an additional manager). An additional manager:

- **Cannot:** change the project settings, add or remove repositories, invite or remove members, register or edit the project vault, or change the owner's organization (roles, workflows, departments).
- **Can:** create cards and comment on cards; hire agents for the project onto their own workstations; manage (change model or role, re-home, fire) the agents they hired; manage the cards they own (edit, specify, set priority, completion action and decision policy, cancel, re-home, release).
- **Read-only:** desk items of cards they do not own, unless the card's decision policy is set to any manager (`Who answers its questions` on the card). On the HR board, the owner's agents are read-only for them.

The owner's catalogue (roles, workflows, departments) applies to the shared project's cards. Cards and agents are claimed and worked by agents, not by managers.

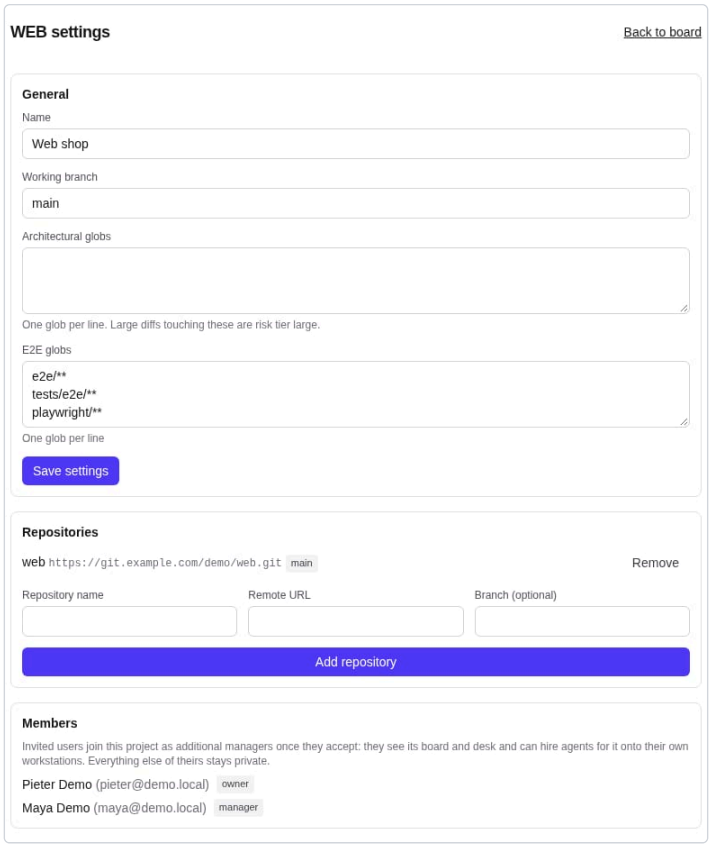


Figure 101 — Project settings of WEB as Maya (additional manager): the Members panel lists the two members and has no invite form and no Remove button (Maya Demo)

7.5 Removing a member

In the **Members** panel of the project settings page, only the project owner sees **Remove** next to an additional manager (next to a pending invitation the button reads **Cancel invitation**). A dialog titled **Remove [name]?** shows what will happen before you confirm. It has **Cancel** and **Confirm removal**.

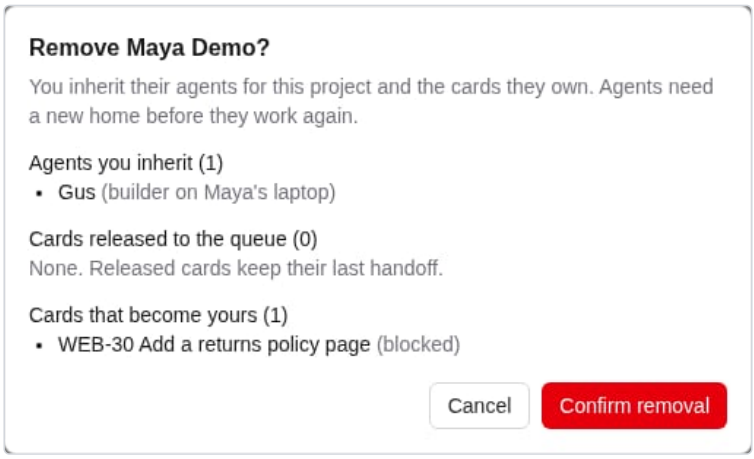


Figure 102 — Remove Maya Demo? dialog: Gus (builder on Maya's laptop) is inherited, 0 cards released, WEB-30 Add a returns policy page (blocked) becomes yours (Pieter Demo)

Removal dialog preview

The dialog lists:

- **Agents you inherit:** the agents they hired for this project (not the DevOps engineer), with role, workstation and the card an agent is holding, if any. They need a new home before they can work again.
- **Cards released to the queue:** the cards those agents are working on. The claim is released; the card keeps its last handoff and goes back to the queue.
- **Cards that become yours:** the cards the removed manager owns, with their status. They become yours.

What Confirm removal does, in this order

1. The claims held by their agents are released. Each card gets a system message in its thread and goes back to the queue with its last handoff attached.
2. Open cards of the project that were homed on the removed manager's workstations forget that home; their stage restarts from the base branch on any workstation.
3. The member is marked removed, their agents move to you with the status `needs_rehome` and no workstation, and the cards they own become yours.
4. A wipe command is queued for each inherited agent, so the old workstation deletes the agent's sandbox when it next connects.

7.6 The DevOps engineer

Every workstation has one DevOps engineer, an agent with the role **DevOps engineer** that is hired automatically when the workstation is paired (the demo's is named Atlas). It never claims cards. It works requests for setting up and maintaining the machine (installing tools, Docker, starting the app for a tester) and keeps the vaults in sync.

The DevOps engineer cannot be hired or fired by hand; it can only be reset, which wipes its sandbox. You, the workstation's owner, give it access and approve or reject what falls outside that access in the **DevOps engineer** panel of the workstation card.

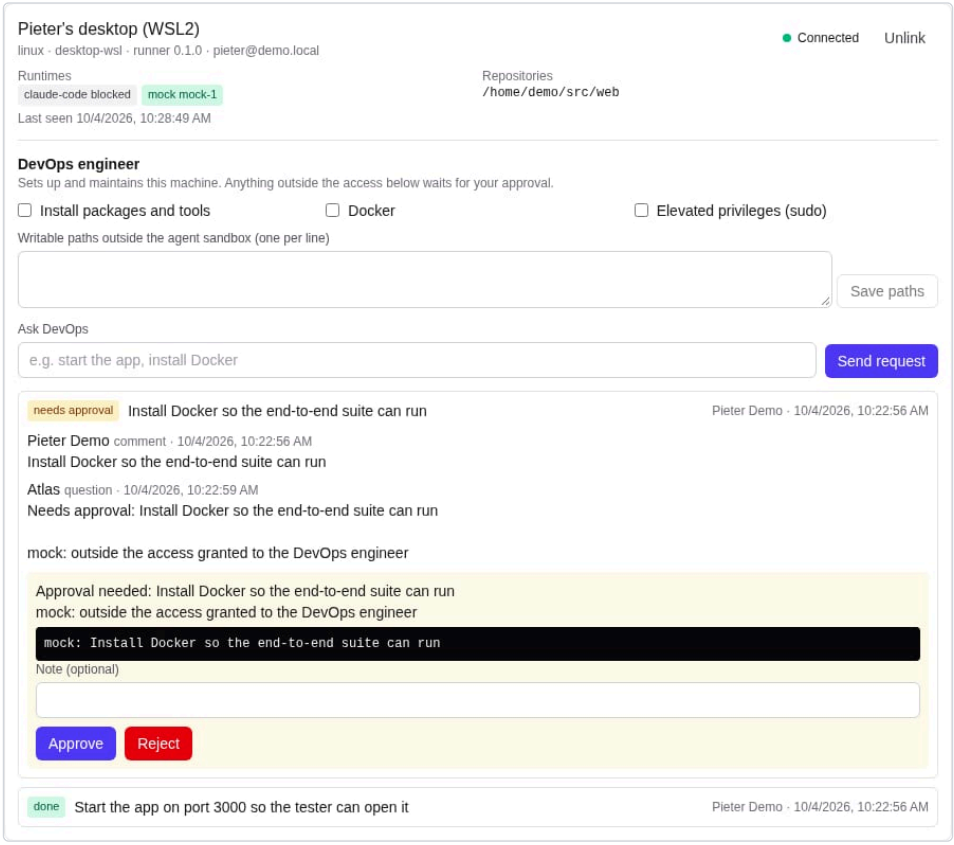


Figure 103 — DevOps engineer panel of Pieter's desktop (WSL2): three access checkboxes (all unchecked), writable paths, the Ask DevOps field, and two requests: Install Docker so the end-to-end suite can run (needs approval, thread by Atlas with Approve and Reject) and Start the app on port 3000 (done) (Pieter Demo)

Access checkboxes

Three checkboxes say what the DevOps engineer may do without asking. A checkbox saves as soon as you click it:

- **Install packages and tools:** it may install software.
- **Docker:** it may install, run and manage Docker containers.
- **Elevated privileges (sudo):** it may run commands with sudo. Use with caution.

All three start unchecked. The access is written into the DevOps engineer's instructions for each request, together with the rule that anything destructive (deleting data, wiping, formatting) or outside the granted access must wait for your approval.

Writable paths

The field **Writable paths outside the agent sandbox (one per line)**, with a **Save paths** button, lists paths the DevOps engineer may write to outside its sandbox, for example `/opt/myapp`. It is empty by default.

Ask DevOps requests

You ask the DevOps engineer something by typing in the **Ask DevOps** field (for example "start the app, install Docker") and pressing **Send request**. Agents ask it with `aom devops` (section 10.7). Every request is a thread, listed under the form. A row shows:

- A status badge: needs approval, queued, running, done or failed.
- The request text, who asked, and when. Click the row to open or close the thread; a request that needs approval is open already.
- In the thread: the messages of you and the DevOps engineer and, once done, the environment changes it made with the commands it ran.
- For a request that needs approval: the action and reason, the commands, an optional **Note**, and the buttons **Approve** and **Reject**.

The approval flow

1. You or an agent send a request. It is queued, then running while the DevOps engineer works on it.
2. If the work is destructive or outside the granted access, the DevOps engineer stops and asks for approval with an action, a reason and the commands (`aom devops-approval`).
3. The request shows needs approval and the same question appears on the decision desk as a pinned, blocking item "DevOps needs approval: ...". Answer it in either place.
4. You click **Approve** or **Reject**, with an optional note.
5. If approved: the request is queued again with your approval attached, runs, and ends done (or failed if it could not finish).
6. If rejected: the request ends failed with the result "Approval rejected" and your note. There is no separate "rejected" badge.

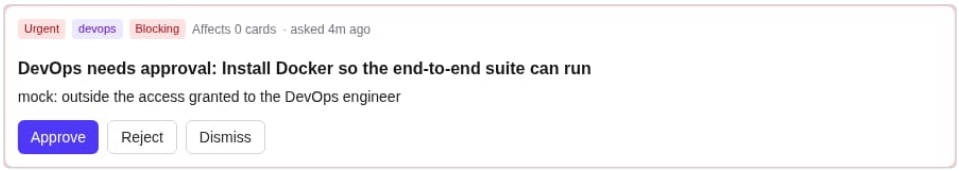


Figure 104 — The same approval on the decision desk: "DevOps needs approval: Install Docker so the end-to-end suite can run", badges Urgent, devops and Blocking, with Approve, Reject and Dismiss (Pieter Demo)

8 Organization designer

8.1 Roles

Organization → Roles tab shows your organization's role catalogue: custom roles you have designed, and built-in roles that ship with the system. Every agent holds a role; a role defines which workflow stages its agents may claim, their tools and their default runtime and model. The page header reads "Roles, workflows and departments. Definitions are versioned and approved on the desk before they are available."

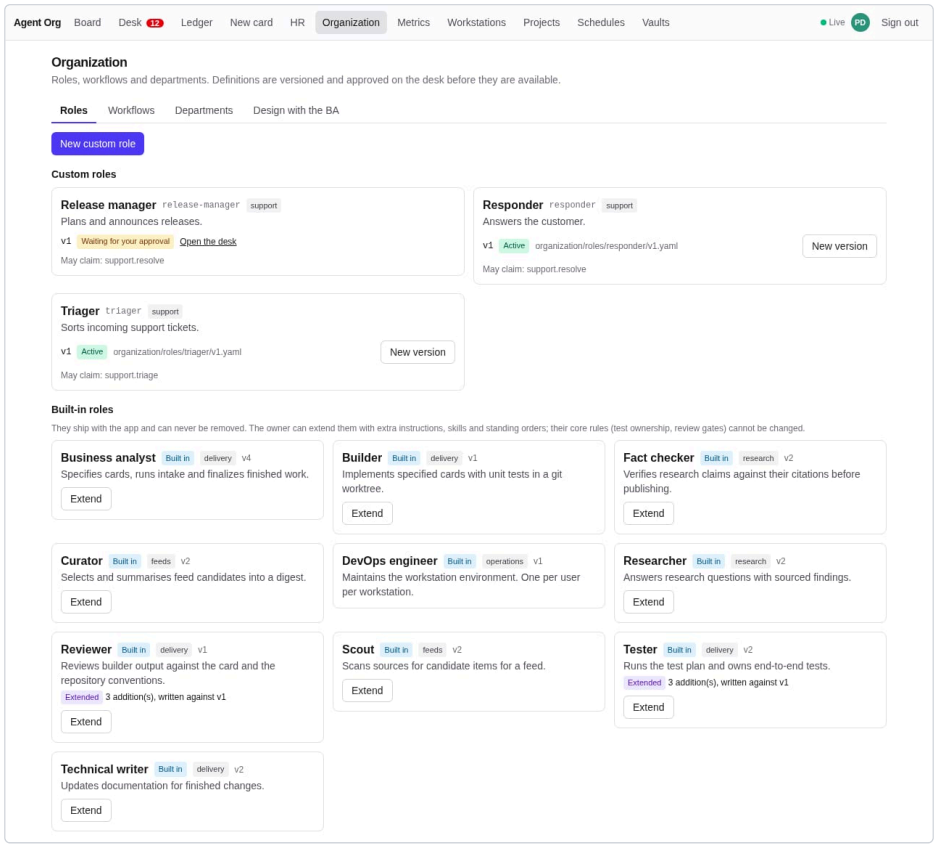


Figure 105 — Organization Roles tab: custom roles Release manager (waiting for approval), Responder and Triager (active), and the ten built-in roles with Extend buttons (none on DevOps engineer); Reviewer and Tester carry the Extended badge (Pieter Demo)

Custom roles

Press **New custom role** to create one. Each custom role is a card with its name, key, department badge, purpose and one line per version: `v1`, a status badge, and the vault path once the definition has been written to the vault. Below it, **May claim** lists the stages the role may claim, as `workflow.stage`. A version has these statuses:

Status	Meaning
Active	Available for hiring agents and claiming stages.
Waiting for your approval	Submitted. It is on your desk as an item; the line has an **Open the desk** link.
Draft	Saved but not submitted. Buttons: **Edit**, **Submit for approval**, **Discard**.
Replaced by a newer version	A newer version was approved; this one stays in the list for reference.
Rejected	You rejected it on the desk. Buttons: **New version** (starts from it) and **Discard**.

Built-in roles

Ten built-in roles ship with the system and can never be removed: Business analyst, Builder, Fact checker, Curator, DevOps engineer, Researcher, Reviewer, Scout, Tester and Technical writer. Each card shows a **Built in** badge, its department and its version. The core rules of a built-in role (for example test ownership and review gates) cannot be changed, but you can extend the role with extra instructions, skills and standing orders. An extended role shows an Extended badge with the number of additions and the version they were written against.

Every built-in role except the DevOps engineer has an **Extend** button. Only the organization owner sees **New custom role** and **Extend**.

8.2 Extending built-in roles

Click **Extend** on a built-in role to open the dialog **Extend [role]**. It says that your additions are added to the role's instructions for every agent in this role, in every project of your organization. At the top it lists the role's core rules, which cannot be changed.

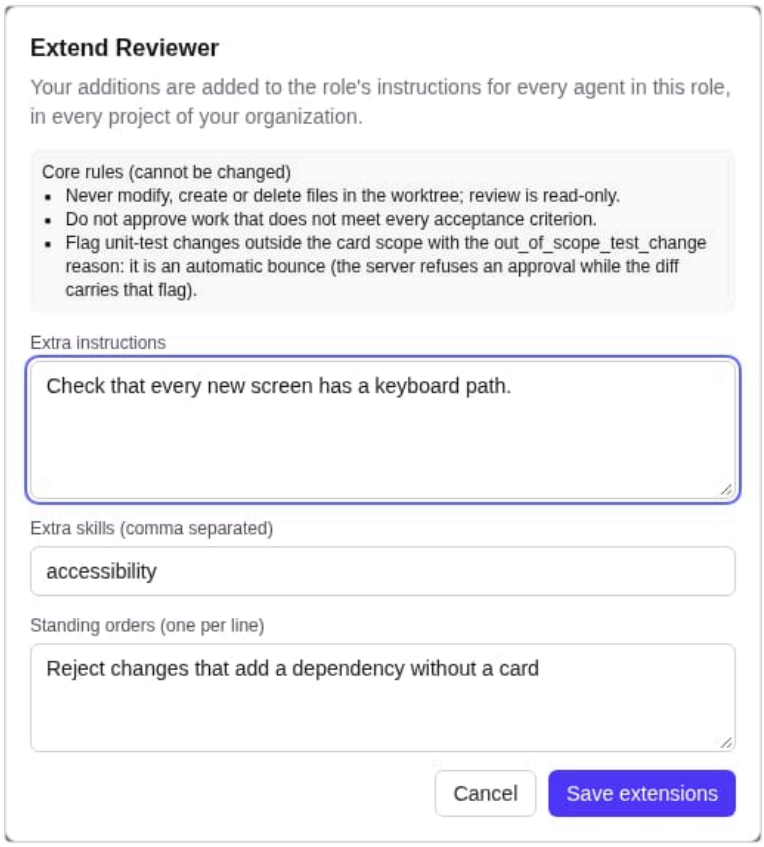


Figure 106 — Extend Reviewer dialog: the three core rules, Extra instructions, Extra skills and Standing orders, with Cancel and Save extensions (Pieter Demo)

Extension fields

- **Extra instructions:** free text added to the role's instructions. The demo has "Check that every new screen has a keyboard path."
- **Extra skills (comma separated):** for example `accessibility`.
- **Standing orders (one per line):** for example "Reject changes that add a dependency without a card".

Save extensions saves them straight away; an extension does not go through the desk. At least one of the three fields must be filled. After saving, the role card shows the `Extended` badge.

New defaults for built-in roles

The shipped definition of a built-in role can get a new version. If you extended that role against an older version, the **HR board** shows a section **New defaults for built-in roles**, for example "Tester has a new default (version 2); your extensions were written for version 1." with the number of agents still on the old version. **Reset to default** opens a dialog that lists what will be removed (your extra instructions, skills and standing orders). Confirm with **Reset and drop extensions**: the extensions are dropped, the core rules do not change, and the agents of that role move to the new version.

8.3 Workflows and the graph

Organization → Workflows tab shows one card per workflow (custom and built-in) and the button **New custom workflow**. Four built-in workflows ship with the system: **code**, **code-bugfix**, **feed** and **research**. Each version of a workflow is one line with its status and, when cards use it, how many cards are in flight on that version. Press **View** on a line to show its graph and stage list, and **Hide** to close them. Custom workflows also have **New version**, and drafts have **Edit**, **Submit for approval** and **Discard**.

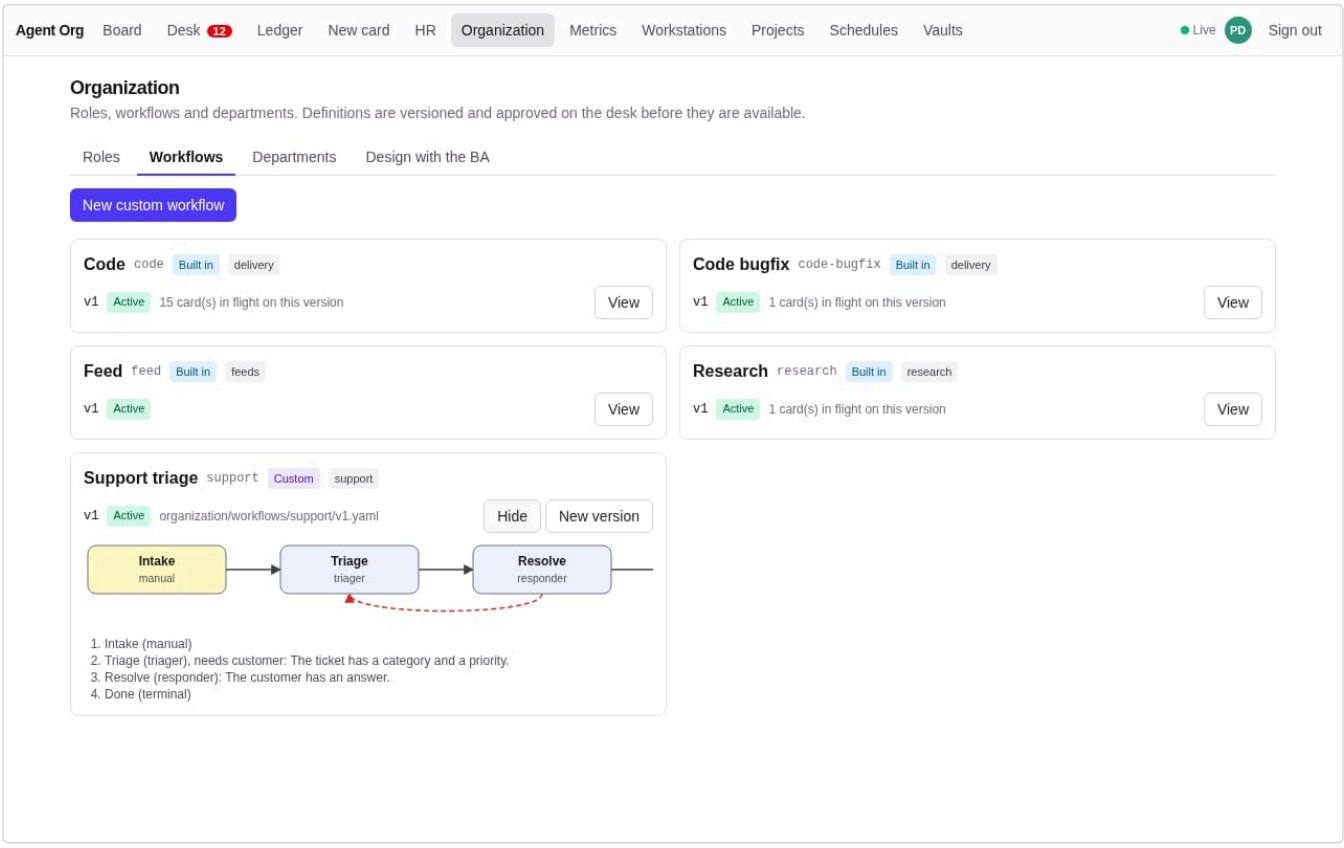


Figure 107 — Workflows tab: Code (15 cards in flight), Code bugfix, Feed, Research and the custom workflow Support triage, opened to show its graph and stage list (Pieter Demo)

The workflow graph

The graph shows stages (boxes) and outcomes (arrows) that move a card from one stage to the next. The second line in a box is the claiming role for an agent stage, or the stage kind. Yellow boxes are manual stages (moved by a person or the BA), blue boxes are agent stages and green boxes are terminal stages. Red dashed arrows are failure routes. A terminal stage has no outgoing arrows. In the figure, Support triage runs Intake (manual), Triage (role triager), Resolve (role responder) and Done; the red dashed arrow leads from Resolve back to Triage.

Stage list

Below the graph, a numbered list gives every stage: its name, the claiming role in brackets (with `senior` if only senior agents may claim it) or its kind, the fields needed to enter it, and its done criteria, for example "Resolve (responder): The customer has an answer."

Cards in flight stay on their workflow version: every card is tied to the workflow version it was created on, and the version line shows how many cards are in flight on it.

8.4 The editor and validation

Click **New custom workflow** (or **New version** / **Edit** on a custom workflow) to open the editor above the list. Only the organization owner can do this. The editor checks your definition while you type and shows the result under **Validation**.

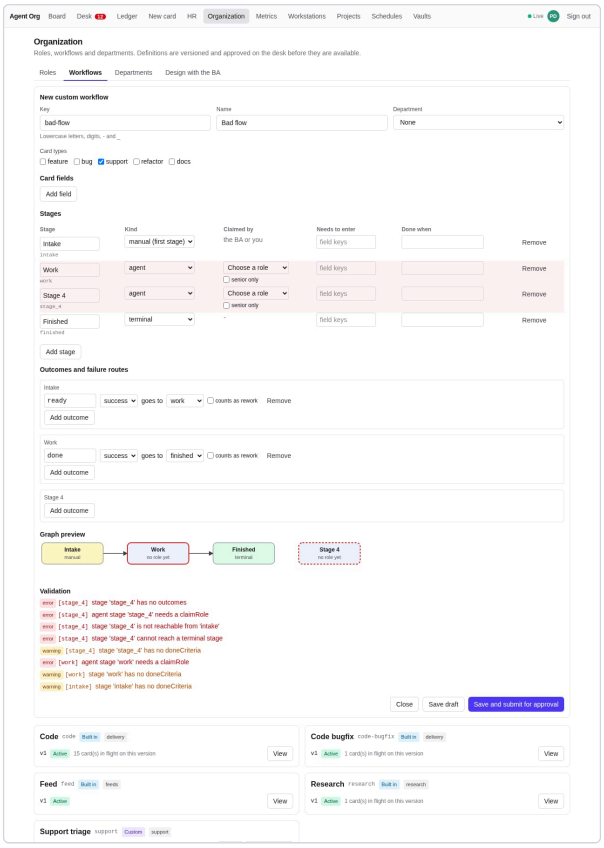


Figure 108 — Workflow editor with a deliberately broken draft (key bad-flow): the Work and Stage 4 rows are marked red and the Validation list shows errors and warnings (Pieter Demo)

Editor fields

- **Key**, **Name** and **Department** (a list of departments, or None). The key is lowercase letters, digits, `-` and `_`, and starts with a letter. It cannot be a built-in key (`code`, `code-bugfix`, `feed`, `research`).
- **Card types:** checkboxes `feature`, `bug`, `support`, `refactor`, `docs`. At least one is required.
- **Card fields: Add field** adds extra fields to the card (key, label, type, required).
- **Stages:** a table with the columns Stage (name; the id is made from it), Kind (`agent`, `manual (first stage)` or `terminal`), Claimed by (a role, and a **senior only** checkbox), Needs to enter (field keys), Done when, and **Remove**. **Add stage** adds a row.
- **Outcomes and failure routes:** for every stage, rows with an outcome id, `success` or `failure`, **goes to** (a stage), a **counts as rework** checkbox and **Remove**. **Add outcome** adds a row.
- **Graph preview** of what you have so far. A stage with an error has a red border; a stage nothing leads to has a dashed red border (Stage 4 in the figure).

Validation rules (from packages/shared/src/workflows/validate.ts and custom.ts)

1. **Shape and ids:** the definition has at least two stages and every field has the right type; stage ids are unique; outcome ids are unique within a stage; the first stage exists.
2. **Terminals and outcomes:** there is at least one terminal stage; a terminal stage has no outcomes; every other stage has at least one outcome and at least one of them is a success outcome.
3. **Roles:** an agent stage names a role (`claimRole`); the role exists in the catalogue; the role may claim that stage (it is in the role's May claim list); a senior-only stage needs a role that has a senior level.
4. **Targets:** every outcome goes to a stage that exists.
5. **Reachable:** every stage can be reached from the first stage.
6. **Can finish:** every stage can reach a terminal stage, and so can the target of every failure outcome.
7. **Fields:** every field a stage needs to enter is a known card field.
8. **Effects:** an outcome that spawns a bug child must go to the same stage; a finalize outcome must go to a terminal stage; resume-parent needs the card type `bug`.
9. **Conditions:** a stage skipped by a `when` condition must test a yes/no field.

Rule 7 applies to the Needs to enter column. Rules 8 and 9 only matter for effects and `when` conditions; the editor has no controls for them, and a custom outcome may not have an effect. A custom workflow also has to meet these extra rules:

- The key matches `^[a-z][a-z0-9_-]*$` and is not a built-in key; it uses the field set `custom`; the card types are limited to those in the editor (not `research`, `feed` or `regression`); the department exists.

- Only the first stage may be manual.
- The DevOps engineer cannot claim a stage.
- Outcomes move cards with a plain stage move: no structured data and no effects. The card-specified event belongs to the first stage only.

Two warnings do not block anything: a stage without done criteria, and more than 12 stages. In the figure, "stage 'work' has no doneCriteria" is a warning.

Save draft vs. Save and submit for approval

- **Save draft:** saves the definition as a draft, even with errors; the errors stay listed. You can edit it again later.
- **Save and submit for approval:** saves, and if there are no errors, puts it on the desk. It becomes `Active` only after you approve it there. With errors it stays a draft.
- **Close** closes the editor.

The editor for roles works the same way. **New custom role** opens the form in the figure below: Key, Name, Department, Purpose, Instructions, Core rules (one per line), Seniority levels (junior, senior), Default runtime, Default model, Effort, Tools (read, edit, shell, web, may commit), Denied shell patterns, Skills and May claim (`workflow.stage`, comma separated; `*` for any stage). The error in the figure, "key: Invalid string: must match pattern", appears because the key is still empty.

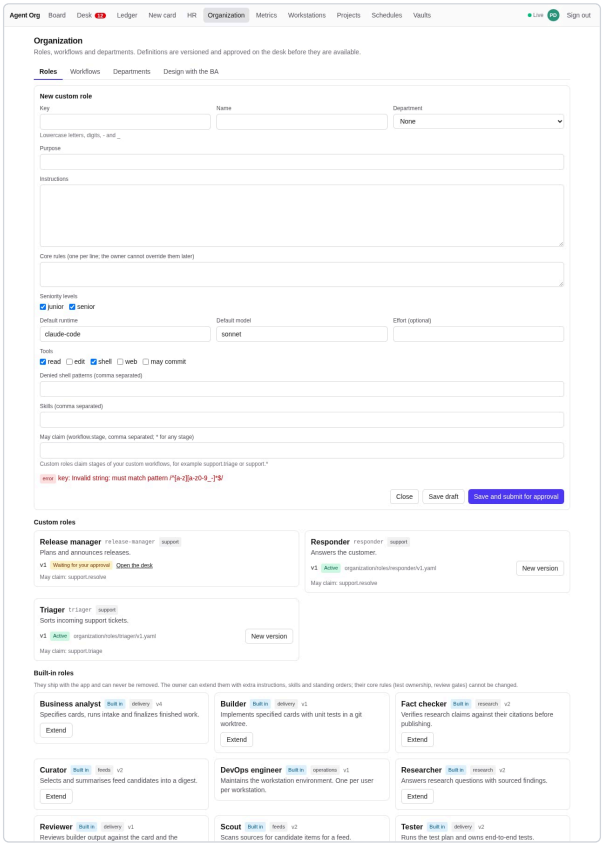


Figure 109 — New custom role form with an empty key and the validation error; the custom and built-in roles are listed below it (Pieter Demo)

8.5 Departments

Organization → Departments tab lists departments. A department groups the roles and workflows of one function. Each card shows the department name, a **Built in** badge for the four that ship with the system (delivery, operations, research, feeds), and the keys of its roles and workflows. Below the explanation is a **New department** field with a **Create department** button (owner only). A role or workflow chooses its department in its editor. A project has no department of its own: it shows the department of its workflow. The **Projects** page and the **HR board** each have a Department filter.

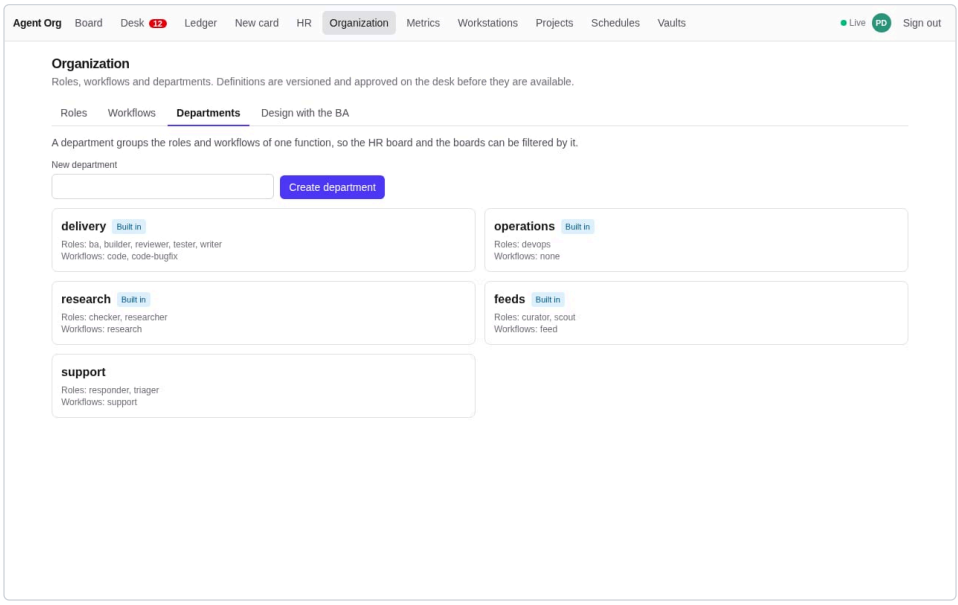


Figure 110 — Departments tab: delivery (roles ba, builder, reviewer, tester, writer; workflows code, code-bugfix), operations, research and feeds (all built in), and the custom department support (roles responder, triager; workflow support) (Pieter Demo)

8.6 Designing with the BA and approving on the desk

Organization → Design with the BA tab is where the business analyst (BA) drafts roles and workflows with you. Only the organization owner can use it. The tab says: "Describe the role or workflow you need. The BA drafts it, asks what is unclear, validates it and puts it on your desk; it is available only after you approve it." If you have no business analyst that is not tied to a project, the tab asks you to hire one on the HR board first. Otherwise choose the **Business analyst** (if you have several) and press **Start a design chat**. The conversation has a **Message** field and a **Send** button.

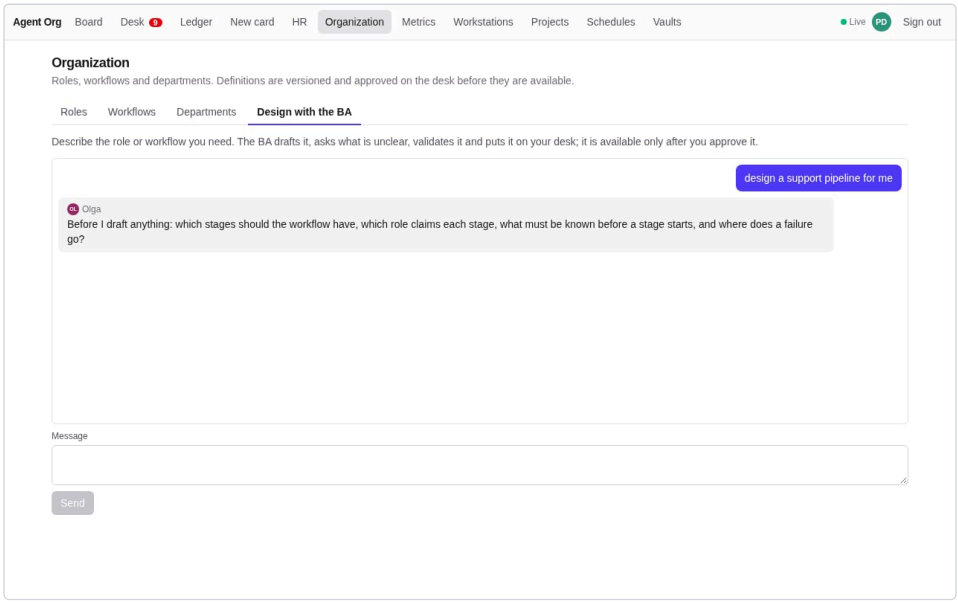


Figure 111 — Design with the BA chat: Pieter Demo wrote "design a support pipeline for me" and Olga (BA) asks which stages the workflow should have, which role claims each stage, what must be known before a stage starts, and where a failure goes (Pieter Demo)

The design process:

1. You describe what you need in the chat.
2. The BA asks what is unclear.
3. The BA drafts a role, a workflow or a department with its `aom org` commands (section 10.7), validates the draft with the same rules as the editor, and fixes the errors it gets back.
4. The BA submits the draft. Nothing is approved in the chat: the draft goes to your desk.
5. On the desk the item is titled **Approve role "[name]" (v1)** or **Approve workflow "[name]" (v1)**. It has the badges **Role or workflow** and **Non-blocking**, a short description, a link **Open the Organization page** ("to see the stage table, graph and validation. Nothing is available until you approve.") and the buttons **Approve**, **Reject** and **Dismiss**.

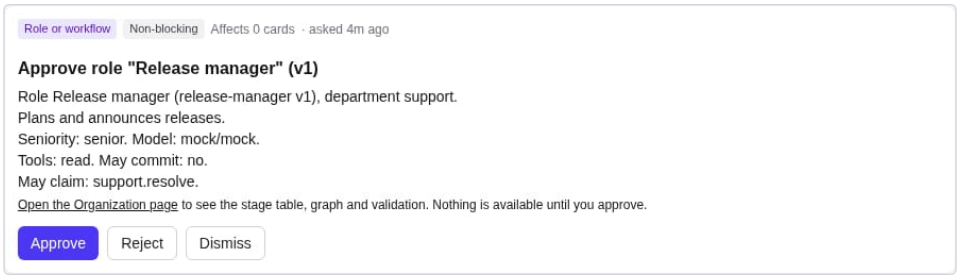


Figure 112 — Desk item Approve role "Release manager" (v1): seniority senior, model mock/mock, tools read, may claim support.resolve, with Approve, Reject and Dismiss (Pieter Demo)

Roles must be approved before workflows that use them. When you approve, the definition is validated again against the catalogue as it is then. A workflow that names a role that is not approved yet is refused with the hint "approve the role first". An approved definition is Active and can be used to hire agents and to create cards. A rejected one stays in the list as Rejected until you discard it.

Where definitions are stored

The version history lives in the site. When you approve a role or workflow, its definition is also written to your vault as a YAML file: `organization/roles/<key>/v<N>.yaml` or `organization/workflows/<key>/v<N>.yaml` (the path is shown on the version line, see the figure in 8.1). This happens only when you have a vault that is not tied to a project (see 7.3). Departments are kept in the site, not in the vault.

8.7 For additional managers on a shared project

If you are an additional manager (not the owner) on a shared project, the Organization page shows a note: **"You are an additional manager here: this is the owner's catalogue. Their roles, workflows and standing orders apply to the shared project's cards, and only the owner can change them."**

You can read the owner's catalogue but not change it: no **New custom role**, **Extend**, **New custom workflow** or **Create department**, and the Design with the BA tab says only the owner designs. A list **Organization** at the top switches between "My organization" and "[KEY]: the owner's organization" for every project shared with you. You see only the owner's active definitions, not their drafts.

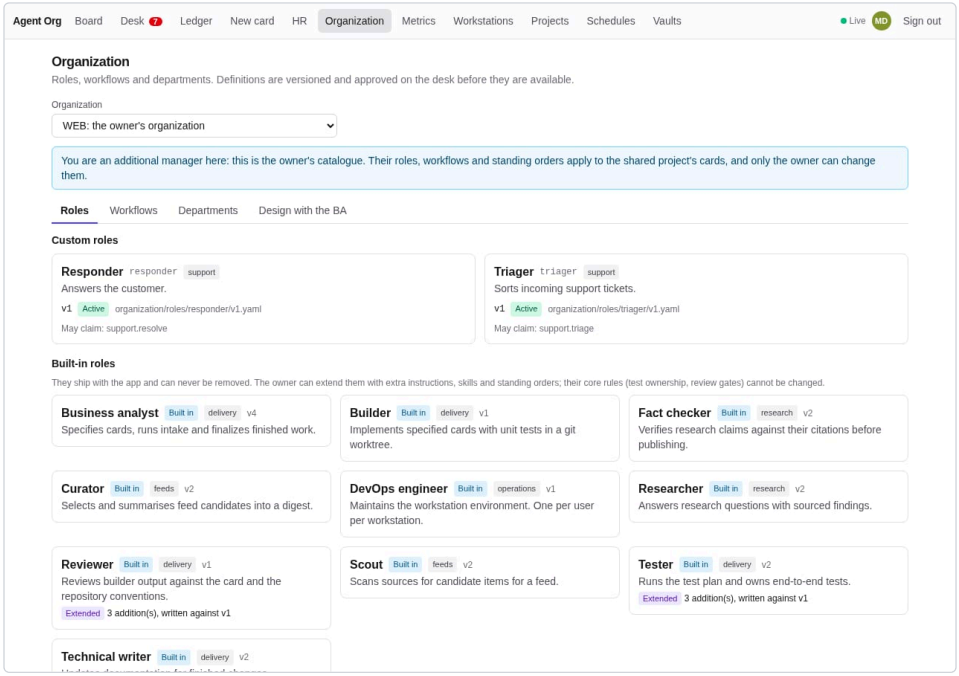


Figure 113 — Organization page as Maya (additional manager): the Organization list set to "WEB: the owner's organization", the read-only note, and the owner's active custom roles Responder and Triager (Maya Demo)

9 Administration and on a phone

9.1 The Admin page

The **Admin** item in the top menu (shown only to administrators) manages users, invitations and workstations for the whole system. Someone who is not an administrator and opens the page is sent back to the home page. Being an administrator gives no access to other people's projects, cards, agents or desk.

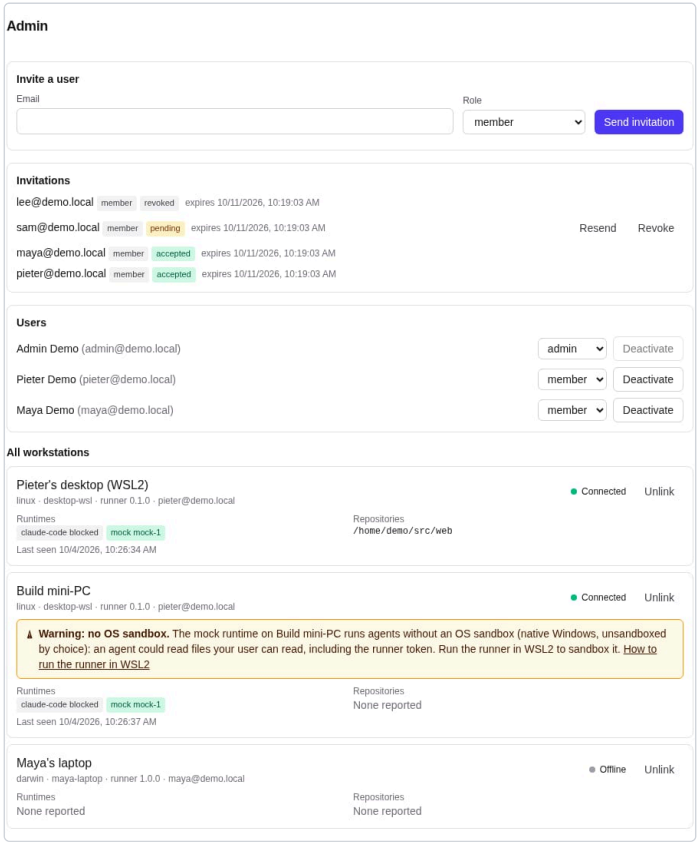


Figure 114 — Admin page: Invite a user, Invitations (lee@demo.local revoked, sam@demo.local pending with Resend and Revoke, maya and pieter accepted), Users (Admin Demo admin, Pieter Demo and Maya Demo members) and All workstations (Pieter's desktop (WSL2), Build mini-PC with the amber sandbox warning, Maya's laptop offline) (Admin Demo)

Invite a user

Enter an email address, choose a role (**member** or **admin**) and press **Send invitation**. The invitee receives an email with an accept link. On the accept page they enter a display name and a password (12 to 256 characters) and then sign in with their email. If a user with that email exists, or a pending invitation for it exists, the server refuses the new invitation. If the email cannot be sent, the invitation still exists and you can use **Resend** later.

Invitations list

Every invitation is listed with the email, a role badge, a status badge and its expiry time:

Status	Meaning	Actions
pending	Invitation sent, not yet accepted	**Resend** (sends a new link and sets a new expiry), **Revoke** (cancels it)
accepted	The invitee accepted and has an account	None
revoked	The invitation was cancelled	None

An invitation expires 7 days after it was sent (or resent). The demo's accepted rows also show an expiry time.

Users list

Every user is listed with the display name and email, and a deactivated badge if the account is deactivated. For each user you can:

- Change the role with the dropdown (`admin` or `member`). It is disabled for yourself. The last active administrator cannot be demoted.
- Click **Deactivate** on an active user, or **Reactivate** on a deactivated one. The button is disabled for yourself. The last active administrator cannot be deactivated.

All workstations

Every workstation paired by any user is listed here with the platform, hostname, runner version and the owner's email, the connection status, runtimes, repositories and last-seen time, and an **Unlink** button. Administrators see this list. The DevOps panel is not shown here; it is on the Workstations page of the workstation's owner, who sees only their own workstations (and can unlink them too).

Creating the first administrator

When you first deploy the system there are no users. Create the first administrator on the server with:

```
pnpm --filter @aom/server admin:create --email you@example.com --name "You"
```

The command asks for anything you leave out: email, display name and a password (12 to 256 characters; it can also be given with `--password` or the `AOM_ADMIN_PASSWORD` variable). From then on that administrator invites everyone else on the Admin page.

9.2 Deactivating and unlinking

Deactivation

On the Admin page, click **Deactivate** next to a user. The server refuses it for yourself and for the last active administrator. Otherwise it does this, in order:

1. For every project where the user is an additional manager (active or invited), it removes them as in section 7.5: their agents' claims are released, open cards homed on their workstations forget that home, their agents move to the project owner with status `needs_rehome`, and the cards they own become the owner's.
2. Every active workstation of the user is unlinked (see below).
3. The account is marked `deactivated`, and all the user's remaining agents (except the DevOps engineers) get the status `needs_rehome` and lose their workstation.
4. The user's sign-in sessions are deleted, so they are signed out and cannot sign in.

Projects the user owns are not changed. Click **Reactivate** on the Admin page to make the account active again. Nothing else comes back by itself: the agents stay in `needs_rehome` until they are re-homed onto a workstation, and the workstations stay unlinked and must be paired again.

Unlinking a workstation

The owner of a workstation (Workstations page) or an administrator (Admin page) clicks **Unlink**. A dialog **Unlink [name]?** says that agents on this workstation will be released and its runner token stops working; **Confirm unlink** does it. The server then marks the workstation unlinked, releases the card claims held on it (each card gets the system message "Claim released: the workstation was unlinked."), ends the shifts still open on it as lost, and fails the commands still queued for it. An unlinked workstation no longer appears in any list. Workstations are unlinked automatically when a user is deactivated.

9.3 On a phone (390 px)

The app also works in a phone browser. The figures below are 390 px wide. The top menu is the same row of items as on the desktop (Board, Desk with its count, Ledger, New card, ...) and scrolls sideways; on the phone only the first items are visible and the rest are reached by swiping. Wide layouts become one column.

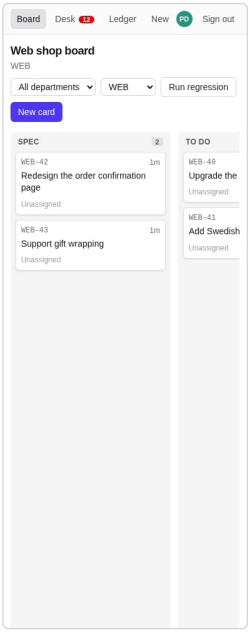


Figure 115 — Board of WEB on a phone: the Spec column (WEB-42, WEB-43) and the start of To do, with the department and project filters, Run regression and New card (390 px, Pieter Demo)

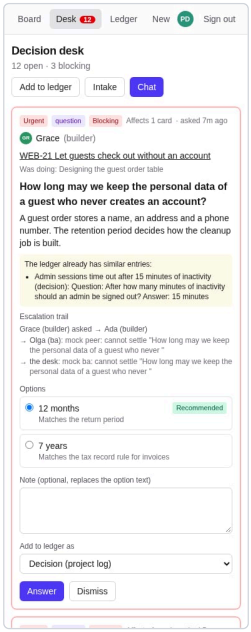


Figure 116 — Decision desk on a phone: 12 open, 3 blocking, with Grace's blocking question for WEB-21 about guest data retention, its options and Answer and Dismiss (390 px, Pieter Demo)

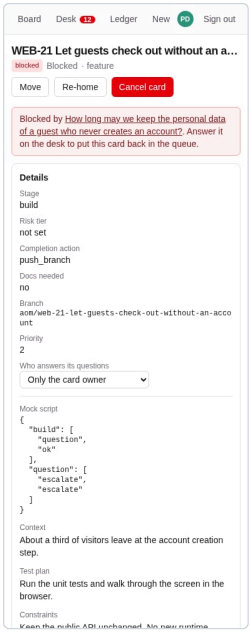


Figure 117 — Card WEB-21 on a phone: blocked banner, Move, Re-home and Cancel card buttons, and the Details panel (390 px, Pieter Demo)

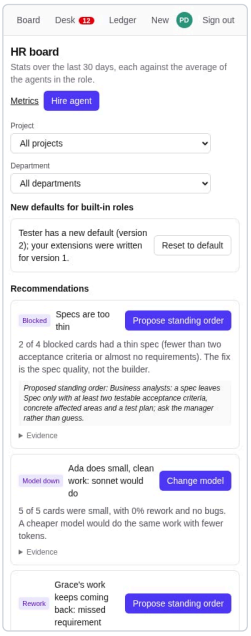


Figure 118 — HR board on a phone: the New defaults for built-in roles section (Tester) and the first recommendations (390 px, Pieter Demo)

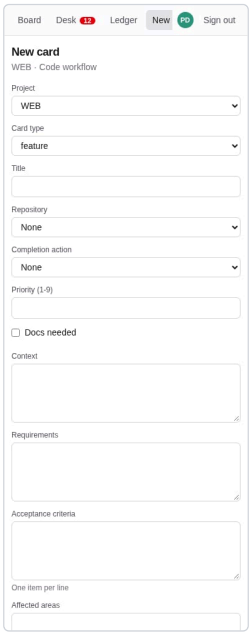


Figure 119 — New card form for WEB on a phone (390 px, Pieter Demo)

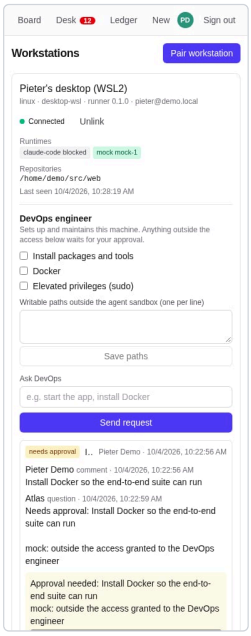


Figure 120 — Workstations page on a phone: Pieter's desktop (WSL2) with its DevOps engineer panel (390 px, Pieter Demo)

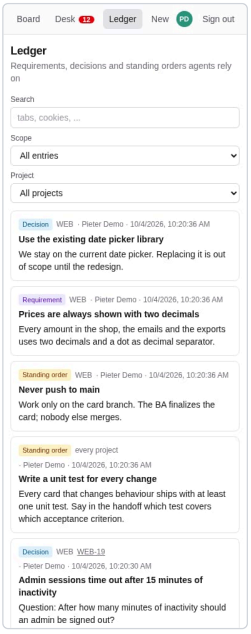


Figure 121 — Ledger on a phone: search, scope and project filters and the first entries (390 px, Pieter Demo)

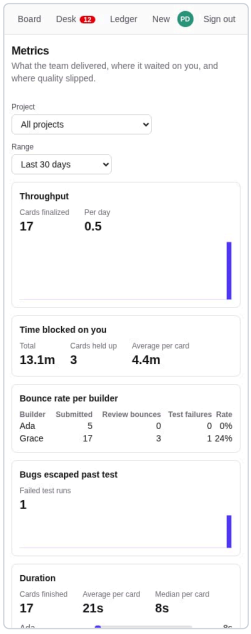


Figure 122 — Metrics on a phone: Throughput, Time blocked on you, Bounce rate per builder and more (390 px, Pieter Demo)

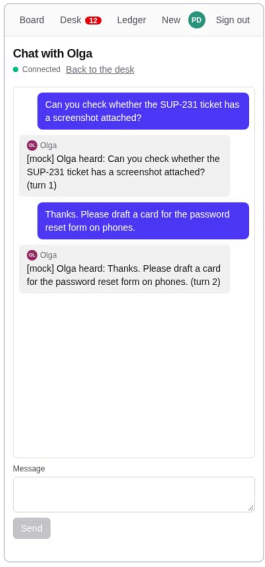


Figure 123 — Chat with Olga as a page on a phone (Connected, Back to the desk): the replies start with [mock] because the demo uses the mock runtime (390 px, Pieter Demo)

A conversation also exists as a page of its own, with a **Back to the desk** link at the top; the last figure shows that page. The chat drawer on the desk fills the whole width on a phone.

10 Workstations and the runner

10.1 The Workstations page

The **Workstations** page lists the workstations you have paired, and has the button **Pair workstation**. Each card shows the workstation's name, platform, hostname, runner version, connection status, runtimes and repositories, when it was last seen, and its DevOps engineer panel (section 7.6). You see only your own workstations here; administrators see all of them on the Admin page (section 9.1). With none paired the page says "No workstations paired. Pair one to run agents on your machine."

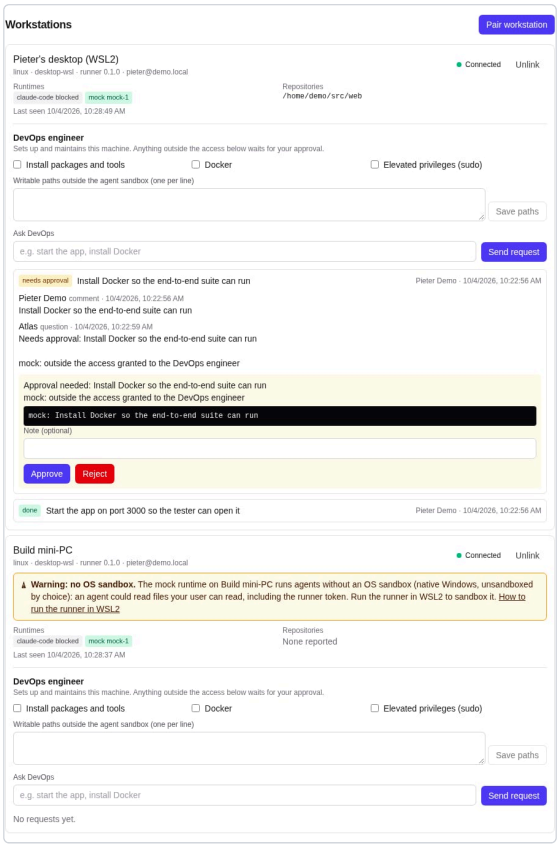


Figure 124 — Workstations page: Pieter's desktop (WSL2) and Build mini-PC, both Connected, with runtimes, repositories and a DevOps engineer panel under each; Build mini-PC has an amber sandbox warning (Pieter Demo)

Workstation card details

- **Connection status:** a green dot and **Connected** when the runner is online, a grey dot and **Offline** when it is not.
- **Platform, hostname and runner version:** for example `linux · desktop-wsl · runner 0.1.0`. The platform is the Node platform name (`linux`, `darwin`, `win32`).
- **Runtimes:** one badge per runtime with its version, for example `mock mock-1`. A green badge means the runtime can run agents; a grey one means it cannot (in the demo `claude-code blocked`). When the OS sandbox is missing, a note under the card says why, for example "Native Windows: install the runner in WSL2".
- **Repositories:** the local paths of the clones the runner found that match a repository of one of your projects (for example `/home/demo/src/web`), or "None reported". Add folders to scan with `aom-runner repos add`.
- **Last seen:** when the workstation last connected.

Click **Unlink** to remove the workstation. A dialog asks you to confirm; section 9.2 lists what unlinking does. The owner of a workstation can unlink it; so can an administrator.

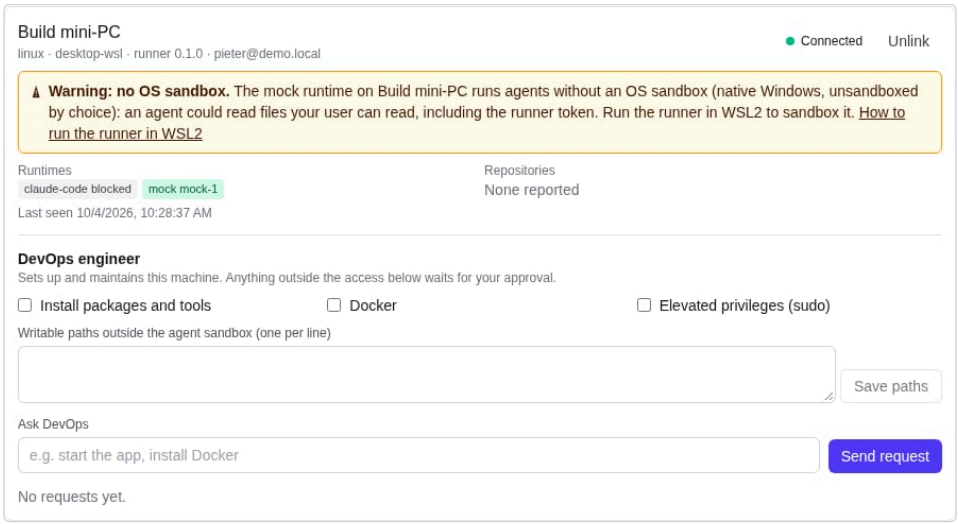


Figure 125 — Card of Build mini-PC with the amber warning "Warning: no OS sandbox", runtimes claude-code blocked and mock mock-1, no repositories reported, and its DevOps engineer panel (Pieter Demo)

> **Good to know**
> The demo makes the mock runtime report "unsandboxed" for Build mini-PC, so the warning text says "native Windows" although the card shows `linux`.

10.2 Pairing a workstation

Pair workstation on the Workstations page opens a dialog. Press **Generate pairing code**: the dialog shows a code (for example `G360-RN71`), the text "Expires [time]. Shown once.", the pair command with the server address and your code filled in, a **Copy command** button and the reminder "Then run aom-runner start. This page updates when it connects." The code is valid for 10 minutes and for one pairing. Run the command on the machine that will host your agents.



Figure 126 — Pair workstation dialog: the code G360-RN71, its expiry time, the pair command and Copy command (Pieter Demo)

The runner CLI: pair, repos add, start, status, doctor, rotate-token

The runner is installed as `aom-runner` (`./aom-runner` in the tarball). Every command accepts `--data-dir <path>`, which overrides the data directory (default: `$AOM_RUNNER_HOME`, else `~/.aom-runner`, on Windows `%LOCALAPPDATA%\aom-runner`).

Command	Purpose
`aom-runner pair --server <url> --code <code> [--name <name>] [--mock]`	Pair this machine with the server using the code from the dialog. `--server` and `--code` are required. `--name` is the workstation name (default: the hostname). `--mock` enables the mock runtime. It writes `config.json` (mode 0600) and keeps the settings of an earlier config.
`aom-runner repos add <path>`	Add one folder to `repoRoots`. The runner scans each root to depth 3 for git repositories that have an `origin` remote, when it connects and every 10 minutes, so no restart is needed. Run it once per folder.
`aom-runner repos list`	Print the repositories found: path, remote URL and branch.
`aom-runner start [--mock]`	Connect to the server and run shifts, in the foreground (stop with Ctrl+C). `--mock` enables the mock runtime for this process (also `AOM_RUNNER_MOCK=1`). To keep it running, use a service (section 10.4).
`aom-runner status`	Print the data directory, server, workstation id, repo roots, whether the mock runtime is enabled, and whether the runner process is running, with its connection state, active shifts and pending outbox entries.

Command	Purpose
`aom-runner doctor`	Check Node (22 or newer), git (2.40 or newer), the config file, the `claude` command (a warning if missing), the sandbox state (`enforced`, `unsandboxed` or `unavailable`, with the reason), Docker (optional), the repositories found, and that the server answers on `/api/health`. It exits with an error if a check failed.
`aom-runner rotate-token`	Ask the server for a new runner token and store it in `config.json`. Stop the runner first; the command refuses to run while it is running. Then start it again.

10.3 Windows: WSL2 is the default

On Windows, run the runner inside WSL2 (Windows Subsystem for Linux). This is the recommended setup: inside WSL2 the runner and its agents get the same OS-level isolation as on native Linux. Sandboxed agents cannot read the runner token or your SSH keys (section 10.6).

Why WSL2

Claude Code's sandbox runs on macOS (Seatbelt) and Linux/WSL2 (bubblewrap + socat) but not on native Windows. On native Windows the `claude-code` runtime therefore reports itself unavailable and no agent is scheduled, unless you opt in to running unsandboxed (section 10.5). Unsandboxed agents can read anything the Windows user can read, including runner credentials and SSH keys. WSL2 avoids this risk.

Setup steps (short version)

1. Enable WSL2: `wsl --install -d Ubuntu` (administrator PowerShell), then reboot and create the Linux user when Ubuntu first starts.
2. Turn on systemd (needed for the service): edit `/etc/wsl.conf` in Ubuntu (add `[boot]` and `systemd=true`), then run `wsl --shutdown` in PowerShell and reopen Ubuntu.
3. Install base tools: `sudo apt update && sudo apt install -y git curl ca-certificates`. Install the GitHub CLI `gh` and run `gh auth login` if agents will open pull requests.
4. Install sandbox dependencies: `sudo apt install -y bubblewrap socat`.
5. Install Node 22 with nvm (instructions on https://github.com/nvm-sh/nvm), then `nvm install 22 && nvm alias default 22`.
6. Install Claude Code (2.1.257 or newer): `npm install -g @anthropic-ai/claude-code`, then run `claude` once to sign in (or put an OAuth token in the config, section 10.8).
7. **Clone repos in the WSL filesystem**, not under `/mnt/c`: `~/src` is much faster, and `/mnt/c` breaks git file modes.
8. Unpack the runner tarball in Ubuntu, then pair it, add your repository folder and run `aom-runner doctor`; the sandbox line must say `enforced`.
9. Run it as a systemd user service (see "Run as a service" below).

For full setup instructions see `docs/RUNNER.md` in the repository.

10.4 macOS and Linux

On macOS and Linux, the runner is installed in your home directory and pairs directly with the system. macOS uses its built-in Seatbelt sandbox and needs nothing extra. Linux (and WSL2) needs `bubblewrap` and `socat` (`sudo apt install -y bubblewrap socat`).

Install and pair

1. Ensure Node 22.23+ and Git 2.40+ are installed.
2. Build the tarball with `pnpm package:runner` (it writes `release/aom-runner-<version>.tgz`) or take a prebuilt one, then `tar xzf aom-runner-0.1.0.tgz && cd aom-runner`.
3. Pair: `./aom-runner pair --server https://aom.example.com --code XXXX-XXXX --name "Desktop"`.
4. Add your repository folders, one command per folder: `./aom-runner repos add ~/src`.
5. Check the setup: `./aom-runner doctor`. The sandbox line should say `enforced`. Then start it: `./aom-runner start`.

Run as a service

systemd (Linux and WSL2): Create `~/.config/systemd/user/aom-runner.service` with `ExecStart` set to the full path of `aom-runner start` and `Restart=always`, then enable and start it:

```
systemctl --user enable aom-runner && systemctl --user start aom-runner
```

launchd (macOS): Create `~/Library/LaunchAgents/com.aom.runner.plist` with `RunAtLoad` and `KeepAlive` set to true and the program arguments `aom-runner` and `start`, then:

```
launchctl load ~/Library/LaunchAgents/com.aom.runner.plist
```

See `docs/RUNNER.md` for complete plist and systemd templates.

10.5 Native Windows (unsandboxed opt-in)

Native Windows cannot sandbox agents. With the default configuration (`sandbox` is `required`), the `claude-code` runtime reports `unavailable` ("Native Windows: install the runner in WSL2") and no shifts are scheduled. To run agents on native Windows anyway, you must opt in explicitly in the runner's `config.json`:

```
{ "runtimes": { "claude-code": { "sandbox": "best_effort", "allowUnsandboxed": true } } }
```

`best_effort` alone is not enough on Windows: without `allowUnsandboxed: true` the runtime stays `unavailable`. (`"sandbox": "off"` is also an explicit opt-in and reports `unsandboxed` on every platform.) With this configuration, the runtime reports `unsandboxed` and the web UI shows a persistent amber warning on the workstation, on every affected agent card, and on the hire and re-home forms (section 10.9). Agents run without an OS-level boundary. The permission rules and the reduced environment still apply, but a prompt-injected agent can read anything your Windows user can read: your SSH keys, browser profiles, the runner token and the provider credentials in `config.json`.

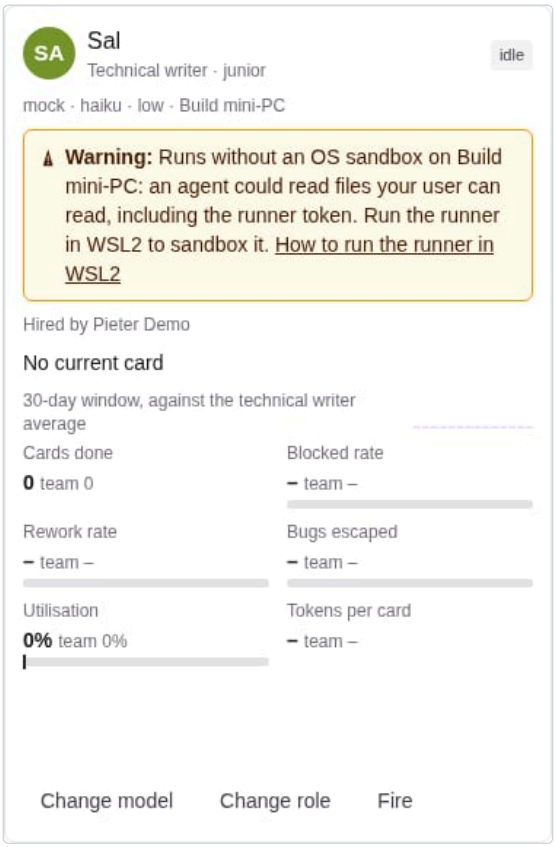


Figure 127 — Agent card of Sal (technical writer, junior, mock haiku, idle) on Build mini-PC: the amber warning "Runs without an OS sandbox on Build mini-PC", stats against the team, and Change model, Change role and Fire (Pieter Demo)

Recommendation: Use WSL2 instead. If you must use native Windows, use a dedicated low-privilege Windows user for the runner.

10.6 What a sandboxed agent can and cannot touch

Each agent has its own folders under the runner data directory: `agents/<id>/home` (its `HOME`), `claude` (its `CLAUDE_CONFIG_DIR`), `scratch` and `tmp`. Two layers limit what a shift can do. Claude Code's OS sandbox (bubblewrap on Linux and WSL2, Seatbelt on macOS) wraps shell commands. Claude Code's permission rules govern its file and web tools, which the sandbox does not cover; they run with `dontAsk`, so anything not allowed is denied. On an unsandboxed workstation (section 10.5) only the permission rules and the reduced environment remain. The lists below follow `apps/runner/src/adapters/claude-policy.ts`.

Writable

- The card's worktree, through the Edit tool. A role with writable globs (a tester: the project's E2E globs) may only change files matching them; a role without the `edit` tool cannot edit at all.
- The agent's own home, scratch and temp folders (the shell's `TMPDIR` is the agent's `tmp`, not the shared `/tmp`).
- The main repository's `.git` directory, because a linked worktree commits into it, except its `hooks` and `config`.

Readable, not writable

- The vault folders passed with `--add-dir` (read for context; writes are denied).

- The runner's `bin` directory with the `aom` shim.
- The Node install the runner runs on, and the directories listed in `toolchainPaths` (section 10.8). The sandbox hides all of `/home` from shell commands; only these and the shift's own folders are opened again.

Denied

- The runner's data directory: `config.json` (runner token, Jira and provider credentials), `outbox`, `outbound`, `active-shifts`, `vaults.json`, `status.json`, `logs`, other agents' folders and other cards' worktrees. The agent's own folders are carved out.
- Credential stores in the real home: `~/.ssh`, `~/.aws`, `~/.azure`, `~/.gnupg`, `~/.kube`, `~/.docker`, `~/.netrc`, `~/.npmrc`, `~/.pypirc`, `~/.git-credentials`, `~/.config/gh`, `~/.config/gcloud`, `~/.claude` and `~/.claude.json`.
- Editing the agent's own Claude config directory and the `.git` folder of its worktree.
- Shell commands for every role: `git push`, `git rebase`, `git checkout`, `git switch` and `git reset --hard`, plus the role's own denied shell patterns.
- Claude Code tools that reach outside the shift: `ListAgents`, `SendMessage`, `RemoteTrigger`, `CronCreate`, `CronDelete`, `CronList`, `ScheduleWakeup`, `PushNotification`, `EnterWorktree` and `ExitWorktree`. Also every tool the role does not have: no `edit` means no Edit, Write and NotebookEdit; no `web` means no WebFetch and WebSearch; no `shell` means no Bash.
- Network from the shell: only package registries and source hosts (npm, Yarn, PyPI, crates.io, the Go proxy, RubyGems, Maven and Gradle hosts, github.com) plus the hosts in `extraAllowedDomains` in the config. A role without `shell` gets none. Web tools follow the role's `web` permission instead.
- The runner's environment: an agent gets only a short list of variables (`PATH`, locale, proxy and certificate variables) and the one provider credential its runtime needs. `GH_TOKEN`, `SSH_AUTH_SOCK` and cloud keys never reach it.

Special case: the aom command

The `aom` command is excluded from the OS sandbox. On Linux a sandboxed process cannot open a Unix socket, so `aom` runs outside it. It only talks to the runner over a private socket with the shift's token and writes nothing itself. It is the way an agent reports to the server (section 10.7).

10.7 The aom command inside a shift

While a shift is running, the environment holds `AOM_IPC_URL` and `AOM_SHIFT_TOKEN` (and `AOM_IPC_SOCKET`, which the command prefers). Outside a shift the command refuses to work. Exit code 0 means recorded, 1 an error or a rejected report (fix it and run it again within the shift), 2 a verb that is not available. `aom --help` prints the list of verbs; there is no help text per verb. The verbs:

Verb	Used by	What it does
`card [--json]`	Any	Print the card and the brief of this shift.
`card create --file`, `card update <REF> --file`, `card specify <REF>`	BA (intake)	Draft a card from a JSON file, change a draft, and specify it.
`ledger search "<query>" [--scope all|feature|decision|standing_order]`	Any	Search answers, decisions and standing orders.
`comment --note`	Any	Post a note to the card thread without ending the shift.
`question --text [--option]... [--recommend n] [--doing] [--non-blocking] [--force]`	Any	Ask the manager a question. It blocks the card unless `--non-blocking`. It checks the ledger for a similar answer first; a non-blocking question that matches needs `--force`.
`handoff --note [--data-file]`	Builder, scout, researcher	Hand the card on to the next stage (a scout hands on candidates).
`publish --file [--title] [--citations-file] [--path]`	Fact checker	Publish the verified research note with its citations.
`digest --file [--note]` or `digest --none`	Curator	Post the feed digest, or an empty one when nothing qualified.
`review approve --score 1-5 --note`	Reviewer	Approve the work with a quality score.
`review changes --reason --score --note [--criteria-missed]`	Reviewer	Send the card back with reasons and a score.
`test pass --evidence [--command]...`	Tester	Report the tests passed, with evidence and the commands run.
`test fail --title --note [--evidence] [--command]...`	Tester	Report a failure; it becomes a bug card with the given title.
`finalize --action pr|push_branch|merge [--note]`	BA	Run the card's completion action.
`answer --item --text [--option]`	Any (question shifts)	Answer a colleague's question.
`escalate --item --note`	Any (question shifts)	Pass a question on: peer, then BA, then the manager's desk.

Verb	Used by	What it does
`intake map --project KEY`, `intake unmapped --note`	BA (intake)	Map the ticket to a project, or ask the manager to pick one.
`source read --ref`	BA (intake)	Read a GitHub issue (Jira is read with the Jira tool).
`outbound draft --kind comment|resolution|decision --body`	BA	Draft an update for the card's ticket; the manager approves it on the desk.
`org catalogue`, `org role draft --file`, `org workflow draft --file`, `org validate <key>`, `org submit <key>`, `org department --name`	BA (design chat)	Read the catalogue, draft a role or workflow from a JSON file, validate it, submit it for the owner's approval, and create a department (section 8.6).
`devops "<request>"`	Any	Ask this workstation's DevOps engineer for help; the shift goes on.
`devops-approval --action --reason [--command]...`	DevOps engineer	Ask the manager to approve an action outside the granted access.
`devops-done --summary [--command]...`	DevOps engineer	Report the request is finished, with the commands that changed the machine.
`outcome <id> --note [--data-file]`	Any	Report an outcome by its id, the general form behind several verbs above.

10.8 toolchainPaths and Claude credentials

config.json layout and modes

The runner's configuration is `config.json` in the runner data directory. It is written with mode 0600 (readable only by the runner's user) and holds the runner token, so never share or commit it. Keys and defaults:

- **serverUrl**, **workstationId**, **runnerToken**: written by `aom-runner pair`; all three are required.
- **repoRoots** (default: empty): folders scanned for git repositories, added with `aom-runner repos add`.
- **runtimes**: settings per runtime.
  - `claude-code`: `bin` (default `claude`); `authMode` (default `default`) with `oauthToken` or `apiKey`; `sandbox` (`required` by default, `best_effort` or `off`); `allowUnsandboxed` (default false); `extraAllowedDomains` (default empty); `toolchainPaths` (default empty); `scrubSubprocessEnv` (default false); optional `intake` with `mcpConfig` and `allowedTools` for the BA's intake chat.
  - `mock`: `enabled` (default false), for tests and demos.
- **jira** (optional): `baseUrl`, `email` and `apiToken`, used to post approved updates to Jira. GitHub uses this machine's `gh` login.
- **maxConcurrentShifts** (default 4): how many shifts run at once on this workstation.
- **vaultSyncIntervalSec** (default 300): how often vaults are pulled and pushed.

toolchainPaths is a key of `runtimes["claude-code"]`. It lists extra directories that sandboxed shell commands may read, for example `~/.dotnet` or `~/.cargo` (`~` means the real home). The sandbox hides `/home`, so a toolchain installed under your home is invisible to agents until you list it; the Node install the runner itself runs on is opened automatically. The home directory itself and credential stores are refused, so never list a directory that holds credentials.

Claude Code credentials

Each agent runs with its own `CLAUDE_CONFIG_DIR` (`agents/<id>/claude`), and the runner passes it only the one credential its runtime is configured with. Which credential that is depends on `authMode`. An `oauthToken` or `apiKey` you put in `runtimes["claude-code"]` is stored in `config.json`, on this workstation only.

authMode: oauth, api_key, or default

- `oauth`: the runner gives each shift the `oauthToken` from the config as `CLAUDE_CODE_OAUTH_TOKEN`. Get a token with `claude setup-token`.
- `api_key`: the runner gives each shift the `apiKey` as `ANTHROPIC_API_KEY`.
- `default` (the default): no token is configured. The runner links your own Claude Code login, `~/.claude/.credentials.json`, into the agent's config directory (a symbolic link; a copy on Windows). Sign in with `claude` once on the workstation.

10.9 The unsandboxed warning everywhere

When a runtime on a workstation runs agents unsandboxed (`"sandbox": "off"`, or `best_effort` with `allowUnsandboxed: true` on a platform without a sandbox such as native Windows), a persistent amber warning appears. It cannot be dismissed and stays as long as the runner reports the state:

- **Workstations page:** the workstation card shows **Warning: no OS sandbox.** and says which runtime runs without a sandbox and what an agent could read, with a link "How to run the runner in WSL2". (A workstation whose runtime is unavailable shows "[runtime] is unavailable: [reason]" with a "Setup guide" link instead.)

- **HR board:** the agent card of each agent on that workstation shows "Warning: Runs without an OS sandbox on [workstation]: ...".
- **Hire form:** hiring onto an unsandboxed workstation shows the warning and a checkbox **I understand this agent runs without an OS sandbox on [workstation]**. If it is not ticked, the form stops with "Confirm that you accept hiring onto an unsandboxed workstation". A request sent to the server without the acknowledgement is refused with the error `unsandboxed_ack_required` (HTTP 409).
- **Re-home dialog:** when you move an agent to another workstation, the same warning shows if the destination is unsandboxed.

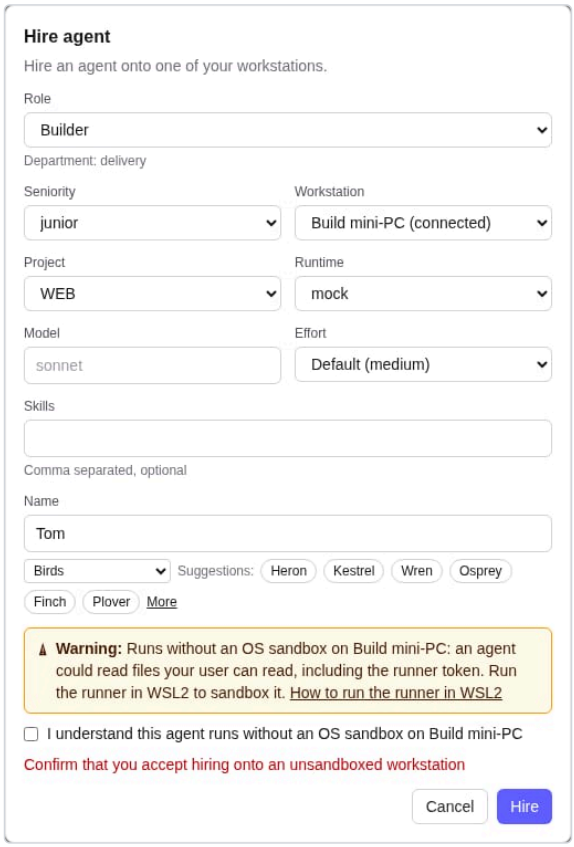


Figure 128 — Hire form with Build mini-PC chosen: the amber warning, the unticked box "I understand this agent runs without an OS sandbox on Build mini-PC", and the error "Confirm that you accept hiring onto an unsandboxed workstation" after pressing Hire (Pieter Demo)

Hiring onto a workstation whose runtime is `unavailable` is refused outright (`sandbox_unavailable`). The warnings are there to remind you of the trade-off: on an unsandboxed workstation an agent can read what your operating system user can read.

11 Running locally and deploying

11.1 Requirements and local development

The server runs in Docker, so any host with Docker works. For development and testing you need Node 22.23, Git 2.40 or later, and Docker. Workstations run the runner on Linux, macOS or Windows via WSL2 (real Claude Code was only run on Linux so far, see chapter 13). Agents run on your workstations, not on the server, so the workstation setup is separate (see `docs/RUNNER.md`).

Install dependencies

- **Node 22.23:** download from `https://nodejs.org` or use `nvm install 22.23 && nvm use`
- **Git 2.40+:** check with `git --version`; update if needed
- **Docker:** for local Postgres and any container-based testing
- **pnpm 12.8.1:** pinned in `package.json`, installed via corepack: `corepack enable`

Set up for local development

```
git clone https://github.com/your-org/agent-org-manager.git
cd agent-org-manager
corepack enable                 # Enable pnpm
pnpm install                    # Install dependencies
docker compose -f docker-compose.dev.yml up -d  # Start Postgres (127.0.0.1:5433, user/password/db aom)
pnpm db:migrate                 # Apply migrations to the aom database
pnpm dev                        # Start server (localhost:4600) and web (localhost:5173)
pnpm --filter @aom/server admin:create --email you@example.com --name "You" --password 'at-least-12-chars'  # Bootstrap first
admin
```

The server reads real environment variables only (it does not load a `.env` file); the built-in defaults match the dev compose file. The web app proxies `/api` to the server and works with WebSocket. Open `http://localhost:5173` and sign in with the admin account.

Run tests and checks

```
pnpm typecheck                  # TypeScript type check
pnpm lint                       # Biome linter
pnpm test                       # Unit and integration tests (vitest)
pnpm e2e                        # End-to-end tests with Playwright (needs Postgres on 5433, Mailpit on 1025, Chromium)
pnpm --filter @aom/server projections:rebuild   # Rebuild all projections from events (stop the server first)
```

11.2 Deploying to a VPS with Docker Compose

Production runs on a VPS (or your own server) with Docker Compose, Postgres, and Caddy for HTTPS. Agents run on separate workstations and connect outbound to the server over WebSocket (no inbound agent ports needed).

Pre-deployment checklist

- Domain name with DNS pointing to your VPS IP
- Postgres password chosen
- SMTP relay configured for invitation emails
- Ports 80 and 443 open (for Caddy TLS)
- Docker and docker-compose installed on the VPS
- Latest code pulled: `git pull origin main`

Deploy step by step

1. **Create .env on the VPS:**

```
cp .env.prod.example .env
# Edit .env with your values (at minimum):
# POSTGRES_PASSWORD=
# AOM_DOMAIN=aom.example.com
# AOM_PUBLIC_ORIGIN=https://aom.example.com
# AOM_SMTP_URL=smtp://user:pass@relay.example.com:587
```

2. **Build and start Docker containers:**

```
docker compose -f docker-compose.prod.yml up -d --build
# Migrations run automatically on boot (AOM_MIGRATE_ON_BOOT=true)
```

3. **Create the first admin account:**

```
docker compose -f docker-compose.prod.yml exec app node dist/admin-create.mjs \
  --email you@example.com \
  --name "You" \
  --password 'your-strong-password'
```

4. **Test the deployment:**

```
curl https://aom.example.com/api/health
# Returns: {"ok":true,"db":true}
```

Deployment complete

Open `https://aom.example.com` in a browser and sign in with the admin account. For local testing with an internal CA certificate, extract the root certificate and add it to your trusted CAs.

11.3 Configuration reference

All server configuration is set via environment variables in `.env` (read on server startup). Defaults are suitable for development; production requires setting at least the domain and database password.

Variable	Default	Purpose
`AOM_PUBLIC_ORIGIN`	`http://localhost:5173`	Full URL users open; determines CSRF origin, cookie Secure flag, invitation links
`AOM_DOMAIN`	`localhost`	Read by Caddy (compose), not by the server: a public domain gets a Let's Encrypt certificate; `localhost` or an IP uses Caddy's internal CA
`AOM_HOST`	`127.0.0.1`	Server bind address (use `0.0.0.0` in Docker)
`AOM_PORT`	`4600`	Server port (expose through Caddy, not directly)
`DATABASE_URL`	`postgres://aom:aom@127.0.0.1:5433/aom`	Postgres connection string; in Docker use `postgres://aom:$POSTGRES_PASSWORD@postgres:5432/aom`
`POSTGRES_PASSWORD`	—	Database password (set this; used by postgres container and DATABASE_URL)
`AOM_SMTP_URL`	`smtp://127.0.0.1:1025`	SMTP relay URL for invitation emails
`AOM_MAIL_FROM`	`Agent Org Manager <noreply@aom.local>`	From address in emails
`AOM_MIGRATE_ON_BOOT`	`true`	Apply migrations on server start
`AOM_SERVE_WEB`	`false`	Serve built web app from server (enabled in `docker-compose.prod.yml`)
`AOM_HTTP_PORT`, `AOM_HTTPS_PORT`	`80`, `443`	Host ports Caddy publishes (compose)
`LOG_LEVEL`	`info`	Logging level: `debug`, `info`, `warn`, `error`
`AOM_RATE_LIMIT`	`true`	Enable rate limiting on the sensitive routes (login, invitations, pairing); there is no global limit

Never in production

Do not set `AOM_TEST_CLOCK`, `AOM_ALLOW_LOCAL_REMOTES` or use `.env.example` in production. Always use `.env.prod.example` as your template and review all values before deploying.

11.4 Backups, upgrades and operations

Backup strategy

Postgres stores the event log (immutable, append-only) and projections. Regular backups are critical. Use your VPS provider's snapshot tools or Docker for manual backups.

```
# Manual backup (example)
docker compose -f docker-compose.prod.yml exec postgres pg_dump -U aom aom > backup-$(date +%s).sql

# Restore from backup
docker compose -f docker-compose.prod.yml exec -T postgres psql -U aom aom < backup.sql
```

Set up automated daily snapshots with your VPS provider. Point-in-time recovery is out of scope for v1.

Upgrades with minimal downtime

1. Ensure all runners are idle (no active shifts on the board)

2. Pull the latest code: `git pull origin main`
3. Rebuild and restart: `docker compose -f docker-compose.prod.yml up -d --build`
4. Migrations run on boot (typically <1 second)
5. Test: `curl https://aom.example.com/api/health`

Total downtime is typically under 5 seconds (container rebuild + migration). Caddy proxies requests during rebuild.

Health check and monitoring

```
curl https://aom.example.com/api/health
# Returns: {"ok":true,"db":true}
```

Monitor container status with `docker compose -f docker-compose.prod.yml ps` and logs with `docker compose -f docker-compose.prod.yml logs -f app`. The app container has a Docker health check on this endpoint (Caddy starts only when the app is healthy); Docker does not restart a container just for being unhealthy, but `restart: unless-stopped` restarts it if it exits.

Troubleshooting

Issue	Check / Fix
Caddy certificate fails (Let's Encrypt)	Verify DNS points to the VPS IP; check Caddy logs: `docker compose -f docker-compose.prod.yml logs caddy`; for internal IP/hostname, set `AOM_DOMAIN=localhost`
Migrations fail on boot	Check database connectivity: `docker compose -f docker-compose.prod.yml logs postgres`; verify `DATABASE_URL` and `POSTGRES_PASSWORD` match; check disk space
High error rates or slow responses	Check app logs: `docker compose -f docker-compose.prod.yml logs app | tail -100`; check database size: `docker compose -f docker-compose.prod.yml exec postgres du -sh /var/lib/postgresql/data`
Runner disconnects frequently	Check server logs and runner connectivity; ensure firewall allows WebSocket on port 443; check runner config has the correct server URL

12 Quality and verification

12.1 Test suites and counts

The numbers below come from a fresh run on 4 October 2026 on the `main` branch. Unit and integration tests need Postgres on port 5433; the end-to-end suite needs Postgres, Mailpit and Chromium.

Suite	Command	Result	What it covers
Shared contract	`pnpm --filter @aom/shared test`	**277 / 277**	Zod schemas, workflows and their validation, roles, authorization predicates, HR rules, prompt building
Server	`pnpm --filter @aom/server test`	**393 / 393**	Event store, projections, dispatcher, commands, routes, runner gateway against a real database
Web	`pnpm --filter @aom/web test`	**104 / 104**	Pages and components with mocked API (MSW)
Runner	`pnpm --filter @aom/runner test`	**288 passed, 1 skipped**	Shift supervisor, sandbox policy, git, adapters, vault sync, external posting
End to end	`pnpm e2e`	**102 / 102**	Real browser against a built server and a mock-runtime runner: every milestone, one spec each
Types and style	`pnpm typecheck`, `pnpm lint`	**0 errors**	TypeScript strict, Biome. Lint reports warnings (mostly `any` in tests); see chapter 13

Verified

Every figure in chapters 3 to 10 was taken from a running app seeded through the real API by the end-to-end project `guide` (section 12.4). The default `pnpm e2e` run does not include it.

12.2 The independent security review and its fixes

A separate review looked at the whole system as an attacker would, with agents treated as untrusted. It found high, medium and low issues. Every one was fixed with a test, in these commits:

Finding	Problem	Fix
H1	A manager of a shared project could add standing orders for the owner's organization	Standing orders need project or organization ownership
H2, M3, M4	Agent sessions were not confined: environment, per-role permissions, writable files, read-only roles	Sandboxed sessions with a minimal environment, per-role Claude permissions, a generic writable-glob guard, and read-only roles are rolled back
H3	Deactivating a user left their runners, memberships and agents working	Deactivation unlinks the runners, removes memberships and parks the agents
M1, M5, M7	Owner-controlled refs, URLs and paths could reach git and the file system	Validation of refs, URLs and paths; runner paths are confined; URLs with credentials are refused; card refs and worktree paths are unique
M2	An agent's permissions could be widened beyond its role	Permissions may only narrow the role definition
M6, M8, low	Hardening items: response headers, chat rights, runner command ownership, outbound leak check, rate limits, log redaction, links, archived-agent hire	CSP and HSTS, chat re-checks on every message, runner command ownership, the outbound leak check, rate limits, log redaction, lease-preserving projection rebuild, http(s)-only links, archived-agent hire check

The threat model and what the sandbox does and does not protect against are written down in `docs/RUNNER.md`, section "Threat model and residual risk".

12.3 The real Claude Code smoke run

The test suites use the mock runtime. A real Claude Code shift was run once on Linux, in the sandbox, with `AOM_CLAUDE_SMOKE=1` (this spends tokens, see `docs/RUNNER.md`). It found four problems that no mock could show:

Failure found	Cause	Fix
No writable temporary directory	The allowed read of the agent directory also covered the writable home and scratch; bubblewrap re-opens an allowed read read-only over the binds beneath it, so every Bash call failed on its first write	Allowed reads no longer include directories that hold writable paths; every agent gets a private tmp (`TMPDIR`, `TMP`, `TEMP`, `CLAUDE_CODE_TMPDIR`); agent sessions no longer inherit the runner's whole environment
`node` hidden by the read block	Sandboxed commands lose read access to `/home` under `blockReadsOutsideWorkingDirectories`, so Node and the `aom` shim were unreachable	The runner's Node install and the configured `toolchainPaths` are re-opened for reading
The `aom` exclusion pattern	The exclusion used a bare `aom`, which only matches the command without arguments	Exclude `aom *` and the shim's absolute path; `aom` runs outside the sandbox because Linux blocks Unix sockets inside it

Failure found	Cause	Fix
Session tools such as `ListAgents`	Session-level tools (`ListAgents`, `SendMessage` and others) ran under the `dontAsk` mode	Session tools are denied by name

Result

After these fixes a real Claude Code builder shift on a small repository completes inside the sandbox: it writes code, commits, and hands off through `aom`. The commits are `366f825` and `dca3389`.

Not covered

The smoke run was done on Linux (the same sandbox WSL2 uses). A real Claude on macOS (Seatbelt) and on native Windows has not been run; see chapter 13.

12.4 How the screenshots of this guide were made

The Playwright project `guide` in `e2e/` builds a fake demo world through the real API and the mock runtime: an organization owner (Pieter Demo), a second manager (Maya Demo) on a shared project, three projects (WEB code, SHOOT research, NEWS feed), hired agents of every role, cards in every column, desk items of every kind, ledger entries, enough shift history for HR statistics and recommendations, one workstation that reports no OS sandbox, and a custom workflow. It then drives the real UI and takes every picture at 1280 pixels wide, and the phone pictures at 390 pixels. The project is excluded from `pnpm e2e`; `docs/guide/README.md` has the exact commands.

13 Decisions taken, known limitations

13.1 Decisions that shape the product

This table lists the most important architectural decisions from `docs/ARCHITECTURE.md` section 24 ("Resolved ambiguities and defaults"), which lists 22 decisions; all are shown here with the numbers used there.

#	Decision
1	One organization per user; agent names are unique per organization forever (including retired agents).
2	Card owner = the user who created or imported the card.
3	Rework and Blocked are not stages. Rework is the `build` stage with `rework_round > 0` (or a bug card); Blocked is a status overlay; board columns are derived.
4	Test failure before finalize creates a bug child card on the `code-bugfix` workflow that shares the parent's worktree and branch and goes to the original builder first; the parent waits in Test until the child passes review, then re-enters the Test queue.
5	Post-finalize failures and regression bugs create standalone bug cards in Spec for the BA (M4) and never block merges.
6	Dependencies gate the first build claim until every dependency is done (finalized); dependent cards still branch from the working branch. Branching from a dependency's branch is out of scope for v1.
7	Card pinning to a workstation: after the first claim creates the worktree, all later stages run on that workstation. Cards do not move between workstations; the manager can re-home a card, which restarts its current stage from the base branch.
8	Offline timeout is implemented as a 1-hour lease refreshed by 15 s heartbeats.
9	Large-tier review = two approvals from two distinct senior reviewers, sequentially.
10	Instructions are written to the agent's own `CLAUDE_CONFIG_DIR/CLAUDE.md`, not the worktree's `CLAUDE.md` (which belongs to the repo and must not be modified or committed).
11	Agent reporting uses the `aom` CLI over a localhost IPC with a per-shift token; fenced `aom-report` JSON in the final message is a fallback.
12	Provider credentials stay in the runner's local config (`CLAUDE_CODE_OAUTH_TOKEN` / `ANTHROPIC_API_KEY`) or are linked from the user's default Claude config; never sent to the site.
13	Vault sync, rebases, worktree management and completion actions are deterministic runner code; LLM DevOps shifts handle only ad-hoc requests.
14	Decision files and taste rules reach the vault through `vault.write` runner commands, since the site holds no vault content.
15	Escalation chain (M7 replaced the M2 default of going straight to the desk): a question goes to the card's previous holder or a senior agent of the asker's role (an online agent only), then to the BA, then to the manager's desk. A question nobody takes within 10 minutes moves on by itself; without a peer or BA it goes straight to the desk.
16	DevOps engineer is auto-hired per (user, workstation) at pairing (it cannot be fired, only reset: `docs/SPEC.md`, Operating model).
17	Project sharing requires the invitee to accept (pending until then).
18	Scheduled overflow is limited to one extra shift per organization at a time.
19	Research and feed roles (researcher, checker, scout, curator) ship as built-in roles.
20	Operational, non-event state: auth sessions, heartbeats and lease refresh, connection flags, shift progress tails, runner inbox and command queue.
21	Risk tier is computed by the runner at build handoff: ≥ 200 changed lines, any architectural glob, or refactor ⇒ large.
22	Finalize before the BA exists (M4–M6): the manager triggers it from the card. From M7 the BA claims the Finalize stage and reports `aom finalize --action ...`; the runner executes the completion action (pr, push_branch or merge; large merges wait for desk approval), and the manager's Finalize button remains as an override.

13.2 Known limitations

Limitation	Effect	Workaround
Single server instance for v1	No horizontal scaling; one VPS instance only.	Size the VPS for it; v1 has no clustering. Event store appends use a global lock (see `docs/DEPLOY.md`).
Point-in-time recovery not built	Database backups are snapshots, not transaction logs.	Set up automated VPS snapshots (daily). Restore from the most recent backup if needed.
No agent-to-agent blocking (intentional)	Agents never wait on another; work always flows forward or is explicitly blocked by a manager question.	By design: reduces queuing and communicates blockers to the manager explicitly.
Agents run only on user workstations	Agents cannot run on the server or on shared CI infrastructure (v1 scoping).	Pair a dedicated low-power machine as a workstation if you need always-on agents.
No cost tracking per agent or project	No token accounting or cost forecasting (v1 scope).	Use the HR board and metrics for token counts per agent; check cost with your provider.
Vault URLs may not carry credentials	A URL with a password or token is refused when you register or edit a vault	Use SSH keys or a git credential helper on the workstation
No native Windows sandbox	Windows requires WSL2; unsandboxed native Windows is an explicit opt-in only.	Use WSL2 (recommended); see docs/RUNNER.md for setup.

Limitation	Effect	Workaround
No agent pool or autoscaling	You must hire agents manually and manage capacity.	The HR board recommends hiring, and a desk alert fires when testers are busy while E2E work waits.
A resumed shift re-runs the agent from the start	After a runner restart within the 1-hour lease the shift keeps its id and worktree, but the agent process is restarted, not re-attached (Claude starts a fresh run on the card branch).	Expect repeated work after a runner crash; keep the runner under systemd/launchd so restarts are quick.
Jira resolution posts a comment only	A resolution update is posted as a comment; the ticket is not transitioned (transitions are workflow specific).	Move the ticket yourself after approving the update.
Definition files are not published retroactively	A definition file is written to the vault when a role or workflow version is approved (without a vault only the site copy exists); a vault added later does not receive earlier approved definitions.	Copy the YAML files into the vault by hand, or approve new versions after the vault exists.
A manually finalized card gets no BA-drafted ticket update	The BA drafts the outbound update in its Finalize shift; if you finalize the card yourself that shift never runs.	Write the outbound update yourself from the card (the manager can write a draft).
No model advice for other models	The HR board recommends model changes only along the ladder haiku, sonnet, opus; models outside it get no advice.	Change such an agent's model by hand.
Real Claude Code only run on Linux	The real adapter was exercised on Linux only. WSL2 uses the same sandbox (bubblewrap and socat); macOS (Seatbelt) and native Windows were not tested with real Claude Code.	On macOS, run `aom-runner doctor` and a small card first. On Windows use WSL2.
MSW dev mocks are partial	The web app can run against MSW mocks (`VITE_MSW=1`), but they cover only part of the API.	Use the real server for anything beyond basic screens.
Biome reports lint warnings	`pnpm lint` finishes with warnings (about 690) and no errors.	None needed; the check passes.
At most 4 shifts per workstation at once	The server tells each runner a limit of 4 concurrent shifts, whatever `maxConcurrentShifts` says in the runner config.	Pair another workstation for more parallel work.
Demo and E2E use a mock runtime	The test suites and the demo do not run real agents; the mock adapter plays a script per card.	Run real Claude Code on a workstation for real behavior (`AOM_CLAUDE_SMOKE=1` runs a smoke test and spends tokens).

13.3 Things that look odd but are intended

Why card refs are not globally unique

Card refs are the project key plus a number (e.g., WEB-1). Project keys are unique within one organization, so two users can each have a project WEB with a card WEB-1. A ref that matches cards in more than one of your projects is answered with `409 ambiguous_ref`; the web app links by card ID.

Why the runner connects outbound, not inbound

The runner on your workstation connects to the server over outbound WebSocket (WSS port 443) and stays connected. This eliminates firewall complexity (no inbound ports), keeps credentials and code on your machine, and works everywhere (VPN, office, home, mobile hotspot). The trade-off is that only one runner per workstation can be active; a second connection with the same token closes the first (the runner exits with code 3).

Why DevOps cannot be fired

The DevOps engineer per workstation can only be reset (sandbox wiped), not fired. Every user gets one DevOps engineer on each workstation, which other agents and the manager ask directly; every request and action is logged on a thread. A reset wipes its sandbox the way a firing does.

Why card dependencies do not affect branching

When a card depends on another, the dependent card must wait for the dependency to finalize before it claims the Build stage. However, it still branches from the working branch (main), not from the dependency's branch. This keeps branches simple and allows parallel builds once dependencies are done. Rebase conflicts are detected by the runner before the completion action.

Why decisions can only be answered by the owner (by default)

Card decision policy defaults to owner_only so the card creator stays accountable for decisions on their work. On a shared project, the policy can be changed to any_manager to distribute decision authority. This is explicit: changing the policy is a deliberate choice, not an accident.

14 Your to-do list before and after first real use

14.1 Before first real use

Agent Org Manager must be deployed and paired before you can run real shifts. Follow this checklist step by step.

- [ ] Deploy to a VPS with Docker Compose (see `docs/DEPLOY.md`): `docker compose -f docker-compose.prod.yml up -d --build`
- [ ] Set `AOM_PUBLIC_ORIGIN` to your HTTPS domain: `https://aom.example.com`
- [ ] Create the first admin account: `docker compose -f docker-compose.prod.yml exec app node dist/admin-create.mjs --email you@example.com --name "You" --password '...' (at least 12 characters)`
- [ ] Sign in and check the deployment: `curl https://aom.example.com/api/health` should return `{"ok":true,"db":true}`
- [ ] Pair a workstation (Linux, macOS or WSL2): `aom-runner pair --server https://aom.example.com --code XXXX-XXXX --name "Desktop"` (create the code first on the Workstations page, Pair workstation; it is valid for 10 minutes)
- [ ] Run `aom-runner start`, and install it as a systemd user service (Linux/WSL2) or launchd agent (macOS) so it comes back after a reboot (see `docs/RUNNER.md`)
- [ ] Set up Claude Code credentials in the runner `config.json`: an OAuth token from `claude setup-token` (recommended, `authMode: "oauth"`), an API key (`authMode: "api_key"`), or the default local Claude login
- [ ] On Windows: WSL2 is required; install bubblewrap and socat for sandboxing: `sudo apt install -y bubblewrap socat`
- [ ] On Linux/WSL2: install sandbox dependencies: `sudo apt install -y bubblewrap socat`
- [ ] Verify the sandbox is enforced: `aom-runner doctor` should report the sandbox as `enforced`
- [ ] Add repository folders: `aom-runner repos add ~/src ~/work` (scanned recursively, depth 3)
- [ ] Verify repos are found: `aom-runner repos list`
- [ ] Create your first project in the web UI: Projects → New project → enter key, name and workflow (`code` for a code project) → Create project
- [ ] Add a repository to the project: Projects → Settings → Add repository (name, remote URL, optional branch)
- [ ] Add toolchain directories if sandboxed agents need tools outside the standard ones (see `toolchainPaths` in `docs/RUNNER.md`)
- [ ] Optional: add Jira credentials to the runner `config.json` if you want to post approved updates to Jira: `{"jira":{"baseUrl":"https://jira.example.com","email":"you@example.com","apiToken":"..."}}` (intake from Jira additionally needs `runtimes.claude-code.intake.mcpConfig`)
- [ ] Hire at least one builder on your workstation: Agents page (the HR board) → Hire agent → choose role Builder, name and model
- [ ] Create a small card on your project: New card → fill title, context, requirements, acceptance criteria → Specify
- [ ] Run one real shift: board → claim a card → watch the shift log in Claude Code as the builder works
- [ ] Observe the handoff: when the shift ends, check the shift log and handoff note on the card detail
- [ ] Optional: list extra read-only toolchain directories in the runner config: `{"runtimes":{"claude-code":{"toolchainPaths":["~/.cargo"]}}}`
- [ ] Optional: pair a second workstation or invite another manager to a shared project (see Projects → Settings → Members)
- [ ] Test that the deployment is stable: run a few more shifts, check logs for errors: `docker compose -f docker-compose.prod.yml logs app -f`
- [ ] You are ready for real work. See chapter 1 for the next steps (hiring a team, feeding work through intake, monitoring the board).

14.2 After first real use

Once you have run shifts and the system is working, set up operational infrastructure and team processes.

- [ ] Hire your core team: one builder senior and junior, one reviewer, one tester, one BA (senior), and one technical writer
- [ ] Create a project for your main work (if you have not already) and a personal project for side work and exploration
- [ ] Invite additional managers to shared projects: Projects → Settings → Members → enter email → they accept the invitation
- [ ] Feed work into the system: create cards from your backlog, or map Jira tickets via intake (BA chat)
- [ ] Set up a vault (git repository) for team knowledge: Settings → Vaults (`/settings/vaults`) → register a work and a personal vault by Git URL
- [ ] Vault sync runs every 5 minutes by default (`vaultSyncIntervalSec`) on each workstation and commits/pushes changes automatically (no manual git needed)
- [ ] Optional: add research or feed projects if your team has side projects, newsletters or knowledge curation work
- [ ] Check the HR board weekly: HR board shows agent performance, bottlenecks and recommendations
- [ ] Establish a decision desk process: desk items route to you when agents ask questions or when approvals are needed
- [ ] Pair additional workstations as your team grows: Workstations → Pair workstation → each user/machine pairs once

- [ ] Review standing orders to encode recurring decisions and constraints: you create them, agents see them on the ledger
- [ ] Set up backups: configure automated daily snapshots on your VPS provider: `docker compose -f docker-compose.prod.yml exec postgres pg_dump -U aom aom | gzip > backup-$(date +%s).sql.gz`
- [ ] Test backup restore once to verify it works (do this on a test DB, not production)
- [ ] Set up a runbook for common incidents (agent stuck, runner disconnected, database backup needed) in your vault
- [ ] Plan upgrades: v1 deployments are upgraded via `git pull && docker compose -f docker-compose.prod.yml up -d --build` with minimal downtime (<5 seconds)
- [ ] Document your project workflows and team processes in your work vault so new team members can find them

15 Appendix: where everything lives, commit history

15.1 Where everything lives

Agent Org Manager is a monorepo with workspaces for shared contracts, the server, the web app, the runner, and tests.

Component	Location	Purpose
Documentation and configuration		
`docs/`	`docs/SPEC.md`, `USER-GUIDE.md`, `RUNNER.md`, `DEPLOY.md`, `ARCHITECTURE.md`	Product spec, user guides, technical design
`README.md`	Root	Project overview, features, links to docs
`Dockerfile`	Root	Multi-stage build: Node, web + server bundle
`docker-compose.*.yml`	`docker-compose.dev.yml`, `.prod.yml`	Local (Postgres only) and production (app, postgres, caddy)
`.env.example`, `.env.prod.example`	Root	Configuration template (dev and production)
`deploy/`	`Caddyfile`	Reverse proxy, TLS, routing config
Shared contract (server and web)		
`packages/shared/`	`src/`	Event schemas, API contracts, workflows, roles, authorization predicates
Events	`src/events/`	Domain events: user, workstation, project, card, agent, shift, desk, ledger, testing, schedule, feed, research, intake, outbound, devops, vault, catalogue
Workflows	`src/workflows/builtin/{code,code-bugfix,research,feed}.ts`	Built-in workflow definitions with stages and outcomes
Roles	`src/roles/{schema,builtin}.ts`	10 built-in roles (builder, reviewer, tester, writer, BA, devops, researcher, checker, scout, curator) with instructions and permissions
API schemas	`src/api/*.ts`	Request/response schemas for every REST route
Protocol	`src/protocol/{runner,browser,ipc}.ts`	WebSocket and IPC message schemas
Server (Fastify + event store)		
Migrations	`apps/server/migrations/NNNN_*.sql`	Postgres DDL, numbered by milestone (M1: 0001–0009, M2: 02xx, M3: 03xx, … M10: 1000, then 1100)
Event store	`apps/server/src/eventstore/{types,pg-event-store,notifier}.ts`	Append-only Postgres event store with projections, subscriptions, pub/sub
Projections	`apps/server/src/projections/*.ts`	Denormalized views (users, workstations, projects, cards, agents, shifts, threads, desk, ledger, …)
Commands	`apps/server/src/commands/*.ts`	Domain logic: auth, invitations, workstations, projects, cards, agents, shifts, desk
Routes	`apps/server/src/routes/*.ts`	HTTP endpoints per domain (auth, admin, invitations, workstations, projects, cards, board, agents, …)
Gateways	`apps/server/src/{runner-gateway,browser-gateway}`	WebSocket handlers for runners and browsers, event fan-out, authorization
Dispatcher	`apps/server/src/dispatcher/{dispatcher,claim,reaper}.ts`	Smart queue: orders cards by priority (scheduled > work > personal), assigns to available agents
Scheduler	`apps/server/src/scheduler/scheduler.ts`	Cron-based task scheduler: fires scheduled cards on time
Auth	`apps/server/src/auth/{password,sessions,guards}.ts`	Login, session management, authorization checks
Web app (React + Vite)		
Pages	`apps/web/src/pages/{Login,Board,CardDetail,CardNew,Projects,ProjectSettings,Workstations,Agents,Admin,Desk,Ledger,Chat,Metrics,Schedules,Vaults,FeedProject,Intake,AcceptInvite,OrgDesigner}.tsx` (the HR board is the Agents page at `/hr`)	Main UI pages
API client	`apps/web/src/api/client.ts`, `src/api/queries/*.ts`	React Query hooks for REST endpoints
WebSocket	`apps/web/src/ws/{socket,invalidate}.ts`	Browser connection, event replay, reactive query invalidation
Components	`apps/web/src/components/`	Reusable UI components (board, cards, forms, dialogs, tables)
Runner (Node CLI on workstations)		

Component	Location	Purpose
CLI	`apps/runner/src/cli.ts`	Commands: pair, start, status, doctor, rotate-token, repos add/list
Connection	`apps/runner/src/connection/{client,outbox}.ts`	Outbound WebSocket to server, reconnect logic, message queue
Shifts	`apps/runner/src/shifts/{supervisor,brief-to-files}.ts`	Agent shift lifecycle: start, monitor, capture output, handle completion
Adapters	`apps/runner/src/adapters/{claude-code,mock}.ts`	Runtime adapters: spawn agent process (Claude Code or mock), parse output, report
Git	`apps/runner/src/git/{git,worktrees,repo-scan}.ts`	Git worktree creation, branch management, rebase conflict detection
Vaults	`apps/runner/src/vaults/sync.ts`	Automated vault pull/commit/push every 5 minutes
DevOps	`apps/runner/src/devops/requests.ts`	Environment change requests, approval routing
aom CLI	`apps/runner/src/aom/aom-cli.ts`	In-session commands: handoff, review, test, question, answer, outbound, publish, ...
Tests and end-to-end		
Unit tests	`apps/server/test/`, `apps/runner/test/`, `packages/shared/test/`	vitest: event store, migrations, auth, dispatcher, protocols, workflows, ...
E2E tests	`e2e/tests/*.spec.ts`	Playwright: full stack, real UI, API, runner, Postgres, Mailpit
E2E support	`e2e/support/*.ts`	Helpers: db reset, vault setup, runner simulation, mail capture, APIs
pnpm scripts (run from root)		
`pnpm dev`	—	Start server and web locally (server :4600, web :5173)
`pnpm test`	—	Run all unit and integration tests
`pnpm e2e`	—	Run end-to-end Playwright tests
`pnpm typecheck`	—	TypeScript type check (strict)
`pnpm lint`	—	Biome linter and formatter
`pnpm db:migrate`	—	Apply SQL migrations to the aom DB
`pnpm --filter @aom/server projections:rebuild`	—	Replay all events to rebuild the projections (stop the server first)
`pnpm --filter @aom/server admin:create`	—	Bootstrap first admin account
`pnpm build`, `pnpm e2e:prod`, `pnpm runner:dev`	—	Build all bundles; scripted production compose check; run the runner against `.dev/runner`
`pnpm package:runner`	—	Build and package the runner tarball

15.2 Commit history by milestone

All 74 commits on main, grouped by milestone (M1 through M10 are all implemented and merged), newest groups last. Within a group the commits are listed oldest first. Full history: `git log --format='%h %ad %s' --date=short`.

Hash	Date	Subject
M1: core loop (7 commits)		
`a3c1c3e`	2026-10-03	feat: scaffold monorepo, shared contract, architecture and plan (M1.0)
`cd05ce5`	2026-10-03	feat(web): login, admin, workstations, projects, board, cards, agents UI (M1.4)
`2eb896a`	2026-10-03	feat(runner): workstation runner with pairing, worktrees, sandbox, mock and Claude Code adapters (M1.3)
`962cd5b`	2026-10-03	feat(server): event store, auth, invitations, pairing, projects, cards, dispatcher, runner and browser gateways (M1.1, M1.2)
`5962779`	2026-10-03	feat: end-to-end core loop, integration fixes, Docker Compose deployment (M1.5)
`d24f0da`	2026-10-03	test: allow overriding test/e2e database and port via env
`44077b9`	2026-10-03	style: format
M2: decision desk (6 commits)		
`ac12be6`	2026-10-03	feat(shared): M2 contract for decision desk, ledger, chat and token rotation
`6e3fb28`	2026-10-03	feat(server): decision desk, questions, ledger, chat relay, password change and runner token rotation (M2.1)
`63c227a`	2026-10-03	feat(runner): aom question and ledger search, chat runner (mock echo + Claude Code), token rotation (M2.2)
`0b84d17`	2026-10-03	feat(web): decision desk, ledger, chat drawer and blocker link (M2.3)
`85faf00`	2026-10-03	test(e2e): decision desk, questions, ledger and chat specs; web chat keeps undelivered text; dispatch on desk resolve (M2.5)

Hash	Date	Subject
`7c158e7`	2026-10-03	feat(server): chats tied to a project use the project audience so members see them
M3: vaults, DevOps, sharing (11 commits)		
`fb7a7d6`	2026-10-03	feat(shared): M3 contract for vaults, DevOps and shared projects
`a7b7478`	2026-10-03	feat(server): vaults, DevOps engineer queue, shared projects and member removal (M3.1)
`cbe9d72`	2026-10-03	test(server): M3 sharing, member removal, HR authz, DevOps queue, vault commands
`c47d1ca`	2026-10-03	feat(runner): vault sync and writes, DevOps shifts, aom devops verbs, per-card vault context (M3.2)
`b776b6f`	2026-10-03	feat(web): vaults, DevOps access and requests, project members, invitations, re-home and decision policy (M3.3)
`aa38d8a`	2026-10-03	chore: ignore agent worktrees
`cc21b14`	2026-10-03	test(e2e): M3 devops, vaults and sharing specs; re-match runner repos on membership changes
`1e375bf`	2026-10-03	fix: deterministic overflow ordering, lazy zod import in the mock agent, retry deadlocked test cleanup
`b3436e4`	2026-10-03	Merge branch 'worktree-agent-a957c9a79344ab31f' (M3: vaults, DevOps engineer, shared projects)
`e7cdc72`	2026-10-03	feat(runner): refuse real Claude sessions in mock/test mode; make core-loop AC8 wait for a running shift
`a751d9d`	2026-10-03	feat: wire M3 into the M2 desk (devops approvals, vault conflicts, decision files, standing orders)
M4: testing and bug loop (5 commits)		
`1a12196`	2026-10-03	feat(shared): M4 contract for the tester and bug loop, finalize and regression
`f47fae7`	2026-10-03	feat(server): tester and bug loop, finalize, full suite, regression, bottleneck (M4.1)
`449f0b3`	2026-10-03	feat(web): test column waiting state, evidence panel, finalize, regression, hire-tester alert (M4.3)
`b4b4e60`	2026-10-03	feat(runner): rebase module, completion actions, tester guard rules, test environment check (M4.2)
`0510c13`	2026-10-03	test(e2e): M4 tester and bug loop specs; bottleneck needs a tester pool; suite cards inherit the mock suite script
M5 and M6: scheduling, feeds, research (5 commits)		
`064a352`	2026-10-03	feat(m5): scheduler, feeds, digests and ratings across shared, server and runner
`84300ec`	2026-10-03	feat(m5,m6): feeds web and e2e, research projects (researcher, checker, publish to vault)
`7647695`	2026-10-04	Merge M5 scheduler+feeds and M6 research into main
`9d87829`	2026-10-04	feat: regression schedules call M4 regression run; discard out-of-scope leftovers; resume parent on bug child cancel; MSW stubs
`a830fdd`	2026-10-04	test: regression schedule E2E step; consume stale shift starts in the server test
M7: BA intake and escalation (11 commits)		
`1e35c6b`	2026-10-04	feat(m7): shared contract for BA intake, escalation chain and BA finalize
`9f14283`	2026-10-04	feat(m7): server intake, escalation chain and BA finalize
`a46bfc9`	2026-10-04	feat(m7): runner question shifts, BA verbs, ticket adapters and a scripted mock BA
`0c6641f`	2026-10-04	feat(m7): web intake page, mapping question and escalation trail on the desk
`aba4c99`	2026-10-04	test(m7): intake, escalation and BA finalize E2E; continue the BA chat once its turn is over
`a46f518`	2026-10-04	fix(web): escalation trail keys without array index
`dd9f105`	2026-10-04	feat(m7): the manager's own Jira skill from the work vault; docs defaults updated
`b1a380d`	2026-10-04	test(m7): roomier waits in the chain tests
`7bee5f4`	2026-10-04	feat(m7,m9): agent answers enter the ledger, duplicate questions merge, stats account for the chain
`cabb29a`	2026-10-04	fix(m7): duplicate questions need near-identical wording and the same numbers or identifiers
`1511f19`	2026-10-04	fix(m7): single digits tell duplicate-question candidates apart
M8: outbox (5 commits)		
`7d1eb8f`	2026-10-04	feat(m8): shared contract for the outbox: outbound events, leak check, outbound.post command, draft API
`d3f0500`	2026-10-04	feat(m8): server outbox: drafts, desk approval with edit, owner-only posting, failures on the desk
`a236e90`	2026-10-04	feat(m8): runner posting for GitHub (gh) and Jira (REST), idempotent executor, aom outbound draft
`6244b51`	2026-10-04	feat(m8): web outbound approval with diff-edit and per-card history; E2E outbox spec; docs
`71cb70e`	2026-10-04	Merge M8: outbox and adapters (resolve conflicts keeping M8 and M9 side by side)
M9: HR board and metrics (4 commits)		
`c9d721e`	2026-10-04	feat(m9): HR track record, stats, recommendations, metrics, role resets, schedule backoff (contract, server, runner)
`7f88f54`	2026-10-04	feat(m9): HR board, work quality, recommendations, metrics dashboard (web)
`70762a6`	2026-10-04	test(m9): HR board E2E spec; metrics range ends at the server clock; first-run cap removed from specs
`0bbc62a`	2026-10-04	merge: M9 HR board, track records, metrics, recommendations
M10: organization designer (5 commits)		
`c0d62cf`	2026-10-04	feat(m10): organization designer contract, server drafts/validation/approval, departments, BA design verbs
`d83b51e`	2026-10-04	feat(m10): Organization page with role catalogue, workflow editor and graph preview, departments, design chat; department filters on HR board and boards
`4e7fd90`	2026-10-04	test(m10): org designer e2e, outbox leak check for retired agents, docs; bundle yaml in ESM builds

Hash	Date	Subject
`2e99b6d`	2026-10-04	chore: format tsup configs
`0ad4d22`	2026-10-04	fix(m10): keep role names plain on the Hire form (department as a hint); scope e2e agent lookup to the org
Security review fixes (6 commits)		
`0f342a0`	2026-10-04	fix(security): standing orders require project/org ownership (H1)
`4e1195e`	2026-10-04	fix(security): deactivating a user unlinks runners, removes memberships, parks agents (H3)
`f55662c`	2026-10-04	fix(security): validate owner-controlled refs, URLs and paths; confine runner paths; reject URL credentials; unique card refs and worktree paths (M1, M5, M7)
`e4608ec`	2026-10-04	fix(security): agent permissions may only narrow the role definition (M2)
`852ee29`	2026-10-04	fix(security): sandboxed, minimal-env agent sessions; per-role Claude permissions; generic writable-glob guard; roll back read-only roles (H2, M3, M4)
`d3fc133`	2026-10-04	fix(security): CSP/HSTS, chat re-checks, command ownership, leak check, rate limits, log redaction, lease-preserving rebuild, http(s) links, archived hire (M6, M8, low)
Real-Claude sandbox fixes (2 commits)		
`366f825`	2026-10-04	fix(runner): Claude sandbox keeps the agent's home, scratch and a private tmp writable
`dca3389`	2026-10-04	fix(runner): real Claude shifts complete in the sandbox
Windows/WSL2 default (2 commits)		
`a3881e6`	2026-10-04	feat: WSL2 is the documented Windows default; unsandboxed agents are a visible opt-in
`e2919d3`	2026-10-04	feat: server refuses unacknowledged unsandboxed hires; runtime changes refresh the web live
Outbox and runner fixes (2 commits)		
`42a9b9c`	2026-10-04	test(runner): stable project id per key in shift fixtures
`87f962d`	2026-10-04	fix(outbox): a command is delivered once per connection; a refused post is not replayed
Docs (2 commits)		
`faa0aae`	2026-10-04	docs: user guide, runner reference, deployment and env var reference
`217b453`	2026-10-04	docs: threat model, Claude sandbox settings, deactivation, permissions, headers
Other (test stability) (1 commits)		
`007de51`	2026-10-04	test: make the suites deterministic under load